free study notes

Phase 4
Distributed Systems & Messaging

42 lessons · notes, diagrams and 252 interview questions

1 · Watch
Scan the QR code on a lesson to open its video.

2 · Read
Notes go deeper than the video: worked examples, trade-offs, mistakes.

3 · Practise
Answer the interview questions out loud before reading the answers.

Distributed systems

1 · Distributed System Fundamentals	SD-H-001
2 · Partial Failures	SD-H-002
3 · Network Partitions	SD-H-003
4 · Failure Detection and Heartbeats	SD-H-004
5 · Clock Synchronization and Clock Skew	SD-H-005
6 · Logical Clocks (Lamport)	SD-H-006
7 · Vector Clocks and Hybrid Logical Clocks	SD-H-007
8 · Leader Election	SD-H-008
9 · Consensus and Raft	SD-H-009
10 · Paxos Overview	SD-H-010
11 · Quorums	SD-H-011
12 · Split-Brain Scenarios	SD-H-012
13 · Distributed Locks	SD-H-013
14 · Gossip Protocols	SD-H-014
15 · Distributed Transactions Overview	SD-H-015
16 · Two-Phase Commit (2PC)	SD-H-016
17 · Saga Pattern	SD-H-017
18 · Eventual Consistency in Practice	SD-H-018
19 · CRDTs	SD-H-019
20 · Idempotent Processing	SD-H-020
21 · Exactly-Once Processing: Claims and Limits	SD-H-021
22 · Linearizability and Serializability	SD-H-022

Messaging and events

23 · Why Messaging	SD-I-001
24 · Message Queues vs Event Streams	SD-I-002
25 · Producers and Consumers	SD-I-003
26 · Topics, Partitions and Offsets	SD-I-004
27 · Consumer Groups and Rebalancing	SD-I-005
28 · Kafka Replication and Durability	SD-I-006
29 · Ordering Guarantees	SD-I-007
30 · Delivery Semantics	SD-I-008
31 · Dead-Letter Queues	SD-I-009
32 · Retries and Delayed Messages	SD-I-010
33 · Backpressure in Messaging	SD-I-011
34 · Event Types: Notifications, State Transfer, Commands	SD-I-012
35 · Event Sourcing	SD-I-013
36 · CQRS	SD-I-014
37 · Transactional Outbox Pattern	SD-I-015
38 · Inbox Pattern and Deduplication	SD-I-016
39 · Schema Evolution and Registries	SD-I-017
40 · Event-Driven Microservices	SD-I-018
41 · Cloud Messaging Services	SD-I-019
42 · Pub/Sub and Fan-Out	SD-I-020

Free to share. Videos: youtube.com/@tapankumarbarik9454 · Notes and practice: backendroadmap.com

Distributed System Fundamentals

many machines that must act like one

Watch the 4-min video

In one line: a distributed system is several computers that cooperate over a network to look like one; you build one to scale, survive failures and get close to users, and you pay with partial failures, unreliable networks, unsynchronised clocks and harder consistency.

1 From one box to many

Your app runs on one server with one database. It's simple: one copy of the data, one clock, and a call either works or the whole thing crashes. Then the database fills up, one crash takes everything down, and users on another continent wait a second for every page. You add machines, and the app becomes a **distributed system**: several computers, connected only by a network, that must behave like one system for the user.

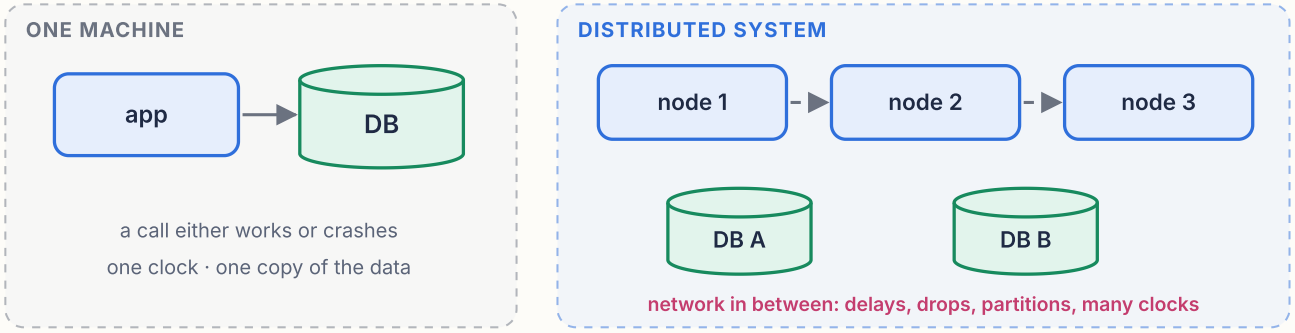


Figure 1. On one machine, failure is all or nothing. Across machines, the network sits between every pair of parts.

2 Why distribute at all

- **Scale:** one machine has a ceiling on CPU, memory, disk and connections. Many machines don't.
- **Availability:** if one machine dies, others keep serving. A single box is a single point of failure.
- **Latency:** put copies of the data near users, in several regions.
- **Organisation:** separate services let separate teams deploy independently.

3 What gets harder

Four new problems arrive the moment there are two machines, and the rest of Phase 4 deals with them one by one:

- **Partial failure** (lesson 2): one part fails while the rest keeps running, and you often can't tell a slow node from a dead one.
- **Unreliable network** (lesson 3): messages are delayed, lost, duplicated or reordered, and links can split the cluster in two.
- **No shared clock** (lessons 5 to 7): each machine's clock drifts, so timestamps can't safely order events.
- **Agreement** (lessons 8 to 13): several nodes must agree on a leader, a value or a lock, even while some of them fail.

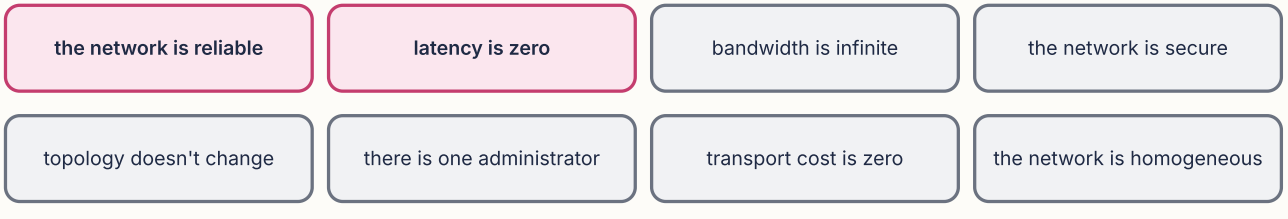


Figure 2. The eight fallacies: assumptions that are true on one machine and false across a network. The first two cause most outages.

4 Consistency: what can a reader see?

With several copies of the data, a reader might see an old copy. Systems choose a point on a scale: **strong consistency** (every read sees the latest write, at the cost of coordination and latency) or **eventual consistency** (copies converge if writes stop, and reads may be briefly stale). The CAP theorem says that during a network **partition** you must pick consistency or availability; PACELC adds that even without a partition, you trade latency for consistency.

WORKED EXAMPLE: FROM ONE DATABASE TO A REPLICATED CLUSTER

A monolith's single Postgres becomes a cluster: one **primary** in zone A takes all writes, and **replicas** in zones B and C copy its changes and serve reads. Now the app survives the loss of a zone and spreads read load. But it inherits new questions: replicas **lag** behind, so a user may not see their own update; someone must detect a dead primary and **promote** a replica; and if the old primary was only cut off, two nodes may both accept writes (split-brain, lesson 12).

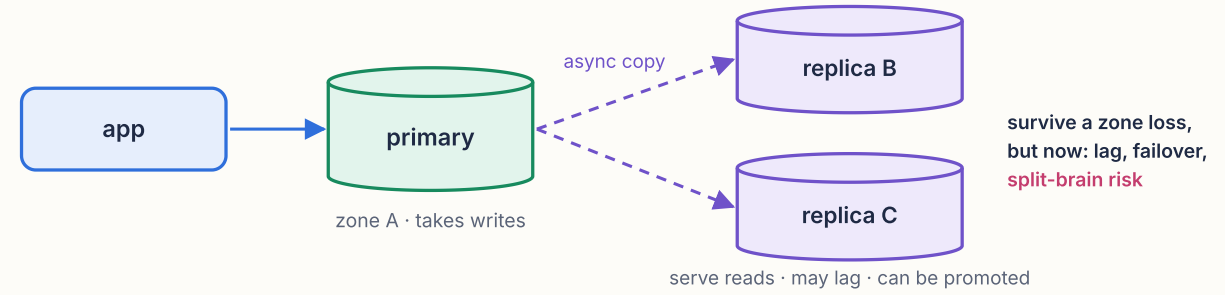


Figure 3. Replication buys availability and read scale, and brings lag, failover and split-brain with it.

5 How to think about a distributed design

For every arrow in your diagram, ask three questions: what if this call is **slow**, what if it **fails**, and what if it **succeeds twice**? Timeouts, retries with backoff, idempotency and replication are the everyday answers. Distribute only when you need to: a well-tuned single database goes a long way.

COMMON MISTAKES

- Treating a remote call like a local function call: no timeout, no retry plan.
- Ordering events across machines by their wall-clock timestamps.
- Splitting into microservices before the single system is actually a bottleneck.
- Assuming replicas are always up to date right after a write.

6 Key terms

Distributed system: several computers cooperating over a network to act as one

Partial failure: some parts fail while others keep working

Partition: a network split: nodes are up but can't reach each other

Eventual consistency: copies converge over time; reads may be stale

Node: one machine or process in the system

Replication: keeping copies of data on several nodes

Strong consistency: every read sees the latest write

CAP / PACELC: consistency vs availability under partition; latency vs consistency otherwise

7 Interview questions

Q1. What is a distributed system, and why build one? BASIC
Several computers that cooperate over a network and appear as one system. You build one to scale past one machine, to stay available when a machine fails, and to serve users from nearby regions.

Q2. What new problems appear once you have more than one machine? BASIC
Partial failures, an unreliable network (delays, loss, partitions), no shared clock to order events, and the need for nodes to agree on things like a leader or the latest value.

Q3. Your single Postgres is at its limit. What do you do before going distributed? SCENARIO
Measure first: add indexes, fix slow queries, add a cache and read replicas, scale the machine up. Shard or split only when one primary truly can't take the write load.

Q4. You add read replicas and a user doesn't see the profile change they just saved. Why, and how do you fix it? SCENARIO
Replication lag: the read hit a replica that hadn't applied the write yet. Read your own writes from the primary for a short window, or route the user to a replica that has caught up to their write position.

Q5. Is a system with one app server and one managed database distributed? TRICKY
Yes, in the sense that matters: the app and database talk over a network, so calls can time out, fail half-way or be retried, and you must handle that. Managed services hide some, not all, of it.

Q6. What does CAP really force you to choose? TRICKY
Only during a partition: either reject some requests to stay consistent, or answer from both sides and risk divergence. When the network is fine you can have both, but you still trade latency for consistency (PACELC).

Next: SD-H-002 · Partial Failures

Partial Failures

some parts fail while the rest keeps running

Watch the 4-min video

In one line: in a distributed system some components fail while others keep running, and a caller often can't tell a lost request from a slow or crashed service, so bound every call with a timeout, retry safely, and isolate failures so one bad dependency can't take everything down.

1 Not all or nothing anymore

On one machine, things usually work or crash completely. In a distributed system, **some parts fail while others keep running**. A request may succeed in three services and fail in the fourth. A database may accept a write and then lose the connection before replying. This is a **partial failure**, and it's the normal state of a large system, not a rare event: with thousands of machines, something is always broken somewhere.

2 The core problem: a timeout tells you very little

When a call doesn't answer in time, the caller can't know what happened. The request may have been lost, the service may have crashed mid-work, it may just be slow, or it may have **done the work** and lost the reply. The last case is the dangerous one: if you retry, you may charge a card twice. That's why retries need **idempotency** (lesson 20).

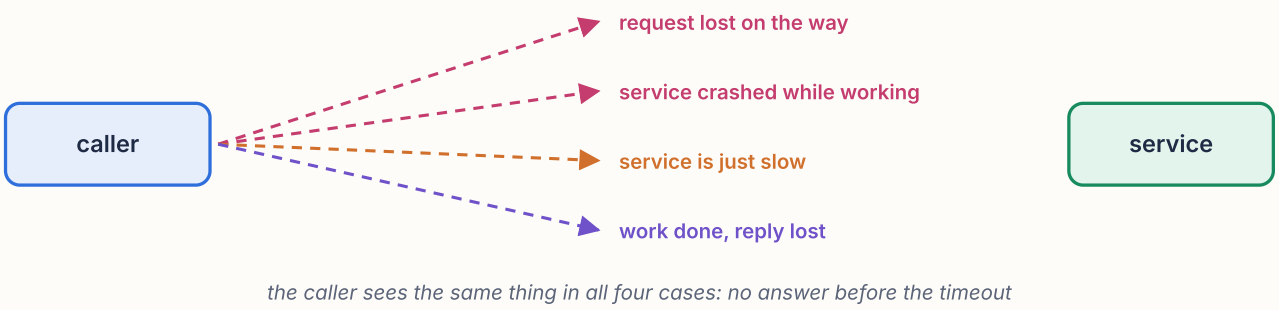


Figure 1. Four different failures, one symptom. The caller only knows that no reply arrived before its deadline.

3 Slow is worse than dead

A dead service fails fast: connections are refused and callers move on. A **slow** service holds every caller's thread, connection and memory while it thinks. Those resources run out, new requests queue up, and the caller fails too. This chain is a **cascading failure**.

WORKED EXAMPLE: A 30-SECOND DEPENDENCY WITH NO TIMEOUT

A downstream service doesn't fail; it starts responding in 30 seconds instead of 50 ms. Your API calls it with no timeout, and each waiting request holds one of 200 worker threads. At 100 requests per second, all 200 threads are stuck within two seconds. Now even endpoints that never touch that dependency can't get a thread, and your whole API is down. Fix: a 1-second timeout on the call, a circuit breaker that opens after repeated failures, a separate thread pool (bulkhead) for that dependency, and a fallback response.

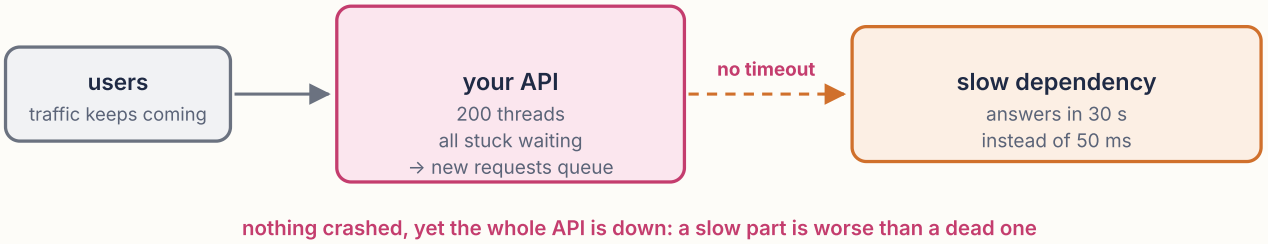


Figure 2. A slow dependency without a timeout ties up every thread, and the failure spreads upstream.

4 The defences

- **Timeouts** on every network call, set from the dependency's real p99, not the default (often infinite).
- **Retries** only for safe (idempotent) operations, with exponential backoff and jitter, and a small retry budget so retries don't double the load.
- **Circuit breakers**: after many failures, stop calling for a while and fail fast, then test with a few requests.
- **Bulkheads**: separate connection or thread pools per dependency, so one can't starve the others.
- **Fallbacks** and graceful degradation: serve cached data, a default, or hide a non-critical feature.

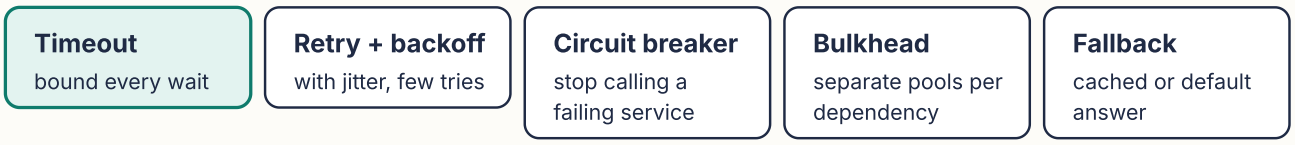


Figure 3. The standard toolkit against partial failure. A timeout comes first: without it, none of the others can act.

5 Design for it

Assume every dependency will be slow, down or wrong at some point. Decide in advance what each feature does then: block, degrade or skip. Test it on purpose with fault injection (chaos testing), because the failure paths are the code paths that never run in a demo.

COMMON MISTAKES

- No timeout, or a timeout longer than the caller's own deadline.
- Retrying non-idempotent calls, like a payment, without an idempotency key.
- Retrying at every layer: 3 retries × 3 layers = 27 calls per user request.
- One shared connection pool for all dependencies.

6 Key terms

Partial failure: some components fail while others keep running

Cascading failure: one failure spreads to callers as their resources run out

Bulkhead: isolated resource pools per dependency

Timeout: maximum time to wait for a reply

Circuit breaker: stops calls to a failing dependency for a while

Graceful degradation: keep core features working when parts fail

7 Interview questions

Q1. What is a partial failure? BASIC

A failure of some components while others keep working, for example one service or one replica down, or a reply lost on the network. The system as a whole is still running, but some requests fail or hang.

Q2. Why is a timeout ambiguous? BASIC

The request may have been lost, the server may have crashed, may be slow, or may have finished the work and lost the reply. The caller can't tell which, so a retry must be safe to repeat.

Q3. One downstream service gets slow and your whole API goes down. Explain and fix. SCENARIO

Calls without timeouts hold threads until the pool is exhausted, so every endpoint stalls. Add timeouts, a circuit breaker, a bulkhead pool for that dependency, and a fallback.

Q4. How do you choose a timeout value? SCENARIO

From the dependency's latency distribution, a bit above its p99, and always below the caller's own deadline. Then pass the remaining deadline down the chain so inner calls don't outlive the request.

Q5. Retries make an outage worse. How? TRICKY

A struggling service gets extra load from every client retrying at once. Use exponential backoff with jitter, cap attempts, use a retry budget, and stop retrying when the circuit breaker is open.

Q6. Is a dead service or a slow service worse for callers? TRICKY

Usually a slow one: dead services fail fast, while slow ones hold the caller's threads and connections, causing cascading failures.

Next: SD-H-003 · Network Partitions

Network Partitions

both sides alive, neither can reach the other

Watch the 4-min video

In one line: a network partition splits nodes into groups that are all running but can't talk to each other; during one you must choose between consistency (only a majority side serves writes) and availability (every side serves and copies diverge), and you should decide that before it happens.

1 What a partition is

Two data centers run one database cluster. The link between them fails. Both sides are healthy and both can still serve their users, but they can't reach each other. That's a **network partition**: the cluster is split into groups that can only talk within themselves. Causes are ordinary: a cut fibre, a misconfigured switch or firewall rule, a cloud networking incident, or a node so overloaded or paused (GC) that it looks unreachable.

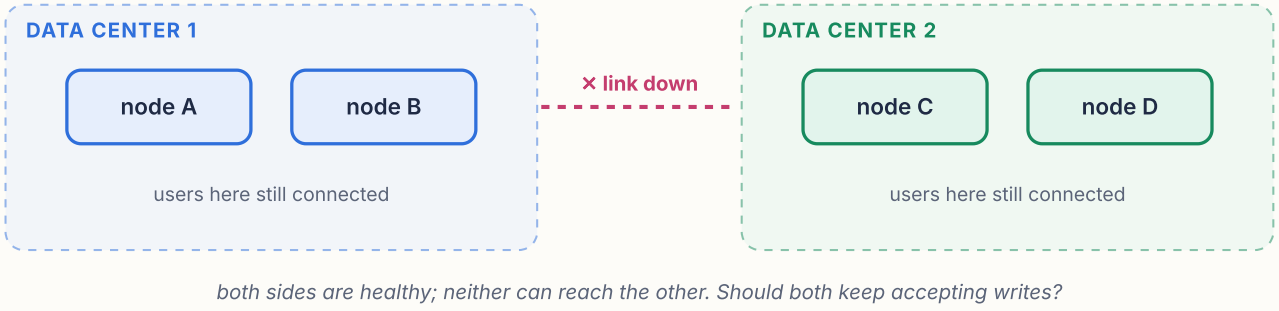


Figure 1. A partition: every node is up, but the link between the two groups is gone.

2 From inside, a partition looks like failure

Node A stops hearing from node C. Did C crash, or is the path to C broken? From A's side it's impossible to tell. Both cases look like silence. So systems must make the same safe choice either way, usually by counting how many nodes they can still reach.

3 The choice: consistency or availability

The **CAP theorem** says that while a partition lasts, a system that replicates data can't be both **consistent** (every read sees the latest write) and **available** (every non-failed node answers). If both sides accept writes to the same data, the copies diverge. If only one side may write, the other side must refuse.

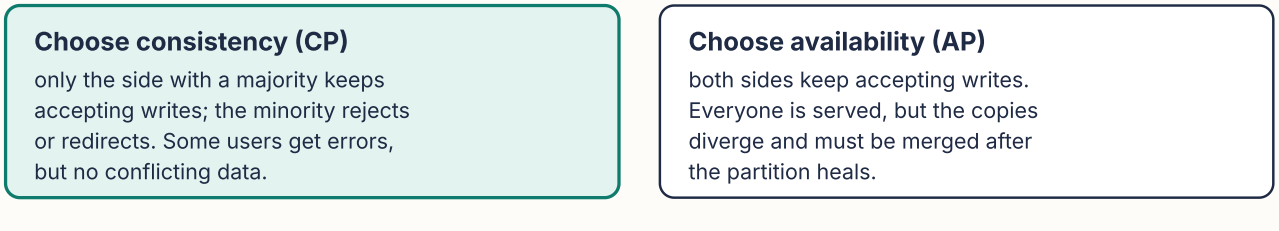


Figure 2. The two options during a partition. Banks and lock services choose CP; shopping carts and like counters often choose AP.

CP systems (etcd, ZooKeeper, Spanner, most leader-based databases) let only the side holding a **majority** (a quorum) keep the leader. **AP systems** (Dynamo-style stores like Cassandra at low consistency levels) keep both sides writable and reconcile later with last-write-wins, vector clocks or CRDTs (lessons 7 and 19).

WORKED EXAMPLE: AN AVAILABILITY-ZONE LINK FAILS

A link failure separates the database primary in zone A from its replicas in zones B and C. B and C can still see each other: together they are a majority of three, so they elect B as the new primary and the app in B and C keeps writing. The old primary in zone A is alone, a minority, so it must **step down** and reject writes. If it kept accepting writes from clients in zone A, there would be two primaries and conflicting data: split-brain (lesson 12). When the link heals, A rejoins as a replica of B.

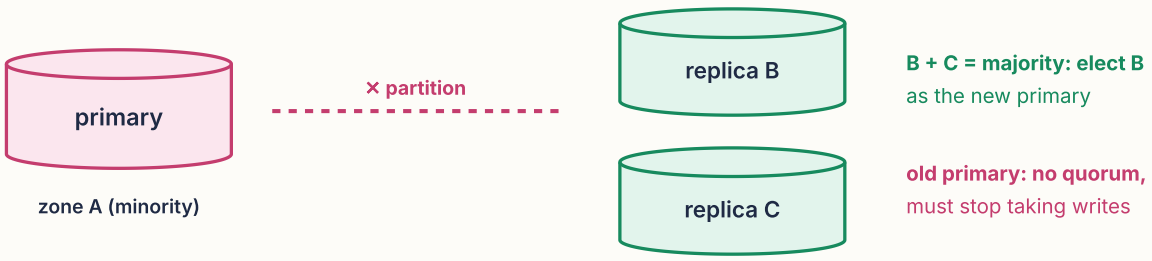


Figure 3. The majority side elects a new primary; the isolated minority must stop accepting writes.

4 Healing and testing

When the partition heals, AP systems must **merge** divergent writes, and CP systems must bring the minority back up to date. Partitions also happen partially (A sees B, B sees C, A can't see C) and asymmetrically (messages flow one way). Test them deliberately with tools that drop traffic between nodes, as Jepsen-style testing does.

COMMON MISTAKES

- Assuming partitions are too rare to plan for.
- Two-node clusters: neither side can ever have a majority.
- Letting an isolated primary keep taking writes because it 'is still healthy'.
- Choosing AP without a plan to merge conflicting writes.

5 Key terms

Network partition: nodes are up but split into groups that can't communicate

CP: stays consistent during a partition, refusing some requests

Split-brain: two sides each believe they are the leader

Quorum / majority: more than half of the nodes

AP: stays available during a partition, allowing divergence

Heal: the partition ends and nodes reconnect

6 Interview questions

Q1. What is a network partition? BASIC

A failure where nodes keep running but some groups can't reach each other, so the cluster is split. It looks the same as remote nodes crashing.

Q2. What does CAP say? BASIC

During a partition, a replicated system must choose: stay consistent by refusing some requests, or stay available and let copies diverge. Outside partitions, it can provide both.

Q3. Your primary is cut off from its two replicas. What should happen? SCENARIO

The replicas, a majority, elect a new primary. The isolated primary, with no quorum, must stop accepting writes; fencing makes sure its writes are rejected if it tries.

Q4. A shopping cart service and a bank ledger: CP or AP? SCENARIO

The cart can be AP: accept adds on both sides and merge them later. The ledger should be CP: refusing a transfer briefly is better than double-spending.

Q5. Why do clusters use an odd number of nodes? TRICKY

To make a majority possible on one side of a split. With 3 or 5 nodes, one side always has more than half; with 2 or 4, an even split leaves no majority, so nothing can write.

Q6. Can a node tell a partition from a crash? TRICKY

No. Both look like missing messages. That's why decisions are based on quorums and leases, not on guessing what happened to the silent node.

Next: SD-H-004 · Failure Detection and Heartbeats backendroadmap.com

Failure Detection and Heartbeats

how a cluster notices a dead node

Watch the 4-min video

In one line: nodes send periodic heartbeats and a node that stays silent past a timeout is suspected dead; there's no perfect detector, so you trade detection speed against false alarms, often with adaptive detectors like phi accrual and a suspicion phase before acting.

1 The question every cluster must answer

In a cluster of fifty nodes, one dies at 3 a.m. How fast do the others notice? Too slow, and requests keep going to a dead node. Too fast, and a healthy node that just paused for garbage collection gets thrown out, triggering a failover nobody needed. A **failure detector** is the component that makes this call.

2 Heartbeats and timeouts

The simplest detector: each node sends a small **heartbeat** message at a fixed interval (say every second). If a peer hears nothing for longer than a **timeout** (say 3 seconds), it marks that node as suspected dead. The timeout is a guess: the node may be dead, slow, paused, or cut off by the network (lesson 3).

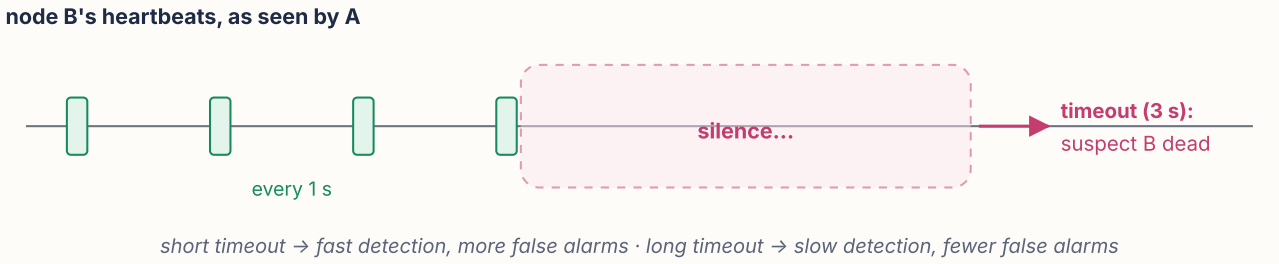


Figure 1. After a few missed heartbeats, the silent node is suspected. The timeout length is the key trade-off.

3 No perfect detector

A detector is judged on two properties: **completeness** (every crashed node is eventually suspected) and **accuracy** (healthy nodes aren't wrongly suspected). In a real network with unbounded delays you can't have both perfectly; you can only tune the balance. That's why actions after detection must be safe even when the detector is wrong.

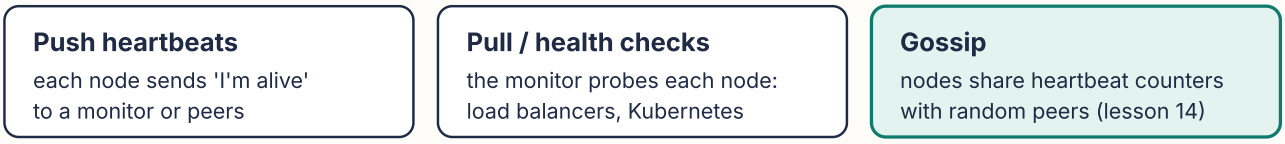


Figure 2. Three ways to collect liveness information. Large clusters prefer gossip because no single monitor sees everything.

4 Adaptive detection: phi accrual

A fixed timeout ignores how the network behaves. The **phi accrual** detector keeps a window of recent heartbeat intervals, and computes a suspicion level **φ** (phi) that grows the longer the silence lasts compared with what's normal for that node. Above a threshold (Cassandra's default is 8), the node is marked down. On a jittery link, φ grows slowly; on a steady one, a short silence is already suspicious.

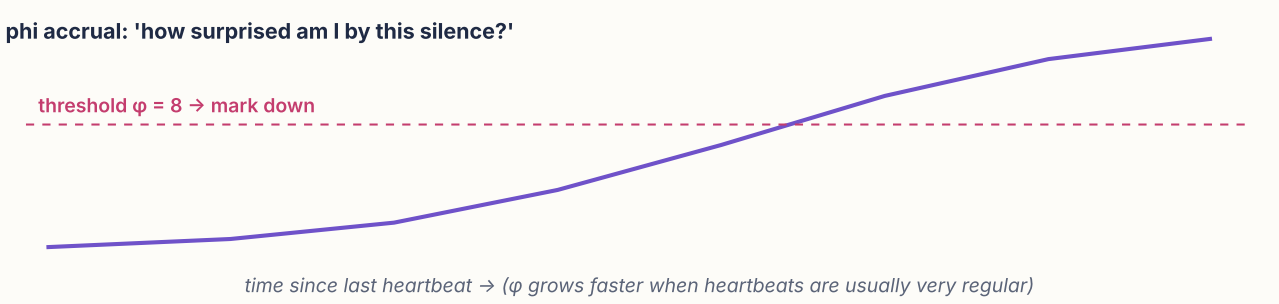


Figure 3. φ rises with the length of the silence; crossing the threshold marks the node down.

WORKED EXAMPLE: CASSANDRA

Cassandra nodes gossip every second, exchanging heartbeat counters for every node they know about. Each node keeps a window of heartbeat arrival intervals for every peer. When node 7's heartbeats stop, φ for node 7 climbs on each observer; past 8, they mark it down locally, stop sending it reads, and store **hints** for writes it misses. If node 7 was only paused by a long GC, its heartbeats return, φ falls, and it's marked up again, with no failover drama, because Cassandra has no single leader to replace.

5 Making detection safe

- Use a **suspicion phase** (SWIM, Consul): ask other nodes to probe the suspect before declaring it dead.
- Make the cost of a false positive small: a node marked dead should be able to rejoin cleanly.
- For leaders, combine detection with **leases and fencing** (lessons 8 and 13) so a wrongly-suspected leader can't do harm.
- Separate **liveness** (is it running?) from **readiness** (should it get traffic?).

COMMON MISTAKES

- Timeouts shorter than a normal GC pause or deploy restart.
- Heartbeats on the same congested path as data, so load looks like death.
- Failing over immediately on one missed heartbeat.
- A single central monitor that itself can fail or be partitioned.

6 Key terms

Heartbeat: periodic 'I'm alive' message

Completeness: crashed nodes are eventually suspected

Phi accrual: adaptive detector giving a suspicion level, not yes/no

Failure detector: decides which nodes are suspected dead

Accuracy: healthy nodes are not suspected

SWIM: gossip-based membership with indirect probes

7 Interview questions

Q1. How do heartbeats detect failure? BASIC

Each node sends periodic heartbeats; if one isn't heard for longer than a timeout, it is suspected dead. The timeout trades detection speed for false alarms.

Q2. Why can't failure detection be perfect? BASIC

Network delays have no fixed upper bound, so a slow or partitioned node looks exactly like a dead one. You can only balance speed against wrong suspicions.

Q3. A healthy node keeps getting marked dead during GC pauses. What do you change? SCENARIO

Raise the timeout above the worst normal pause, use an adaptive detector like phi accrual, add indirect probes before declaring death, and tune GC to shorten pauses.

Q4. How fast should a database cluster fail over? SCENARIO

Fast enough for the availability target, slow enough to ride out normal blips: typically a few to tens of seconds, plus fencing so a wrongly-replaced leader can't keep writing.

Q5. What is the risk of a too-aggressive failure detector? TRICKY

False positives trigger needless failovers and rebalancing, which add load and can cause more false positives: a flapping cluster. In leader systems it can create two leaders without fencing.

Q6. Why do large clusters use gossip for failure detection? TRICKY

There is no single monitor to overload or partition; every node learns about failures from random peers in O(log n) rounds, and indirect probing reduces false alarms.

Next: SD-H-005 · Clock Synchronization and Clock Skew backendroadmap.com

Clock Synchronization and Clock Skew

why timestamps can't order events across machines

Watch the 4-min video

In one line: every machine's clock drifts, and NTP only keeps them within milliseconds of each other, so timestamps from different servers can't reliably order events; use monotonic clocks for durations, and logical clocks, version numbers or bounded-uncertainty clocks when order matters.

1 Later isn't always later

Two servers write to the same record a few milliseconds apart, and you keep the one with the later timestamp. Simple, right? But if the second server's clock is 50 ms behind, the newer write carries the older timestamp and is silently thrown away. Across machines, **timestamps lie** by small, unpredictable amounts.

2 Why clocks disagree

Each server counts time with a quartz crystal whose frequency depends on temperature, so clocks **drift** by roughly tens of microseconds per second. The difference between two clocks at a moment is called **clock skew**. **NTP** (Network Time Protocol) periodically corrects each clock against time servers, but network delays limit its accuracy to about a millisecond on a good LAN, and tens of milliseconds over the internet. Corrections can also make a clock **jump backwards**, and leap seconds add their own surprises.

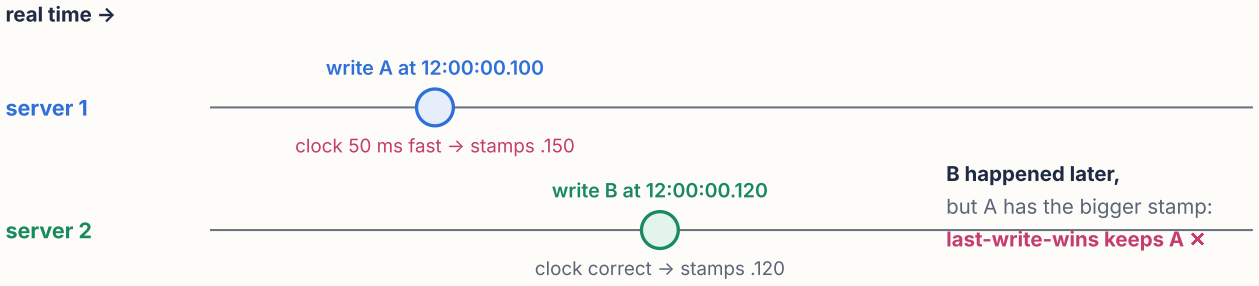


Figure 1. A 50 ms skew is enough to reverse the order of two writes when you compare timestamps.

WORKED EXAMPLE: LAST-WRITE-WINS LOSES AN UPDATE

A key-value store resolves conflicts with **last-write-wins** (LWW): the write with the highest timestamp wins. A user updates their address on server 1, whose clock runs 50 ms fast, then 20 ms later fixes a typo through server 2, whose clock is correct. The typo fix carries the smaller timestamp, so every replica keeps the version with the typo, and nothing reports an error. Fixes: use a version number or compare-and-set on the record, route a user's writes to one leader, or use logical or hybrid clocks (lessons 6 and 7).

3 Two kinds of clock

Operating systems offer a **wall clock** (time of day, e.g. `time.time()`) and a **monotonic clock** (e.g. `time.monotonic()`). The monotonic clock never goes backwards, which makes it the right choice for timeouts, rate limits and latency measurement. It means nothing across machines, though.

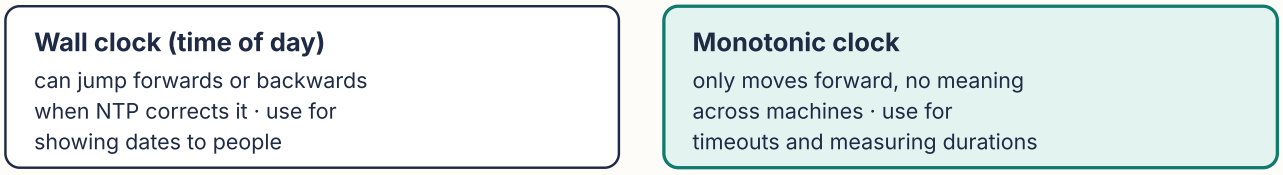


Figure 2. Use the monotonic clock to measure; use the wall clock only to display or log a date.

4 When you do need real time: bounded uncertainty

Google Spanner's **TrueTime** uses GPS receivers and atomic clocks in every data center and returns an interval, not a point: the true time is somewhere between *earliest* and *latest*, typically a few milliseconds apart. Before making a commit visible, Spanner **waits out** the uncertainty, so commit timestamps respect real-time order. AWS Time Sync and PTP give similar tight bounds. Without hardware like this, assume skew of at least several milliseconds.



Figure 3. TrueTime admits its uncertainty and waits it out, which makes timestamp order safe.

5 Practical rules

- Never use wall-clock timestamps from different machines to decide which write wins, unless losing a write is acceptable.
- Generate order at one place: a leader, a database sequence, or a log offset.
- Store timestamps in UTC, and monitor NTP offset on every host; alert if it exceeds a few milliseconds.
- Leases and lock expiry must allow for skew and pauses (lesson 13).

COMMON MISTAKES

- Measuring a timeout with `time.time()`, which can jump.
- Sorting events from many services by their local timestamps and trusting the order.
- Assuming NTP means 'synchronised exactly'.
- Last-write-wins on data where a lost update matters.

6 Key terms

Clock drift: a clock running slightly fast or slow

Clock skew: difference between two clocks at the same moment

NTP: protocol that corrects clocks against time servers

Monotonic clock: never goes backwards; for durations

Last-write-wins: keep the write with the highest timestamp

TrueTime: Spanner's clock API with an explicit uncertainty bound

7 Interview questions

Q1. Why can't you order events across servers by timestamp? BASIC

Clocks drift and are only roughly synchronised by NTP, so skew of milliseconds or more is normal; an event that happened later can carry an earlier timestamp.

Q2. Monotonic or wall clock for a timeout? BASIC

Monotonic: it never jumps backwards when NTP corrects the time, so durations are measured correctly.

Q3. Users report that some profile updates disappear. The store uses last-write-wins across regions. Explain. SCENARIO

Clock skew between regions gives a newer write an older timestamp, so it loses the conflict. Use versioning with compare-and-set, a single leader per record, or merge-friendly data types.

Q4. How would you order events from many microservices for an audit log? SCENARIO

Don't rely on their clocks: write events to one partitioned log (a Kafka partition per entity) and use the log offset as the order, or use logical or hybrid timestamps.

Q5. How does Spanner get externally consistent timestamps? TRICKY

TrueTime returns an interval with bounded uncertainty from GPS and atomic clocks; Spanner waits until the commit timestamp is certainly in the past before making the commit visible.

Q6. Can a clock go backwards on a server? What breaks? TRICKY

Yes: NTP can step it back, or a VM can be paused and resumed. Durations measured with the wall clock go negative, TTLs and leases misbehave, and IDs based on time can collide.

Next: SD-H-006 · Logical Clocks (Lamport)

Logical Clocks (Lamport)

ordering by cause, not by the time of day

Watch the 4-min video

In one line: a Lamport clock is a counter on each node, incremented on every event and carried on every message, so if event a could have caused b, then a's timestamp is smaller than b's; ties broken by node ID give a total order every replica agrees on, but timestamps alone can't tell you whether two events were concurrent.

1 Order without real time

If wall clocks can't order events across machines (lesson 5), what can? In 1978 Leslie Lamport showed that you don't need real time at all. What matters is **causality**: which events could have influenced which.

2 Happens-before

Event *a* **happens before** event *b* (written a → b) if: both are on the same process and a comes first; or a is the sending of a message and b is its receipt; or there's a chain of such steps from a to b. If neither a → b nor b → a, the events are **concurrent**: neither could have known about the other.

3 The Lamport clock

Each process keeps one integer counter. It increments the counter for each local event, attaches the counter to each message it sends, and on receiving a message sets its counter to `max(local, received) + 1`. Result: **if a → b, then L(a) < L(b)**. The converse is not true: a smaller timestamp doesn't prove causality.



Figure 1. Three rules are the whole algorithm.

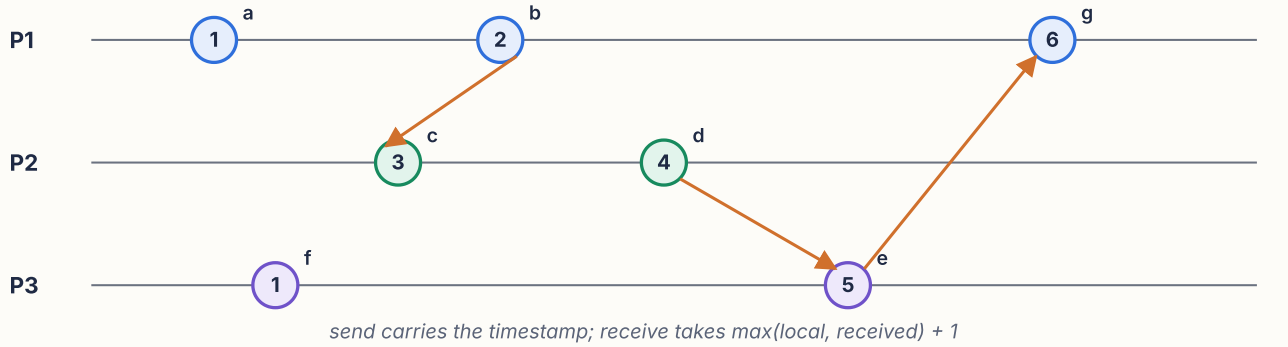


Figure 2. Lamport timestamps on three processes. a → b → c → d → e → g, so their numbers increase. f (timestamp 1) is concurrent with all of P1's and P2's early events.

4 From partial order to total order

Two events can share a timestamp. Break ties with the process ID, ordering by the pair **(timestamp, node ID)**. Now every pair of events has a definite order, and every node that sorts the same events gets the **same sequence**. That order respects causality, though for concurrent events it is arbitrary.

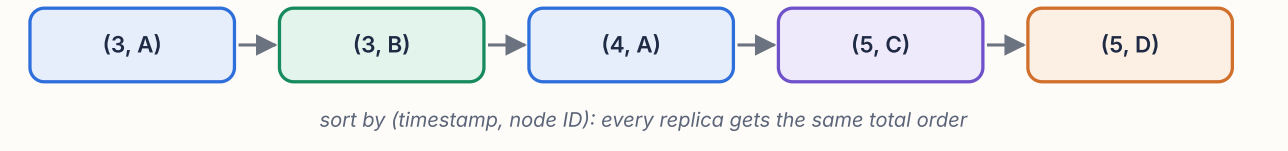


Figure 3. Pairs (timestamp, node) give one sequence all replicas agree on.

WORKED EXAMPLE: ORDERING REPLICATED LOG ENTRIES

Three replicas accept operations and exchange them. Each entry carries a Lamport timestamp and the ID of the replica that created it. Replica A creates `set x=1` at (3, A); replica B, which had already received A's earlier messages, creates `set x=2` at (4, B). Every replica applies entries sorted by (timestamp, ID), so all end with x=2, and an operation that depends on another is always applied after it. Lamport's paper used exactly this idea to build a mutual-exclusion algorithm with a shared request queue.

5 Limits

- Lamport timestamps can't detect **concurrency**: L(a) < L(b) might mean a caused b, or that they were unrelated. For that you need vector clocks (lesson 7).
- They have no link to real time, so 'what happened at 3 p.m.?' can't be answered.
- The total order is fixed only after you've heard from everyone, so it alone doesn't give fault-tolerant agreement; that needs consensus (lesson 9).

COMMON MISTAKES

- Concluding 'a caused b' from L(a) < L(b).
- Forgetting to update the counter on receive, which breaks the guarantee.
- Comparing timestamps without a tie-breaker, so replicas disagree on equal values.

6 Key terms

Happens-before (→): a could have influenced b

Lamport clock: per-node counter carried on messages

Tie-breaker: node ID used when timestamps are equal

Concurrent events: neither happened before the other

Total order: every pair of events is ordered the same way everywhere

7 Interview questions

Q1. What guarantee does a Lamport clock give? BASIC

If event a happens before b, then a's timestamp is smaller than b's. It does not guarantee the reverse.

Q2. How do you update a Lamport clock on receive? BASIC

Set it to the maximum of the local counter and the timestamp in the message, then add one.

Q3. Three replicas must apply the same operations in the same order. How can Lamport clocks help? SCENARIO

Stamp each operation with (Lamport time, replica ID) and apply in sorted order; causally related operations stay in order and all replicas agree on the rest.

Q4. You need to know whether two edits conflicted. Are Lamport timestamps enough? SCENARIO

No: they can't distinguish concurrent edits from causally ordered ones. Use version vectors or vector clocks to detect concurrency.

Q5. Why not just use synchronised physical clocks? TRICKY

Skew means a causally later event can get an earlier timestamp. Lamport clocks cost one integer and never violate causality.

Q6. What's the 'clock condition'? TRICKY

For any events a → b, C(a) < C(b). Lamport clocks satisfy it; vector clocks satisfy the stronger version where the converse also holds.

Next: SD-H-007 · Vector Clocks and Hybrid Logical Clocks backendroadmap.com

Vector Clocks and Hybrid Logical Clocks

detecting concurrent updates

Watch the 4-min video

In one line: a vector clock keeps one counter per node, so comparing two versions tells you whether one came after the other or whether they were concurrent and conflict; hybrid logical clocks combine physical time with a counter to get timestamps close to real time that still respect causality.

1 The disappearing cart item

Two users add items to the same shopping cart, on two different replicas, at the same moment. Neither update saw the other. With last-write-wins, one item silently disappears. To merge correctly, the system must first **know** that the two versions were concurrent. Lamport clocks (lesson 6) can't tell that; vector clocks can.

2 How a vector clock works

Each version of a value carries a **vector**: one counter per node, like `[X:2, Y:1]`. When node X updates the value, it increments its own entry. When versions meet, compare them entry by entry:



Figure 1. Comparing two vector clocks gives one of three answers. The third one is the conflict a Lamport clock can't see.

When a node receives a version, it takes the element-wise **maximum** with what it has. So **a → b if and only if V(a) < V(b)**: vector clocks capture causality exactly.

WORKED EXAMPLE: DYNAMO'S SHOPPING CART

Replica X stores a cart with a book at version `[X:1]`. During a partition, one session adds headphones on X, giving `[X:2]`. Another session reads the old cart and adds a charger on replica Y, giving `[X:1, Y:1]`. When the partition heals, neither vector is greater: X has more for X, Y has more for Y. So the store keeps both as **siblings** and returns both on the next read. The cart logic merges them by union, and writes `[book, headphones, charger]` at `[X:2, Y:1]`, which now dominates both. (A union can bring back a deleted item; Dynamo accepted that trade-off.)

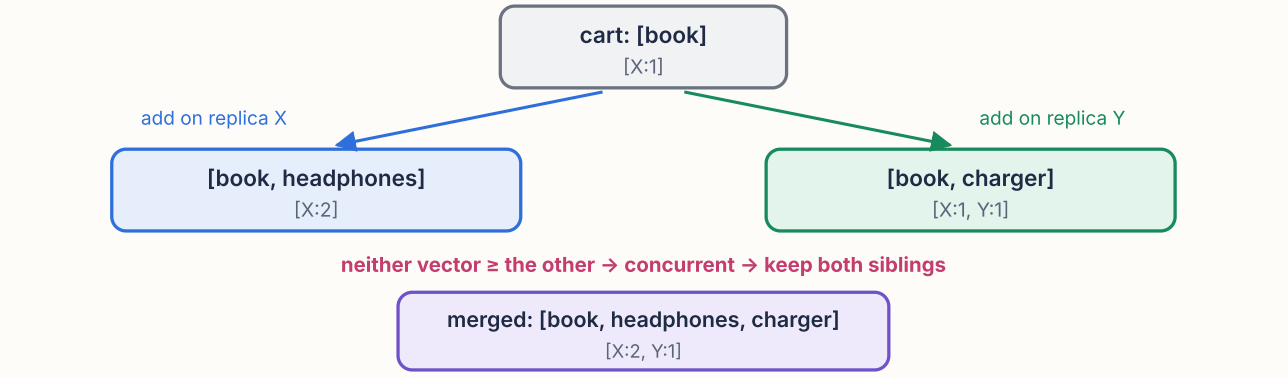


Figure 2. Concurrent versions are detected and kept, then merged by the application.

3 Costs and variants

The vector grows with the number of nodes (or clients) that write. Real systems use **version vectors** per replica rather than per client, prune old entries, or use **dotted version vectors** (Riak) to avoid false conflicts. Many stores give up and use last-write-wins for simplicity; that's fine only when losing a concurrent write is acceptable.

4 Hybrid logical clocks

Sometimes you want timestamps that are **close to real time** (for 'as of' reads, TTLs and debugging) and still respect causality, without TrueTime hardware. A **hybrid logical clock** (HLC) is a pair: the largest physical time seen so far, plus a small logical counter that breaks ties and covers clock lag. On send or receive it takes the max of local physical time and the received HLC, bumping the counter when physical time hasn't moved. CockroachDB and YugabyteDB use HLCs together with a maximum allowed clock offset.

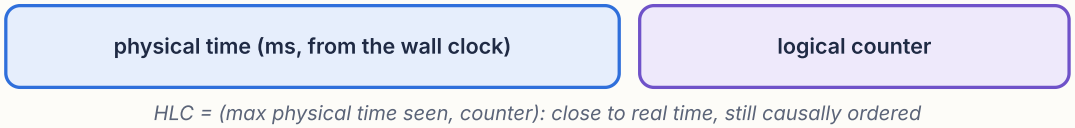


Figure 3. An HLC fits in 64 bits and can be compared like a timestamp.

COMMON MISTAKES

- Using last-write-wins where concurrent writes must both survive.
- Per-client vector entries that grow without bound.
- Detecting conflicts but having no merge rule for the application.
- Treating an HLC as exact physical time.

5 Key terms

Vector clock: one counter per node; detects concurrency

Siblings: concurrent versions kept until merged

Hybrid logical clock: physical time + logical counter

Version vector: vector clock attached to data versions per replica

Dominates: every entry ≥ and at least one >

6 Interview questions

Q1. What can vector clocks tell you that Lamport clocks can't? BASIC

Whether two events were concurrent. Comparing vectors shows if one happened before the other or neither did.

Q2. How do you compare two vector clocks? BASIC

If every entry of A is ≤ B's (and they differ), A happened before B. If it's the other way round, B happened before A. Otherwise they're concurrent.

Q3. Two replicas accepted writes to the same key during a partition. How do you avoid losing one? SCENARIO

Track versions with vector clocks, detect that they're concurrent, keep both as siblings, and merge them with an application rule or a CRDT.

Q4. Your vector clocks are growing huge. Why, and what do you do? SCENARIO

Entries are per client or per short-lived node. Use per-replica version vectors, prune old entries, or switch to dotted version vectors.

Q5. Why do CockroachDB and YugabyteDB use HLCs instead of vector clocks? TRICKY

They need one compact, comparable timestamp per transaction that's close to real time for MVCC and 'as of' reads; HLCs give that while respecting causality, given a bound on clock offset.

Q6. Does a vector clock merge the data for you? TRICKY

No. It only tells you there's a conflict. The application or a CRDT has to decide the merged value.

Next: SD-H-008 · Leader Election

Leader Election

making sure exactly one node is in charge

Watch the 4-min video

In one line: leader election picks exactly one node to do a job or accept writes; do it with a lease in a consensus-backed store like etcd or ZooKeeper, renew it while you work, and use fencing tokens so a leader that lost its lease without knowing can't do damage.

1 Three instances, three charges

Your system runs a nightly billing job. For availability you deploy three instances. Now all three run the job, and every customer is charged three times. You need **exactly one** active instance, and if it dies, another must take over automatically. That's **leader election**.

2 Where you need a leader

- **Singleton jobs:** schedulers, cron-like tasks, cleanup workers.
- **Single-writer replication:** one primary database accepts writes (lesson 1).
- **Coordination:** Kafka's controller, Kubernetes controllers, a partition's leader.

Lease in a CP store
etcd, ZooKeeper, Consul, Kubernetes Lease object

Built-in consensus
Raft or Paxos inside the system (lesson 9)

Bully / ring algorithms
classic textbook methods; unsafe under partitions

Figure 1. Ways to elect a leader. In practice, use a lease in a store that already runs consensus.

3 Leases: leadership with an expiry

A **lease** is leadership for a limited time. Candidates try to create a key in a strongly consistent store (etcd, ZooKeeper, Consul); the store guarantees only one succeeds. The winner must **renew** the lease before it expires (say, every 3 s for a 10 s lease). If it crashes or is cut off, the lease expires and the key is deleted; the others, which **watch** the key, race to become the next leader. Because the store itself runs Raft or Paxos, it never hands out two leases at once.

WORKED EXAMPLE: A BILLING SCHEDULER WITH ETCD

Three scheduler instances campaign through an etcd election with a 10-second lease. Instance 2 wins and runs the billing job, renewing its lease every few seconds. Instances 1 and 3 block on the election, watching the key. At 2:13 a.m. instance 2's machine dies. Its lease expires 10 s later, etcd deletes the key, instance 3 wins the next campaign and resumes the job from the last checkpoint. In Kubernetes, the same pattern uses a `Lease` object (client-go's leaderelection).

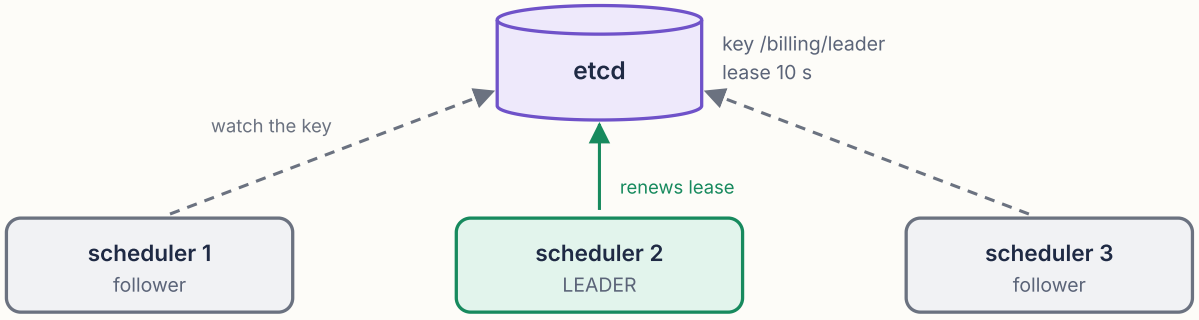


Figure 2. The leader holds a lease and renews it; followers watch the key and campaign when it disappears.

4 The zombie leader problem

A leader can be paused (a long GC, a frozen VM) past its lease. Everyone else has elected a new leader, but the old one wakes up still **believing** it's in charge and finishes its write. Checking the lease before writing doesn't help, because the pause can happen right after the check. The fix is a **fencing token**: a number that increases with every new lease (etcd's revision, ZooKeeper's zxid). The leader sends it with each write, and the storage rejects tokens older than the newest it has seen.

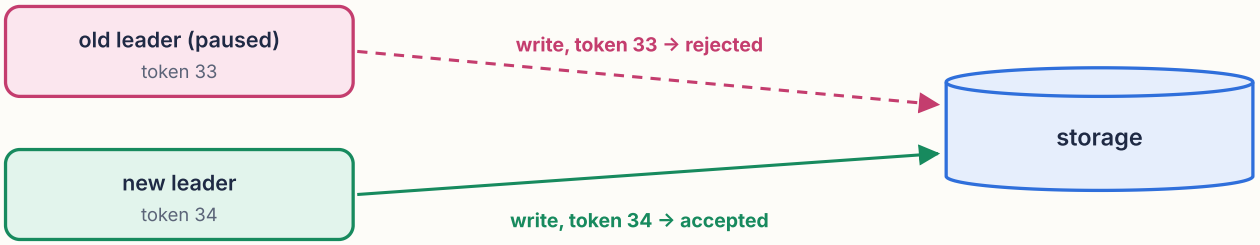


Figure 3. Fencing: the storage, not the leader, decides. Writes with an outdated token are rejected.

COMMON MISTAKES

- Electing a leader with a lock in a single Redis node, which can lose the lock on failover.
- No fencing, so a paused ex-leader still writes.
- Lease shorter than normal GC pauses, causing leadership flapping.
- Two-node clusters, where neither side can win a majority after a split.

5 Key terms

Leader: the one node allowed to do a job or accept writes

Campaign: trying to become leader

Fencing token: increasing number that lets storage reject an old leader

Lease: leadership that expires unless renewed

Watch: get notified when a key changes

Zombie leader: an old leader that doesn't know it lost

6 Interview questions

Q1. Why do you need leader election? BASIC

When exactly one node must perform a task or accept writes, like a scheduler or a primary database, and another must take over automatically if it fails.

Q2. How does a lease-based election work? BASIC

Candidates try to create a key with a lease in a consensus store; one wins and keeps renewing; if it stops, the lease expires, the key disappears, and the watchers campaign again.

Q3. Design failover for a singleton cron service on Kubernetes. SCENARIO

Run 2 to 3 replicas using the Kubernetes Lease API for election, make the job resumable with checkpoints, and pass a fencing token or use idempotent writes so a stale leader can't double-run work.

Q4. Your old leader paused for 20 s and then overwrote the new leader's data. Fix it. SCENARIO

Add fencing tokens from the lease revision and make the storage reject writes with older tokens; also lengthen the lease beyond normal pauses.

Q5. Why not elect a leader with a simple Redis SETNX? TRICKY

A single Redis node is a single point of failure, and with async replication a failover can lose the key, letting two nodes believe they lead. It also has no built-in fencing.

Q6. Can two leaders ever exist at the same time with leases? TRICKY

Briefly, yes: an old leader can believe it holds the lease while a new one is elected (pauses, clock drift). That's why safety must come from fencing at the resource, not from the leader's belief.

Next: SD-H-009 · Consensus and Raft

Consensus and Raft

agreeing on one history despite failures

Watch the 4-min video

In one line: consensus lets a group of nodes agree on one ordered log of commands even when some crash; Raft elects a leader per term, the leader replicates log entries, and an entry is committed once a majority stores it, so a cluster of 2f+1 nodes survives f failures.

1 Why agreement is hard

Kubernetes stores every pod, service and secret in **etcd**. If that data were lost or inconsistent, the cluster would fall apart, yet any etcd node can crash at any time. How do several machines agree on one history despite crashes, lost messages and partitions? With a **consensus** algorithm. Raft (2014) was designed to be understandable; it powers etcd, Consul, CockroachDB, TiKV and Kafka's KRaft mode.

2 The goal: a replicated state machine

Every node keeps a **log** of commands. If all nodes apply the same commands in the same order from the same starting state, they reach the same state. Consensus is the problem of agreeing on that log.

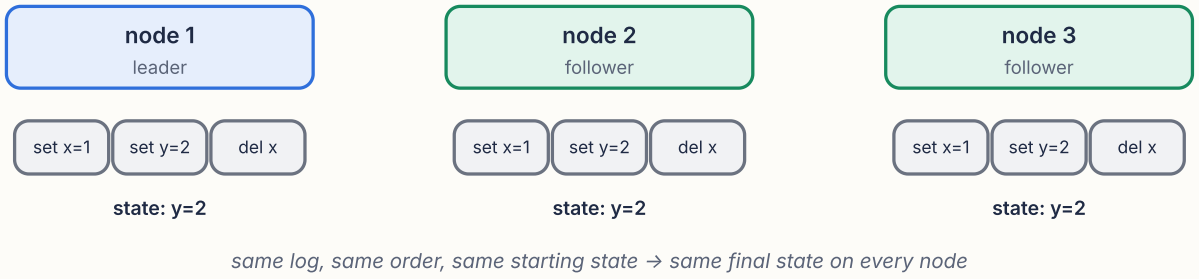


Figure 1. Raft replicates a log; each node applies it to its own copy of the state.

3 Terms and leader election

Raft has one **leader**; the others are **followers**. Time is divided into numbered **terms**, each with at most one leader. The leader sends regular heartbeats. A follower that hears nothing for a **randomised election timeout** (e.g. 150 to 300 ms) becomes a **candidate**: it increments the term, votes for itself and requests votes. Each node votes at most once per term, and only for a candidate whose log is at least as up to date as its own. A majority of votes makes it leader. Random timeouts make split votes rare; if one happens, a new term starts.

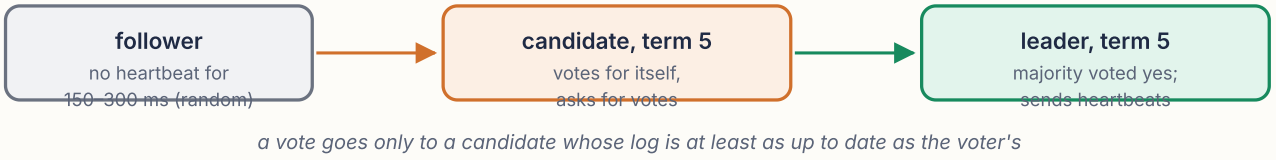


Figure 2. From follower to leader. The up-to-date-log rule guarantees the new leader already holds every committed entry.

4 Log replication and commit

Clients send commands to the leader. The leader appends the entry to its log and sends `AppendEntries` to followers. Once a **majority** (including itself) has stored the entry, it's **committed**: the leader applies it, replies to the client, and tells followers to apply it too. Any future leader must have a majority's votes, and any two majorities overlap, so a committed entry can never be lost.

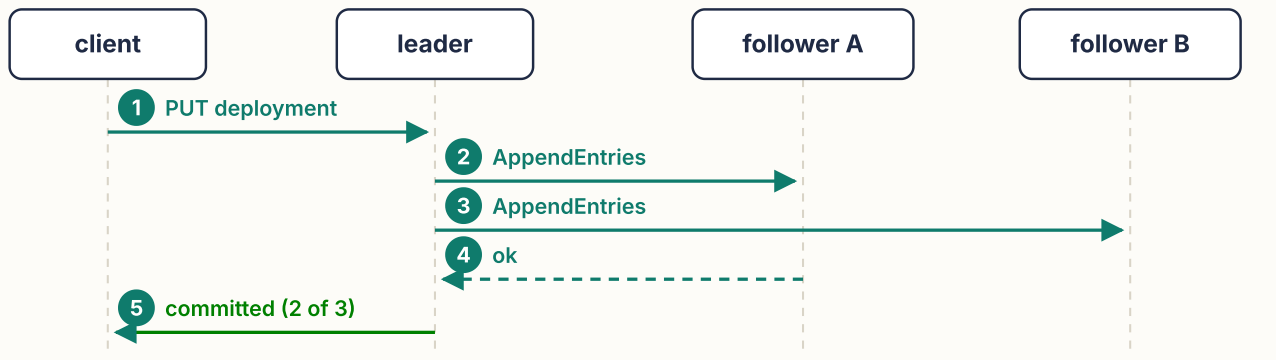


Figure 3. A write commits when 2 of 3 nodes store it; the third can catch up later.

WORKED EXAMPLE: A THREE-NODE ETCD

The Kubernetes API server writes a new deployment to the etcd leader. The leader appends it to its log and replicates it. Follower A acknowledges within a millisecond; follower B is slow. With 2 of 3 stored, the entry commits and the API server gets its answer, without waiting for B. If the leader then crashes, A or B times out, starts term 6, and the up-to-date-log rule makes sure the winner has the deployment. The cluster tolerates **one** failure; with five nodes it tolerates two.

5 Practical points

- Cluster sizes are odd: 3 or 5 (rarely 7). More nodes add fault tolerance but slow writes.
- Every write costs a round trip to a majority, so spread nodes across zones, not distant regions, unless you accept the latency.
- Logs are compacted into **snapshots**; membership changes use joint consensus or single-node changes.
- Reads must also be careful: a leader confirms it's still leader (read index or lease) before serving linearizable reads.

COMMON MISTAKES

- Running two or four nodes: no added tolerance over one or three.
- Spreading a Raft group across continents and wondering why writes take 200 ms.
- Serving reads from any follower and calling it strongly consistent.
- Using etcd as a general-purpose database for large data.

6 Key terms

Consensus: nodes agree on one value or log despite failures

Term: numbered period with at most one leader

Committed: stored on a majority; can't be lost

Replicated state machine: same log applied everywhere

Election timeout: random wait before a follower runs for leader

Quorum: majority: 2 of 3, 3 of 5

7 Interview questions

Q1. What problem does Raft solve? BASIC
It keeps a replicated log consistent across nodes despite crashes and lost messages, so the nodes act as one reliable state machine.

Q2. When is an entry committed in Raft? BASIC
When the leader has stored it on a majority of nodes in its current term; then it's safe to apply and acknowledge.

Q3. How many failures does a 5-node Raft cluster tolerate, and why not use 6? SCENARIO
Two. A 6-node cluster needs 4 for a majority and still tolerates only two failures, while every write waits for one more node.

Q4. The leader is partitioned from the other two nodes. What happens? SCENARIO
The two form a majority, elect a new leader in a higher term and keep committing. The old leader can't reach a majority, so its writes never commit, and it steps down when it sees the higher term.

Q5. Why are election timeouts randomised? TRICKY
So followers don't all become candidates at once and split the vote forever; one usually times out first and wins.

Q6. Can a Raft leader serve reads from its own state without talking to anyone? TRICKY
Only if it's sure it's still leader. Otherwise a deposed leader could return stale data. Raft uses a read index (a heartbeat round to a majority) or a time-based leader lease.

Next: SD-H-010 · Paxos Overview

Paxos Overview

the original consensus algorithm, in plain words

Watch the 4-min video

In one line: Paxos makes a majority of acceptors agree on one value in two phases (prepare/promise, then accept/accepted) using increasing proposal numbers, and a proposer must adopt any value already accepted; Multi-Paxos runs it per log slot with a stable leader, and is what Raft simplified.

1 Why learn Paxos

Before Raft there was **Paxos**. Leslie Lamport described it in the late 1980s, and it became the foundation of Google's Chubby, Spanner and Megastore. It's also famous for being hard to understand. You rarely implement it yourself, but interviews and papers refer to it constantly, and it shows the core idea every consensus algorithm uses.

2 Three roles



Figure 1. The roles in Basic Paxos. In practice one server plays all three.

3 Basic Paxos: agreeing on one value

Phase 1, prepare: a proposer picks a proposal number *n* higher than any it has used and sends `prepare(n)` to the acceptors. An acceptor that hasn't promised a higher number replies with a **promise** never to accept proposals below *n*, and reports the highest-numbered value it has already accepted, if any.

Phase 2, accept: with promises from a majority, the proposer sends `accept(n, v)`. The key rule: **v must be the value from the highest-numbered accepted proposal reported**, and only if none was reported may it choose its own value. An acceptor accepts unless it has since promised a higher number. When a majority accepts, the value is **chosen**, permanently.

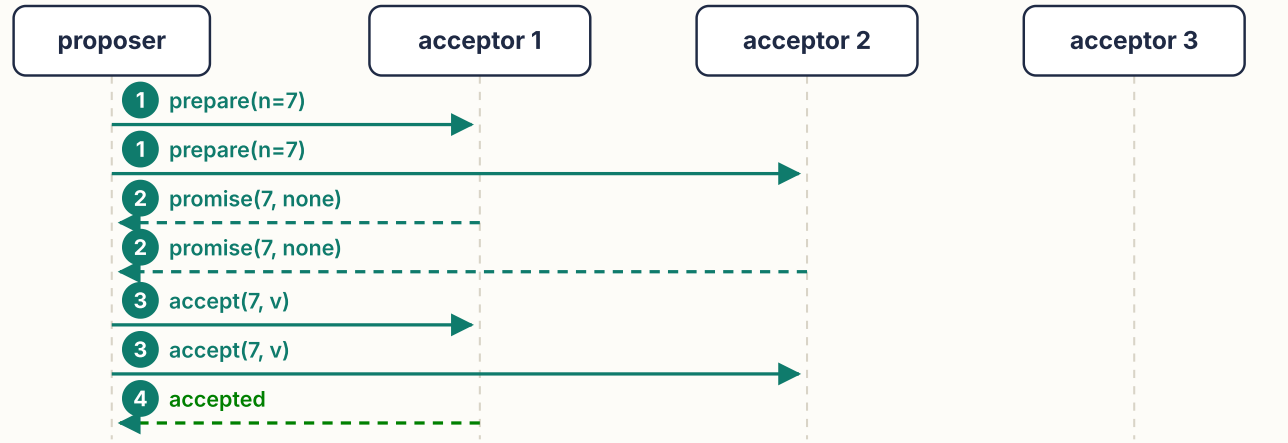


Figure 2. Basic Paxos with a majority of 2 out of 3 acceptors. The third acceptor can be down.

Why it's safe: any two majorities overlap in at least one acceptor. A later proposer's prepare reaches at least one acceptor that accepted the chosen value, learns about it, and has to propose that same value. Why it might not finish: two proposers can keep pre-empting each other with higher numbers (**livelock**); electing a single distinguished proposer avoids that.

4 Multi-Paxos

Real systems need a log, not one value. **Multi-Paxos** runs a Paxos instance per log slot, and lets a stable leader run phase 1 once for many slots, so each new command needs just one round trip. That's effectively what Raft spells out in detail.

WORKED EXAMPLE: GOOGLE'S CHUBBY

Chubby is a lock service with five replicas. They use Multi-Paxos to elect a master and replicate a log of operations. Clients find the master and send it all reads and writes; each write is committed once three of five replicas accept it. The master holds a lease, so it can serve reads locally without a round trip. If the master fails, the others run Paxos to choose a new one, typically within seconds. Many Google systems, from GFS to Bigtable, used Chubby for leader election and configuration.

	Paxos	Raft
decides	one value (Multi-Paxos: a log)	a log, from the start
leader	optional, an optimisation	required, strong leader
spec	short paper, many gaps	full algorithm, easier to build
used in	Chubby, Spanner, Megastore	etcd, Consul, CockroachDB

Figure 3. Paxos and Raft give the same guarantees; Raft fixes the parts Paxos leaves open.

COMMON MISTAKES

- Thinking a value is chosen when one acceptor accepts it: it needs a majority.
- Letting a proposer ignore previously accepted values.
- Building your own Paxos instead of using etcd, ZooKeeper or a proven library.

5 Key terms

Proposer / acceptor / learner: the roles in Paxos

Promise: acceptor won't accept lower-numbered proposals

Multi-Paxos: Paxos per log slot with a stable leader

Proposal number: unique, increasing ballot ID

Chosen: accepted by a majority; final

Livelock: competing proposers block progress

6 Interview questions

Q1. What are the two phases of Paxos? BASIC

Prepare/promise, where a proposer reserves a proposal number and learns any accepted values; then accept/accepted, where a majority accepts the value.

Q2. Why does Paxos need a majority? BASIC

Any two majorities share at least one acceptor, so a later proposal always learns about a value that may already be chosen, which keeps decisions consistent.

Q3. Two proposers keep overriding each other and nothing is decided. Why, and what's the fix? SCENARIO

Duelling proposers keep issuing higher numbers; use a single leader (distinguished proposer) and random back-off.

Q4. Paxos or Raft for a new system? SCENARIO

Neither by hand: use etcd, ZooKeeper or a library. If you must implement, Raft is easier to get right because it specifies leader election, log replication and membership changes.

Q5. Why must a proposer adopt the highest-numbered accepted value? TRICKY

That value may already be chosen by some majority; proposing anything else could make two different values chosen.

Q6. How does Multi-Paxos reduce latency? TRICKY

A stable leader runs phase 1 once for many slots, so each command needs only the accept phase: one round trip to a majority.

Next: SD-H-011 · Quorums

Quorums

how many replicas must say yes

Watch the 4-min video

In one line: with N replicas, a write waits for W acknowledgements and a read asks R replicas; if R + W > N, every read set overlaps the latest write set, so tuning W and R trades latency and availability against consistency, and repairs bring stale replicas back up to date.

1 The stale replica problem

You store every value on three replicas. A write succeeds on two of them; the third is still missing it. Now a read arrives. If it asks only the stale replica, the user sees old data. If it waits for all three, one slow node makes every read slow. **Quorums** are the middle way.

2 N, W and R

N is the number of replicas for a key. A write is successful once **W** replicas confirm it; a read queries **R** replicas and returns the newest version among their answers (by version or timestamp). The rule to remember: if **R + W > N**, the set of replicas you read from and the set that took the latest write must share at least one replica, so the read sees that write.

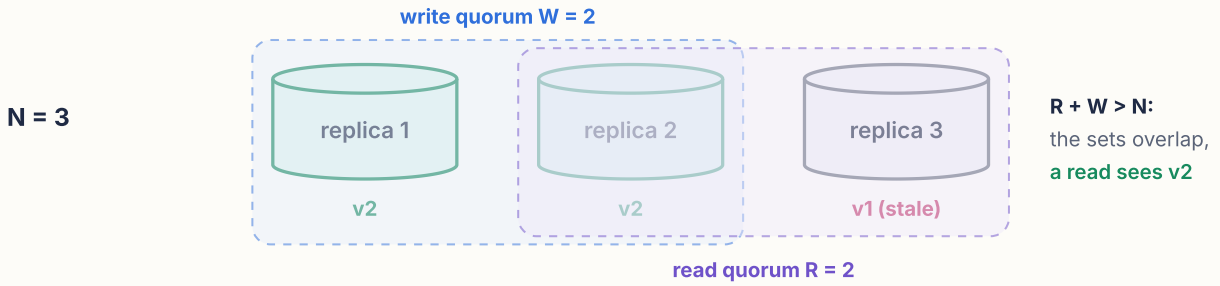


Figure 1. With N = 3, W = 2 and R = 2, any two replicas you read include at least one with the new value.

N=3, W=3, R=1	fast reads, any node down blocks writes
N=3, W=2, R=2	balanced: survives one node down for both
N=3, W=1, R=1	fastest, may read stale data (R+W ≤ N)
N=5, W=3, R=3	survives two nodes down

Figure 2. Common settings and what each one trades.

WORKED EXAMPLE: CASSANDRA QUORUM

A keyspace uses replication factor 3. The app writes with consistency level `QUORUM`, so 2 of 3 replicas must acknowledge, and reads with `QUORUM` too. One replica can be down or slow and both reads and writes still succeed, and every read sees the latest acknowledged write. Writing with `ONE` and reading with `ONE` would be faster, but a read could hit a replica that hasn't received the write yet. Across data centers, `LOCAL_QUORUM` keeps the quorum inside the local data center to avoid cross-region latency.

3 Keeping replicas in sync

Quorums let some replicas fall behind, so systems repair them: **read repair** (when a read sees a stale replica, write the newest value back to it), **anti-entropy** (a background process compares replicas with Merkle trees and copies missing data), and **hinted handoff**.

4 Sloppy quorums

Dynamo-style stores can accept a write on a stand-in node when some of the key's home replicas are unreachable: a **sloppy quorum**. The stand-in keeps a **hint** and hands the data back when the home replica returns. This raises write availability, but R + W > N no longer guarantees you read the latest value until the hint is delivered.

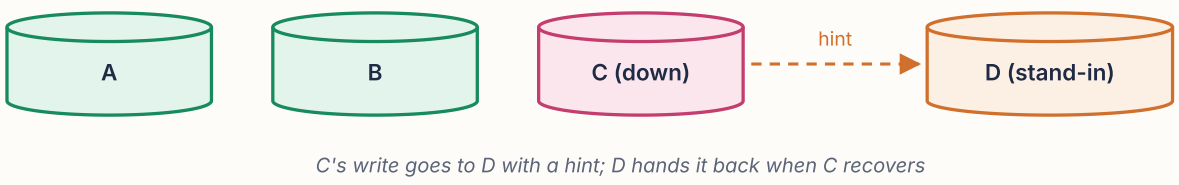


Figure 3. Hinted handoff keeps writes flowing while a replica is down.

COMMON MISTAKES

- Assuming R + W > N gives full linearizability: concurrent writes and failed writes that partially applied can still surprise you.
- Using ONE/ONE and expecting to read your own writes.
- Forgetting repairs: replicas drift and deleted data can come back (tombstones expire).
- Cross-region QUORUM on every request, paying WAN latency each time.

5 Key terms

N: number of replicas per key

Quorum: usually a majority: ⌊N/2⌋ + 1

Anti-entropy: background replica comparison (Merkle trees)

W / R: acks needed for a write / replicas asked on a read

Read repair: fix stale replicas during reads

Sloppy quorum: stand-in nodes take writes during failures

6 Interview questions

Q1. What does R + W > N guarantee? BASIC

That the replicas read and the replicas that acknowledged the latest write overlap, so a read sees the latest successfully written value.

Q2. With N = 3, how many failures can QUORUM reads and writes survive? BASIC

One. Each needs 2 of 3 replicas, so one can be down or slow.

Q3. You need very fast reads and can tolerate slower writes. Which W and R? SCENARIO

W = N and R = 1: every write reaches all replicas, so any single replica is up to date. But any one node down blocks writes.

Q4. Users in a multi-region Cassandra cluster complain about latency. What do you change? SCENARIO

Use LOCAL_QUORUM so each request waits only for replicas in the local data center, and replicate to other regions asynchronously.

Q5. Can you still read stale data with R + W > N? TRICKY

Yes: during a write in progress, after a write that failed on some replicas, with sloppy quorums, or when clocks decide the 'newest' version wrongly.

Q6. What's hinted handoff, and what's the catch? TRICKY

A stand-in node stores a write for a down replica and forwards it later. It improves availability, but until the hint is delivered a quorum read may miss that write.

Next: SD-H-012 · Split-Brain Scenarios

Split-Brain Scenarios

when two nodes both think they're the leader

Watch the 4-min video

In one line: split-brain happens when a cluster splits and both sides act as leader, accepting conflicting writes; prevent it with majority quorums, an odd number of voters (or a witness), and fencing that stops the old leader, because 'it stopped answering' never proves it stopped working.

1 Two primaries

Your database primary stops answering health checks. The failover system promotes a replica. But the old primary wasn't dead; it was cut off from the monitor, and app servers next to it are still writing to it. Now there are **two primaries** accepting writes, and their data diverges. That's **split-brain**, and it's one of the most damaging failures in distributed systems, because it silently corrupts data instead of just causing downtime.



Figure 1. The monitor can't tell 'dead' from 'unreachable' (lessons 3 and 4). Promoting without stopping the old primary creates two leaders.

2 Why it happens

- Failover decided by one observer that is itself partitioned.
- Two-node clusters: after a split, each side is exactly half, and a naive rule lets both win.
- A leader paused by GC or a frozen VM, which wakes up still believing it leads (lesson 8).
- Manual failover during an incident, without first making sure the old primary is down.

WORKED EXAMPLE: POSTGRES WITH A HOMEMADE FAILOVER SCRIPT

A Postgres cluster has a primary and one replica, with a simple script that promotes the replica if the primary fails three health checks. A network glitch hides the primary from the script for 20 seconds. The replica is promoted, DNS is switched, but some app servers keep their old connections to the original primary. For ten minutes, orders are written to both. Cleaning up means comparing two databases row by row. The fix: a tool like Patroni that keeps the leader key in etcd with a lease, a primary that **demotes itself** when it can't renew the key, and fencing (revoke its network access or kill it) before promotion.

3 Defences

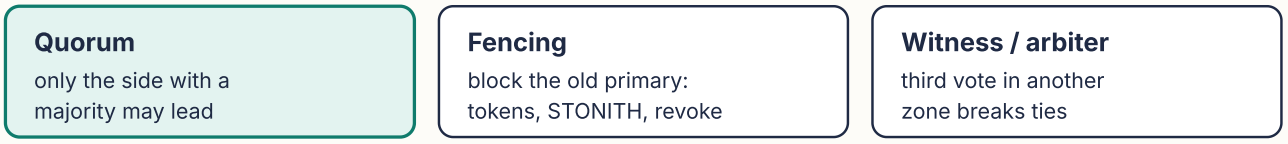


Figure 2. Majority, fencing and a tie-breaking witness: use them together.

Quorum: a node may act as leader only while it can reach a majority of voters; the minority side stops. **Fencing:** make it impossible for the old leader to do harm: fencing tokens checked by storage, revoking its credentials or network access, or power-cycling it (STONITH, 'shoot the other node in the head'). **Witness:** with only two data nodes, add a lightweight third voter in another zone so a majority always exists.

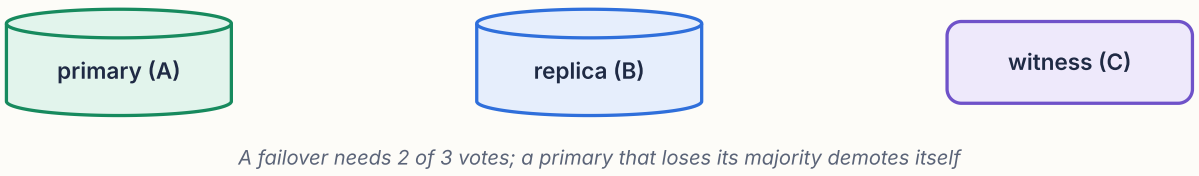


Figure 3. Two data nodes plus a witness: three voters, so one side of any split has a majority.

COMMON MISTAKES

- Failing over on one observer's opinion.
- Two-node clusters with automatic failover and no witness.
- Promoting a replica without fencing the old primary.
- Relying on DNS changes to move all clients instantly.

4 Key terms

Split-brain: two nodes act as leader at once

STONITH: forcibly power off the other node

Self-demotion: a leader steps down when it loses its lease or quorum

Fencing: stopping the old leader from acting

Witness / arbiter: voter without data that breaks ties

5 Interview questions

Q1. What is split-brain? BASIC

A state where two parts of a cluster each believe they're the leader and accept writes independently, so the data diverges.

Q2. How do quorums prevent it? BASIC

Only a side with a majority of voters may elect or keep a leader; two disjoint majorities can't exist, so at most one side leads.

Q3. You have two database nodes in two zones. How do you make automatic failover safe? SCENARIO

Add a witness in a third zone so there are three voters, keep leadership in a consensus store with a lease, and fence the old primary before promoting.

Q4. Split-brain already happened for ten minutes. What now? SCENARIO

Stop writes to one side, choose a survivor, find writes made only on the other side, reconcile them (replay or manual review), then rebuild the losing node as a replica.

Q5. Why isn't 'the primary stopped answering health checks' enough to promote a replica? TRICKY

The primary may only be unreachable from the monitor while still serving clients. Without fencing, promotion creates a second primary.

Q6. What's the role of fencing tokens here? TRICKY

Storage rejects writes with an older token, so even if a deposed leader keeps running and writing, its writes are refused.

Next: SD-H-013 · Distributed Locks backendroadmap.com

Distributed Locks

one worker at a time, even when things pause

Watch the 4-min video

In one line: a distributed lock lets one process at a time do something across machines, but a lock with an expiry can be lost during a pause without the holder knowing, so for correctness pair it with fencing tokens checked by the protected resource, and prefer consensus-backed stores.

1 It worked for months

Only one worker should process the nightly payout job, so you add a lock in Redis. It works for months, until one night a worker pauses, its lock expires, another worker takes the lock and starts paying out, and then the first one wakes up and continues. Every merchant is paid twice.

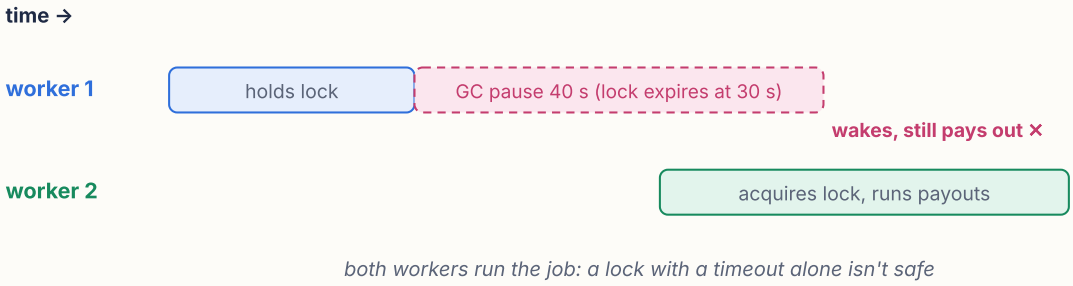


Figure 1. The first worker's lock expired during its pause, but the worker didn't know.

2 Two reasons to lock

Efficiency: avoid doing the same work twice (rebuilding a cache, sending a report). If the lock occasionally fails, you waste some work. **Correctness:** two holders would corrupt data or move money twice. These need a much stronger design. Always ask which one you need.

3 Locks need expiry, and expiry is the danger

A lock without an expiry is held forever if its owner crashes, so locks get a **lease** or TTL. But the holder can be paused by GC, a VM migration, or a slow disk past its TTL, and it can't detect that from inside: it might check 'do I still hold the lock?' and then pause right after the check. Clock jumps make TTLs even less reliable.

4 Fencing tokens

The safe fix moves the check to the **resource**. Each time the lock is granted, the lock service returns a number that only grows: a **fencing token**. The worker sends it with every write, and the resource remembers the highest token seen and rejects anything lower. A paused ex-holder's writes are refused.

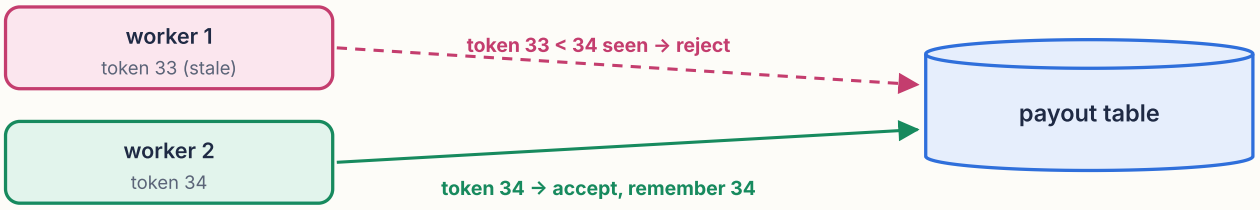


Figure 2. The resource, not the worker, enforces who may write.

WORKED EXAMPLE: PAYOUTS WITH AN ETCD LOCK

Three workers can run the nightly payout job. Each tries to acquire an etcd lock with a 30-second lease, and the winner gets a token: the lock key's revision, say 34. It keeps the lease alive while working, and writes each payout with `UPDATE payouts SET … WHERE batch_id = $1 AND last_token < 34`, storing 34. If it pauses and a new holder gets revision 35, the old worker's updates match no rows. Each payout also has an idempotency key, so a retried payout is never sent twice.

5 Which lock service

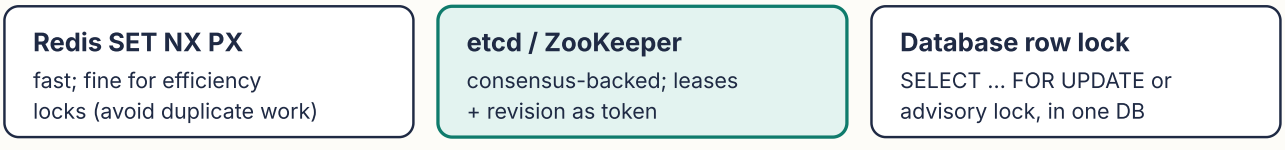


Figure 3. Choose by need: efficiency or correctness.

A single Redis node loses its locks on failover; the multi-node **Redlock** algorithm is debated because it depends on timing assumptions. Use Redis locks for efficiency. For correctness, use etcd or ZooKeeper (consensus, leases, monotonically increasing revisions) plus fencing, or let the database enforce it with a row lock or unique constraint. Often the best lock is none: make the operation idempotent.

COMMON MISTAKES

- Releasing a lock you no longer own: always check the value (a random owner ID) before deleting.
- TTL shorter than the job, with no renewal.
- Trusting a lock for money movement without fencing or idempotency.
- Locking on one Redis node and calling it highly available.

6 Key terms

Distributed lock: mutual exclusion across processes and machines

Fencing token: increasing number checked by the resource

Efficiency vs correctness lock: avoid duplicate work vs prevent corruption

Lease / TTL: lock expires unless renewed

Redlock: Redis multi-node lock algorithm, debated

7 Interview questions

Q1. Why must a distributed lock have a timeout? BASIC

If the holder crashes, the lock would never be released. A TTL or lease frees it, but then a slow or paused holder can lose it unknowingly.

Q2. What is a fencing token? BASIC

A strictly increasing number issued with each lock grant; the protected resource rejects requests with a token lower than the highest it has seen.

Q3. Design a lock so only one worker processes a payout batch. SCENARIO

Acquire a lease in etcd or ZooKeeper, use the revision as a fencing token, check it in the database update, renew the lease while working, and make each payout idempotent.

Q4. Your cache rebuild runs on 10 pods and hammers the DB. Do you need a strong lock? SCENARIO

No, an efficiency lock is enough: Redis SET NX with a TTL. An occasional duplicate rebuild is harmless.

Q5. Why isn't checking the lock right before writing enough? TRICKY

The process can pause between the check and the write, and the lock can expire and be taken by someone else in that gap.

Q6. What's the criticism of Redlock? TRICKY

It relies on bounded clock drift and pauses for safety and offers no fencing token, so under GC pauses or clock jumps two clients can believe they hold the lock.

Next: SD-H-014 · Gossip Protocols backendroadmap.com

Gossip Protocols

spreading news through a cluster like a rumour

Watch the 4-min video

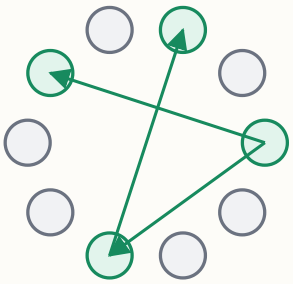
In one line: in a gossip protocol every node periodically swaps state with a few random peers, so information reaches all n nodes in about log n rounds with no central server; it's used for membership, failure detection and metadata, and gives eventual, not instant, agreement.

1 No central server

A cluster has a thousand nodes. A new node joins, or one dies. How does everyone find out? A central server could tell everyone, but it becomes a bottleneck and a single point of failure. Instead, nodes can spread the news the way a rumour spreads: each one tells a few others, who tell a few others. That's a **gossip** (or epidemic) protocol.

2 How it works

Every interval (say, one second), each node picks one to three peers at random and exchanges a summary of what it knows, each entry tagged with a version number such as a heartbeat counter. Each side keeps the newer version of every entry. Because the number of informed nodes roughly multiplies each round, news reaches the whole cluster in **O(log n)** rounds, while each node only does constant work per round.

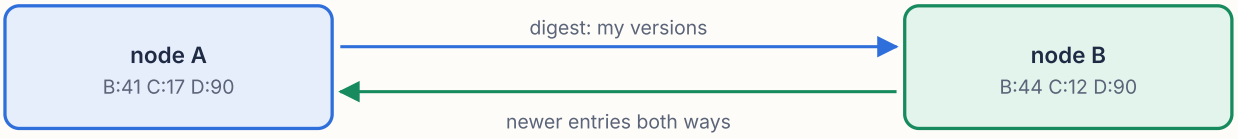


round 2: 4 of 10 know

each round, every node that knows tells a few random peers:

- 10 nodes → about 3 rounds
- 100 nodes → about 5 rounds
- 1,000 nodes → about 8 rounds
- 10,000 nodes → about 11 rounds

Figure 1. Exponential spread: a thousand nodes learn something in about ten rounds.



after: both have B:44, C:17, D:90 (highest heartbeat version wins)

Figure 2. One exchange: the nodes compare versions and both end up with the newest of each.

3 What clusters gossip about

Membership
who is in the cluster, joins and leaves

Liveness
heartbeat counters, suspected failures

Metadata
token ranges, schema versions, load

Figure 3. Typical gossiped state. Big payloads still go through normal replication.

WORKED EXAMPLE: CASSANDRA CLUSTER MEMBERSHIP

Every second, each Cassandra node gossips with up to three other nodes, exchanging its view of every node's state: heartbeat generation and version, status (joining, normal, leaving), token ranges and schema version. When node 12 joins, a seed node hears it first; within a few seconds every node knows node 12 and which data it owns. If node 7's heartbeat version stops increasing in everyone's gossip, the phi accrual detector (lesson 4) marks it down. **Seed nodes** are just well-known starting points for new nodes, not a central authority.

4 Trade-offs

- **Robust and scalable:** no single point of failure; lost messages are repaired in later rounds.
- **Eventually consistent:** for a few seconds, nodes may disagree about membership.
- **Bandwidth grows with state size:** send digests first and only the differences.
- Variants: **push**, **pull**, **push-pull**, and **SWIM** (used by Consul and Serf), which adds indirect probing for accurate failure detection.

COMMON MISTAKES

- Using gossip for data that needs strong consistency, like a leader decision.
- Gossiping full state every round in a large cluster.
- Too few seed nodes, or all in one zone.

5 Key terms

Gossip / epidemic protocol: random peer-to-peer state exchange

Digest: summary of versions sent before full data

SWIM: gossip membership with indirect probes

Fan-out: peers contacted per round

Seed node: well-known contact point for joining

Convergence: all nodes reach the same view

6 Interview questions

Q1. How does a gossip protocol spread information? BASIC

Each node periodically exchanges state with a few random peers and keeps the newer version of each entry; the news spreads exponentially, reaching everyone in about log n rounds.

Q2. Why use gossip instead of a central coordinator? BASIC

No bottleneck or single point of failure, constant work per node, and natural tolerance to lost messages and failed nodes.

Q3. How would 5,000 nodes learn about a new node within seconds? SCENARIO

Gossip every second with a fan-out of 2 to 3; about 13 rounds of doubling reach 5,000, so roughly 10 to 15 seconds in the worst case, faster with push-pull.

Q4. Would you use gossip to elect a leader for writes? SCENARIO

No. Gossip gives eventual agreement, so two nodes could both think they lead for a while. Use consensus (Raft, etcd) for that.

Q5. How does gossip handle conflicting information? TRICKY

Every entry has a version, such as a heartbeat counter or generation number, and the higher version wins on merge.

Q6. What limits gossip at very large scale? TRICKY

Bandwidth and CPU grow with the size of the state, not just the node count. Digests, deltas and hierarchy keep it manageable.

Next: SD-H-015 · Distributed Transactions Overview

Distributed Transactions Overview

all or nothing, across services

Watch the 4-min video

In one line: when one business action changes data in several services or databases there's no single transaction to make it all-or-nothing; you choose between two-phase commit (atomic but blocking and coupled) and sagas (local transactions plus compensations, eventually consistent), or you redesign the boundaries.

1 The problem

A customer places an order. You must create the order, charge the card and reserve the stock. In a monolith with one database, that's one transaction: all or nothing. Split into order, payment and inventory services, each with its own database, and there's no transaction that covers all three. If the charge succeeds and the stock reservation fails, the customer pays for nothing.

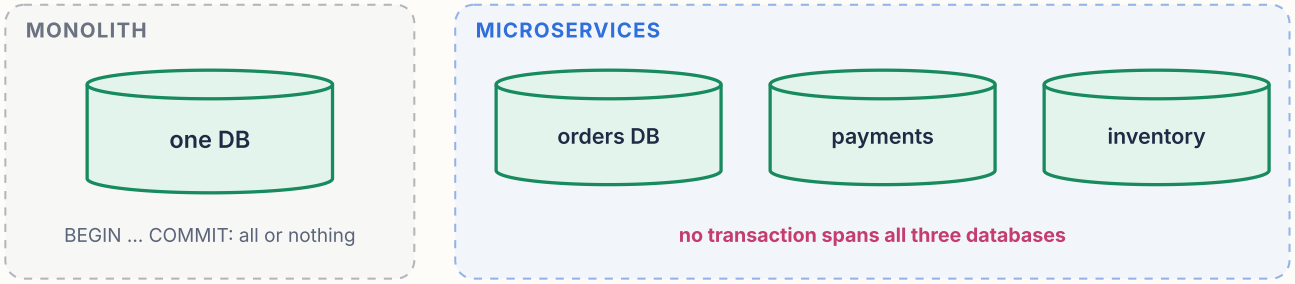


Figure 1. One database gives you ACID for free. Separate databases don't share a transaction.

2 What 'atomic' means across services

We want **atomicity** (all steps happen or none do, from the business point of view), and usually also **isolation** (nobody sees half-done states). Across services you can get atomicity with coordination, or settle for steps that are **eventually** all done or all undone. Full isolation across services is rarely affordable.



Figure 2. The four ways to approach it. Sagas with an outbox are the usual choice for microservices.

3 Two-phase commit

A **coordinator** asks every participant to **prepare** (lock and promise to commit), then tells all to **commit** only if all said yes (lesson 16). It's truly atomic, but locks are held across network round trips, a coordinator failure can leave participants blocked, and every participant must support XA. Good inside one database cluster or a few cooperating databases; a poor fit for independent microservices and external APIs like a card processor.

4 Sagas

A **saga** splits the action into local transactions, one per service. Each step commits on its own. If a later step fails, earlier steps are undone by **compensating actions** (refund the card, release the stock). The system is briefly inconsistent, but it ends either fully done or fully compensated (lesson 17). To make each step reliable, services save state and publish events atomically with the **transactional outbox** (lesson 37), and consumers deduplicate (lesson 38).

WORKED EXAMPLE: ORDER, PAYMENT, INVENTORY AS A SAGA

The order service creates the order as **PENDING** and writes an `OrderCreated` event to its outbox in the same transaction. The payment service consumes it, charges the card with an idempotency key, and emits `PaymentCompleted`. The inventory service tries to reserve stock and fails: `OutOfStock`. The payment service reacts by refunding, and the order service marks the order **CANCELLED** and emails the customer. No step ever held a lock across services.

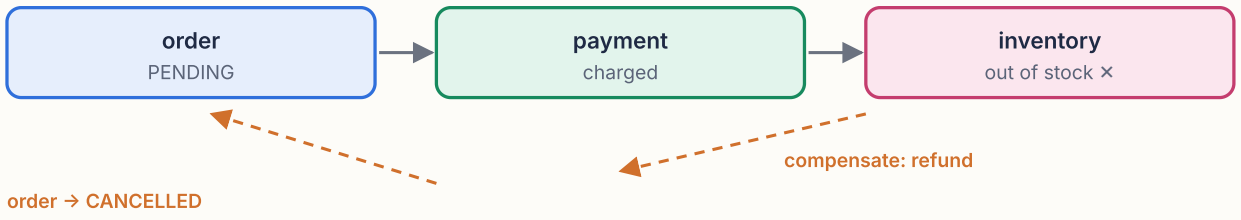


Figure 3. A saga undoes completed steps with compensations when a later step fails.

5 Before you reach for either

Ask whether the data really belongs in different services. If two tables must always change together, they may belong in one service and one database. Distributed transactions are often a sign that service boundaries cut through one consistency boundary.

COMMON MISTAKES

- Calling three services one after another and hoping none fails.
- Writing to the DB and then publishing to Kafka without an outbox.
- Sagas without idempotent steps, so retries charge twice.
- Showing PENDING states to users as if they were final.

6 Key terms

Distributed transaction: one logical change across several data stores

Saga: sequence of local transactions with compensations

Transactional outbox: save data and event in one local transaction

2PC: prepare, then commit, under a coordinator

Compensating action: business undo of a completed step

7 Interview questions

Q1. Why can't microservices just use a normal transaction? BASIC
Each service owns its own database; a database transaction can't span other databases or external APIs, so there's no single commit point.

Q2. 2PC or saga: what's the core trade-off? BASIC
2PC gives atomicity and isolation but blocks and couples participants; a saga avoids locks and coupling but is only eventually consistent and needs compensations.

Q3. Design checkout across order, payment and inventory services. SCENARIO
Use a saga: pending order, charge with an idempotency key, reserve stock; on failure refund and cancel. Publish events through outboxes and deduplicate in consumers.

Q4. When is 2PC still reasonable? SCENARIO
Within one organisation's databases that support XA, short transactions, low latency between them, and when strict atomicity matters more than availability.

Q5. What isolation problems do sagas have? TRICKY
Other transactions can see intermediate states, like a charged but not yet reserved order. Mitigate with semantic locks (PENDING status), commutative updates, and re-checking values.

Q6. Why might needing a distributed transaction indicate a design problem? TRICKY
Data that must change atomically usually belongs to one consistency boundary; splitting it across services creates the problem in the first place.

Next: SD-H-016 · Two-Phase Commit (2PC)

Two-Phase Commit (2PC)

an atomic commit across several databases

Watch the 4-min video

In one line: in 2PC a coordinator asks every participant to prepare (durably promise to commit), and commits only if all vote yes; it's atomic, but participants hold locks while waiting and block if the coordinator fails after they voted, which is why it's used within databases and rarely between microservices.

1 One transfer, two databases

Move $100 from an account in one database to an account in another. The debit and the credit must both happen, or neither. Two separate databases, one atomic outcome. **Two-phase commit** (2PC) is the classic protocol for that.

2 The protocol

Phase 1, prepare (voting): the coordinator sends `PREPARE` to every participant. Each one does all the work, writes it durably to its log, holds its locks, and votes **YES** (it promises it can commit, even after a crash) or **NO**. **Phase 2, commit:** if every vote is YES, the coordinator writes its decision to its own log and sends `COMMIT`; if any vote is NO or missing, it sends `ABORT`. Participants apply the decision and release their locks.

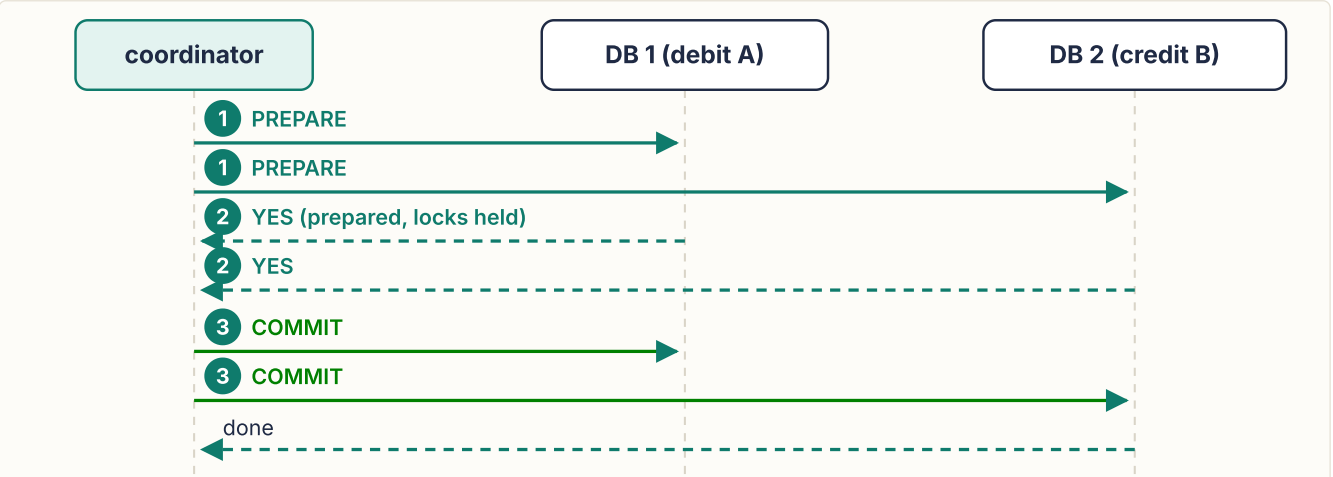


Figure 1. 2PC for a transfer. Once a participant votes YES, it can no longer decide on its own.

WORKED EXAMPLE: AN XA TRANSFER BETWEEN TWO POSTGRES DATABASES

The transaction manager debits account A in database 1 and credits account B in database 2, then runs `PREPARE TRANSACTION 'tx-81'` on both. Both succeed. It records 'commit tx-81' in its own log, then runs `COMMIT PREPARED 'tx-81'` on both. If database 2 had failed the credit (say, account closed), it would vote NO and both would run `ROLLBACK PREPARED`. If the manager crashes after the prepares, both transactions stay **in doubt**, holding row locks on A and B, until it recovers and reads its log.

3 Why 2PC has a bad reputation

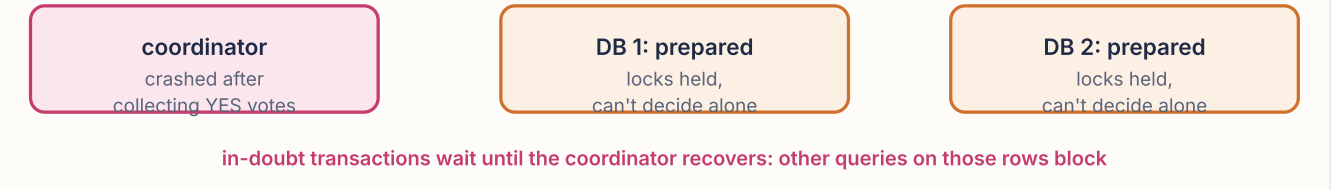


Figure 2. The blocking problem: prepared participants must wait for the coordinator's decision.

- **Blocking:** participants that voted YES can't commit or abort alone if the coordinator disappears.
- **Latency and locks:** at least two round trips, with locks held throughout, so throughput drops under contention.
- **Availability:** any one participant down means no transaction commits.
- **Coupling:** every participant must speak the same protocol; external APIs (a card processor, an email service) can't take part.

4 Where 2PC works well

Distributed databases such as Spanner and CockroachDB run 2PC across shards, but each participant and the coordinator are themselves **Raft or Paxos groups**, so they don't disappear, and the blocking problem mostly goes away. Kafka transactions use a similar two-phase idea to write to several partitions and commit consumer offsets atomically. 3PC (three-phase commit) tries to avoid blocking but isn't safe under network partitions, so it's rarely used.

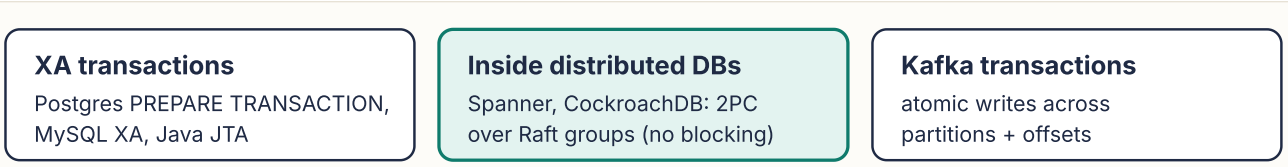


Figure 3. 2PC in practice: mostly inside systems that make the coordinator fault-tolerant.

COMMON MISTAKES

- Using XA between microservices owned by different teams.
- Forgetting to monitor and resolve in-doubt (prepared) transactions.
- Assuming 2PC protects calls to external, non-transactional APIs.

5 Key terms

Coordinator: drives the protocol and decides

Prepare / vote: durably promise to commit if told

XA: standard interface for 2PC across resources

Participant: a database taking part

In-doubt transaction: prepared, waiting for the decision

3PC: non-blocking variant, unsafe under partitions

6 Interview questions

Q1. Describe the two phases of 2PC. BASIC

Prepare: participants do the work durably, hold locks and vote yes or no. Commit: if all voted yes, the coordinator logs and sends commit; otherwise abort.

Q2. What is the main weakness of 2PC? BASIC

It blocks: if the coordinator fails after participants vote yes, they hold locks and can't decide until it recovers.

Q3. Would you use 2PC for order, payment and shipping microservices? SCENARIO

No: it couples the services, holds locks across network calls, and can't include external payment APIs. Use a saga with an outbox.

Q4. Your database has prepared transactions from last week holding locks. What happened and what do you do? SCENARIO

The coordinator crashed or lost track after prepare. Check its log for the decision and commit or roll back each one manually, then add monitoring for in-doubt transactions.

Q5. How do Spanner and CockroachDB avoid 2PC's blocking? TRICKY

The coordinator and participants are replicated with consensus, so the coordinator's decision survives node failures and another replica can finish the protocol.

Q6. Why doesn't 3PC solve the problem in practice? TRICKY

It assumes bounded network delays; under partitions, different groups can reach different decisions, breaking atomicity.

Next: SD-H-017 · Saga Pattern

Saga Pattern

long transactions as steps you can undo

Watch the 4-min video

In one line: a saga splits a multi-service transaction into local steps, each with a compensating action; if a step fails, the completed steps are compensated in reverse, coordinated either by an orchestrator or by services reacting to events, and every step must be idempotent and retryable.

1 Book a trip

A customer books a trip: a flight, a hotel and a rental car, from three different providers. The flight and hotel succeed. The car fails. You can't hold locks across three companies' systems for minutes, and they don't speak 2PC. What you can do is **cancel** the flight and the hotel. That's the saga idea, first described for long-lived database transactions in 1987.

2 Steps and compensations

A saga is a sequence of **local transactions** T1, T2, ... Tn. Each has a **compensating transaction** C1, C2, ... that semantically undoes it. If Tk fails, the saga runs Ck-1 ... C1 in reverse order. The outcome is all steps done, or all completed steps compensated. In between, other users can see partial states: sagas give up isolation.

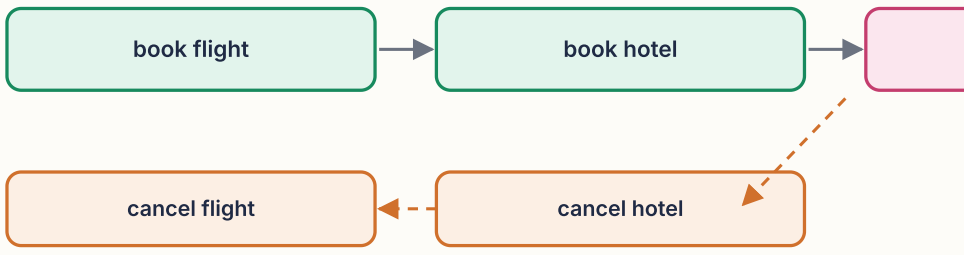


Figure 1. The car fails, so the hotel and then the flight are cancelled.

3 Orchestration or choreography

Orchestration: one component (the orchestrator) tells each service what to do and tracks the saga's state; easy to see and change the flow. Workflow engines such as Temporal, AWS Step Functions or Camunda make the orchestrator durable. **Choreography:** each service listens for events and reacts (FlightBooked → book hotel); no central brain, but the flow is spread across services and harder to follow as it grows.



Figure 2. Orchestration centralises the flow; choreography distributes it through events.

WORKED EXAMPLE: A TRIP SAGA IN TEMPORAL

The workflow books the flight, then the hotel, then the car, registering a compensation after each success. The car API returns 'no availability'. The workflow runs the registered compensations in reverse: cancel hotel, cancel flight, retrying each with backoff until it succeeds. Each activity sends an idempotency key (`trip-512-flight`), so retries after a timeout never double-book. Because Temporal persists the workflow's history, a crash of the worker in the middle simply resumes the saga where it stopped.

4 Rules for good sagas

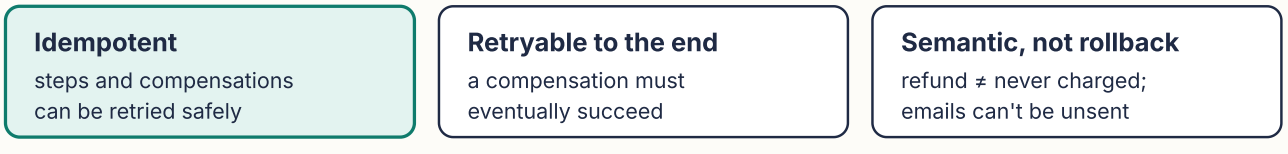


Figure 3. Compensations are business actions, not database rollbacks.

- Order steps so the ones most likely to fail, or hardest to undo, come first; put irreversible steps (send email, ship) last.
- Use **semantic locks**: mark records as PENDING so other flows know they're in progress.
- Make the saga's state durable (a workflow engine or a saga table), and publish events through an outbox.
- Plan for a compensation that can't succeed automatically: alert a human.

COMMON MISTAKES

- Compensations that aren't idempotent, so a retry refunds twice.
- Keeping saga state only in memory.
- Choreography across ten services with no way to see where a saga is stuck.
- Treating PENDING data as final in other features.

5 Key terms

Saga: sequence of local transactions with compensations

Orchestration: a central coordinator drives the steps

Semantic lock: a status flag marking work in progress

Compensating transaction: semantic undo of a completed step

Choreography: services react to each other's events

Pivot step: the point after which the saga only moves forward

6 Interview questions

Q1. What is a saga? BASIC

A way to run a multi-service transaction as a series of local transactions, each with a compensating action that's run in reverse if a later step fails.

Q2. Orchestration vs choreography? BASIC

Orchestration has a central coordinator that calls each step and tracks state; choreography has services react to events. Orchestration is easier to observe; choreography is more decoupled.

Q3. Design hotel and flight booking with a saga. SCENARIO

Orchestrator books flight then hotel with idempotency keys; on hotel failure it cancels the flight. Saga state is durable, steps are retried with backoff, and failures that can't be compensated alert a person.

Q4. A compensation keeps failing. What do you do? SCENARIO

Retry with backoff since it must eventually succeed; if it's permanently impossible, park the saga in a failed state and escalate for manual handling.

Q5. What anomalies can sagas cause? TRICKY

Lack of isolation: other transactions see intermediate states (dirty reads), lost updates if two sagas change the same data, and fuzzy reads. Use semantic locks and re-validation.

Q6. Why put irreversible steps last? TRICKY

Once something can't be undone, like shipping or an email, nothing after it may fail and require compensation, so it should be the final step.

Next: SD-H-018 · Eventual Consistency in Practice backendroadmap.com

Eventual Consistency in Practice

stale for a moment, correct in the end

Watch the 4-min video

In one line: eventually consistent replicas may briefly disagree but converge once updates stop; it buys latency and availability, and is safe to use when you add session guarantees like read-your-writes, design conflict-free updates, and keep strong consistency for the few things that truly need it.

1 99 or 100?

You like a post and the count goes from 99 to 100. Your friend, on another server, still sees 99 for a few seconds. Nothing is broken: the replicas simply haven't caught up yet. That's **eventual consistency**: if no new updates arrive, all replicas eventually return the same value. Most of the internet runs this way, from DNS to social feeds to shopping carts.

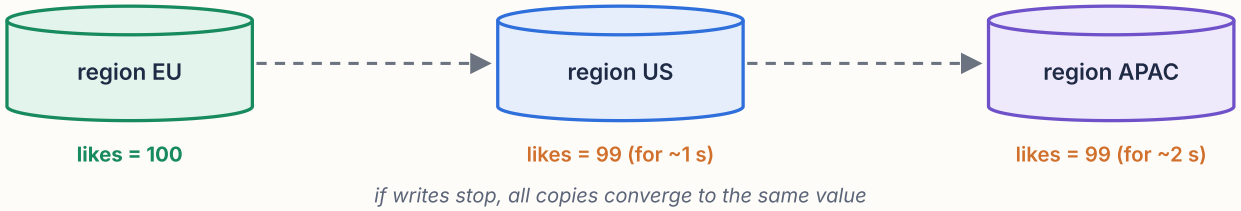


Figure 1. Replicas accept writes locally and copy them asynchronously; for a short window they disagree.

2 Why accept it

Strong consistency needs coordination: a write waits for other replicas, often in other regions, and during a partition some requests must fail (lesson 3). Eventual consistency lets each replica answer immediately, so reads and writes stay fast and available. The price is **staleness** and possible **conflicts**.

3 Making it feel consistent: session guarantees

Users mostly notice inconsistency with their own actions. Four cheap guarantees remove most of the confusion:



Figure 2. Session guarantees. Read-your-writes alone fixes the most common complaint.

Implement them by routing a user to the same replica (sticky sessions), reading from the leader for a few seconds after that user writes, or passing a version token (the log position of the user's last write) and waiting until the replica has reached it.

WORKED EXAMPLE: LIKE COUNTS

Each like is written to the nearest region's replica, and the count is a **CRDT counter** (lesson 19), so concurrent likes from different regions merge by adding, never by overwriting. The person who liked sees 100 immediately, because the app updates the count locally and reads from the region it wrote to. Others see it within a second or two. The 'did I like this?' flag uses read-your-writes, so the heart never flips back. Payments for promoted posts, by contrast, go through a strongly consistent ledger.

4 Handling conflicts

- **Last-write-wins:** simple, but loses concurrent updates and depends on clocks (lesson 5).
- **Merge functions:** union of cart items, max of timestamps, sum of counters; CRDTs make this automatic.
- **Ask the application or user** to resolve, as with Git merges.
- **Avoid them:** route all writes for one key to one leader.



Figure 3. Consistency is a spectrum. Pick per feature, not per company.

COMMON MISTAKES

- Reading from a replica right after a write and showing the user old data.
- Eventual consistency for balances, inventory limits or unique usernames.
- No monitoring of replication lag, so 'eventually' becomes minutes.
- Last-write-wins on data where merges are easy and losses hurt.

5 Key terms

Eventual consistency: replicas converge if updates stop

Read-your-writes: a user always sees their own updates

Replication lag: how far a replica is behind the leader

Staleness: how old a read may be

Monotonic reads: no going back in time across reads

Convergence: all replicas reach the same state

6 Interview questions

Q1. What is eventual consistency? BASIC

A guarantee that if no new updates are made, all replicas will eventually return the same value; reads in between may be stale.

Q2. What's read-your-writes consistency, and how do you provide it? BASIC

A user always sees their own updates. Read from the leader briefly after a write, use sticky routing, or wait for the replica to reach the user's last-write position.

Q3. Users edit their profile and the old version shows on refresh. Fix it. SCENARIO

The read hit a lagging replica. Add read-your-writes: route that user's reads to the primary for a few seconds or use a version token, and monitor lag.

Q4. Which features of a social app can be eventually consistent, and which can't? SCENARIO

Likes, feeds, follower counts and view counts can; payments, account balances, unique usernames and permission changes need strong consistency.

Q5. How long is 'eventually'? TRICKY

No bound is promised; in practice it's replication lag, usually milliseconds to seconds, but it can grow under load or partitions, so measure and alert on it.

Q6. Can an eventually consistent system lose data? TRICKY

Yes, with last-write-wins on concurrent writes or if a node holding unreplicated writes dies. Use quorums, durable logs and merge-friendly data types to avoid it.

Next: SD-H-019 · CRDTs

CRDTs

data types that merge without conflicts

Watch the 4-min video

In one line: CRDTs (conflict-free replicated data types) are counters, sets, maps and text whose replicas can be updated independently and merged in any order with the same result, because the merge is commutative, associative and idempotent; that gives strong eventual consistency without coordination.

1 Merging without a referee

Two people edit the same shared document offline. When they reconnect, their changes must merge, with no central server deciding and no conflicts to fix by hand. **CRDTs** make that possible by designing the data type so that any two replicas can always be merged, and every replica that has seen the same updates ends up in the same state. This property is called **strong eventual consistency**.

2 The simplest one: a grow-only counter

A **G-counter** keeps a vector with one entry per node. To increment, a node bumps only its own entry. The value is the sum of all entries. To merge two replicas, take the **maximum** of each entry. No increment is ever lost or counted twice, whatever order the merges happen in.

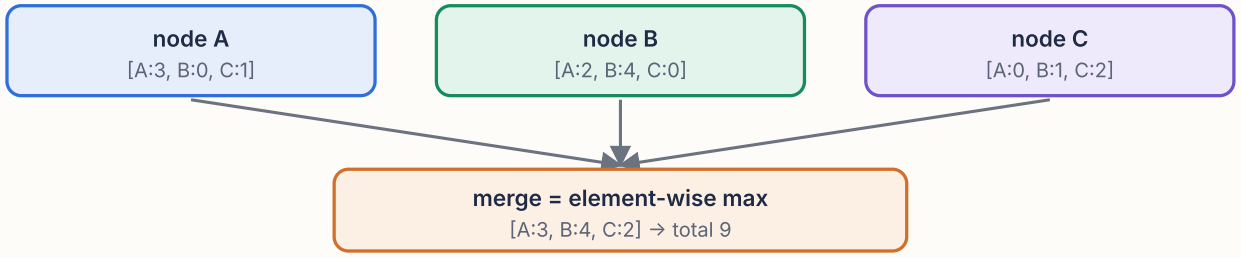


Figure 1. Merging G-counters by element-wise max. Each node only ever increases its own slot.

3 Why it works



Figure 2. If merge has these three properties, replicas converge no matter how messages are delayed, reordered or duplicated.

Mathematically the states form a **join-semilattice** and merge is the join. In practice: design the state so it only 'grows' in a well-defined direction and the merge picks the upper bound. There are two styles: **state-based** CRDTs send whole states (or deltas) and merge them; **operation-based** CRDTs send operations that must commute.

G-Counter	grow-only counter (views)
PN-Counter	two G-counters: increments − decrements (likes)
G-Set / 2P-Set	add-only set / add then remove forever
OR-Set	add and remove, add wins over concurrent remove (carts)
LWW-Register	single value, latest timestamp wins
Sequence (RGA, YATA)	ordered text for collaborative editing

Figure 3. Common CRDTs and what they're used for.

WORKED EXAMPLE: A LIKE COUNTER AND A SHOPPING CART

Likes use a **PN-counter**: one G-counter for likes and one for unlikes, value = P − N. A user in the EU and one in the US like at the same moment; each region bumps its own slot, and after merging both regions show +2. A cart uses an **OR-set** (observed-remove set): each add is tagged with a unique ID, and a remove deletes only the tags it has seen. If one device removes the charger while another, concurrently, adds it again, the new add has an unseen tag, so the charger stays: add wins. Redis Enterprise active-active, Riak and Azure Cosmos DB offer CRDT types; Automerge and Yjs bring them to collaborative apps.

4 Limits

- Not every rule fits: 'balance must never go below zero' or 'username must be unique' need coordination.
- **Metadata grows:** tombstones and per-node entries need garbage collection.

- The merged result is always well defined, but not always what a user expects; design the semantics deliberately.

COMMON MISTAKES

- A counter that stores just a number and merges by max or last-write-wins: concurrent increments are lost.
- Using CRDTs for invariants that need a global check.
- Never compacting tombstones.

5 Key terms

CRDT: conflict-free replicated data type

State-based CRDT: replicas exchange and merge states

OR-Set: set where a concurrent add wins over remove

Strong eventual consistency: same updates seen → same state

Operation-based CRDT: replicas exchange commuting operations

Tombstone: marker for a removed element

6 Interview questions

Q1. What is a CRDT? BASIC

A replicated data type whose replicas can be updated independently and merged in any order to the same state, because the merge is commutative, associative and idempotent.

Q2. How does a G-counter work? BASIC

Each node increments its own entry in a vector; the value is the sum; merging takes the element-wise maximum.

Q3. Design a global like counter across regions without coordination. SCENARIO

Use a PN-counter CRDT replicated per region, update locally, merge asynchronously; show the local value with read-your-writes for the liker.

Q4. Could you use a CRDT for a bank balance with no overdraft? SCENARIO

Not alone: the no-overdraft rule needs a global check. You could use escrow (pre-split the allowed spend between regions) or route withdrawals through a single leader.

Q5. Why must the merge be idempotent? TRICKY

Networks duplicate and retry messages; merging the same state twice must not change the result, or counts would grow on every retry.

Q6. How do collaborative editors like Google Docs or Figma relate to CRDTs? TRICKY

Google Docs historically used operational transformation with a central server; many newer tools, and libraries like Yjs and Automerge, use sequence CRDTs so peers can merge without a server.

Next: SD-H-020 · Idempotent Processing backendroadmap.com

Idempotent Processing

doing it twice has the same effect as once

Watch the 4-min video

In one line: messages and retries arrive more than once, so make every handler idempotent: record processed message IDs in the same transaction as the side effect, use upserts and absolute updates, and pass idempotency keys to external APIs, so a duplicate changes nothing.

1 Charged twice

A payment consumer reads a message: charge the customer $40. It charges the card, then crashes before acknowledging the message. The broker redelivers it. The consumer charges again. The customer pays $80. Nothing malfunctioned: redelivery is how brokers guarantee you don't **lose** messages.

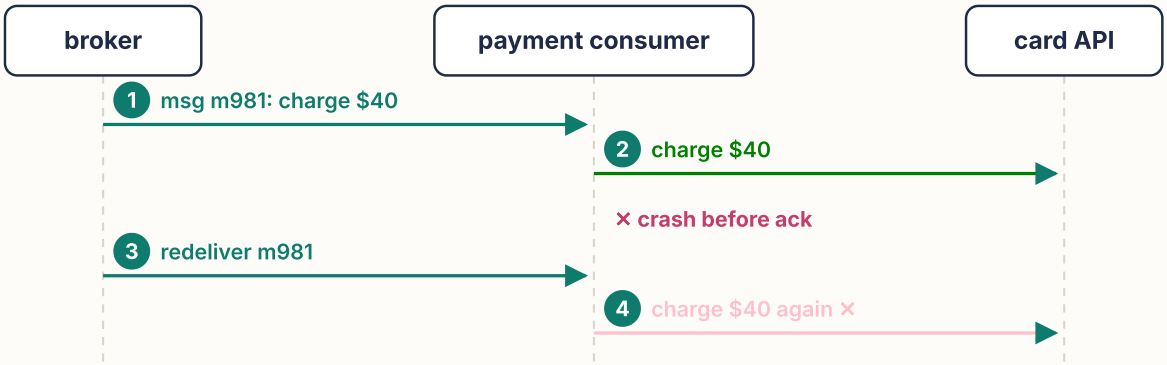


Figure 1. At-least-once delivery means a crash between the work and the ack causes a duplicate.

2 What idempotent means

An operation is **idempotent** if doing it twice has the same effect as doing it once. `SET balance = 60` is idempotent; `balance = balance − 40` is not. In distributed systems you can't prevent duplicates (retries, redeliveries, double clicks, replays), so you make processing them harmless.



Figure 2. Four ways to make a handler idempotent. Most systems combine two of them.

3 The dedup table pattern

Give every message a unique ID. In one database transaction, insert the ID into a `processed_messages` table with a unique constraint **and** apply the change. If the ID already exists, the insert fails, the transaction rolls back, and you just acknowledge the message. Because the record of processing and the effect commit together, a crash can never leave one without the other. This is also called the **inbox pattern** (lesson 38).

WORKED EXAMPLE: THE PAYMENT CONSUMER

The payment consumer receives message `m981`. In one transaction it inserts `m981` into `processed_messages`, records a pending charge, and commits. It calls the card provider with idempotency key `m981`, so if it crashes and retries, the provider returns the original charge instead of making a new one. Then it marks the order PAID and acknowledges. When the broker redelivers `m981`, the unique insert fails and the consumer just acks: one charge, however many deliveries.

```
BEGIN;
INSERT INTO processed_messages (id) VALUES ('m981');   -- unique key
--   duplicate? → unique violation → ROLLBACK, ack, skip
UPDATE orders SET status = 'PAID' WHERE id = 42;
COMMIT;
-- then acknowledge the message
```

Figure 3. Dedup and effect in one transaction. External calls get the same ID as an idempotency key.

4 Details that matter

- Use a **stable** ID created by the producer (a UUID or business key), not the broker's delivery ID, which may change on redelivery.
- Keep dedup records at least as long as duplicates can arrive (the broker's retention or retry window), then expire them.
- External side effects (emails, card charges) can't join your transaction: use the provider's idempotency keys, or a status column checked before sending.

- Prefer absolute writes (`SET`, upsert) over relative ones (`+=`).

COMMON MISTAKES

- Checking 'already processed?' in one transaction and doing the work in another.
- Dedup in an in-memory set that disappears on restart.
- Acking before doing the work, which turns duplicates into lost messages.
- Generating a new idempotency key on each retry.

5 Key terms

Idempotent: repeating it has the same effect as once

Dedup table / inbox: processed message IDs with a unique key

Upsert: insert or update by key

At-least-once: messages may be redelivered, never lost

Idempotency key: client-chosen ID the server uses to deduplicate

6 Interview questions

Q1. What is idempotent processing, and why is it needed? BASIC

Processing where duplicates have no extra effect. Brokers and retries deliver messages at least once, so without it, crashes and retries cause double effects.

Q2. How do you deduplicate messages reliably? BASIC

Store each message's unique ID in a table with a unique constraint in the same transaction as the business change; a duplicate fails the insert and is skipped.

Q3. A consumer sends welcome emails and users sometimes get two. Fix it. SCENARIO

Record 'welcome sent' per user in the database with a unique constraint before or atomically with sending, or use the email provider's idempotency key; also avoid acking before processing.

Q4. How long should you keep processed message IDs? SCENARIO

At least as long as a duplicate can still arrive: the broker's redelivery and retention window plus any replay period, then expire them with a TTL or partitioned table.

Q5. Is 'UPDATE stock = stock − 1' idempotent? How do you fix it? TRICKY

No. Record the message ID with the change, or use an absolute value with a version check, so a duplicate can't subtract twice.

Q6. Why not just use exactly-once delivery? TRICKY

End-to-end exactly-once side effects aren't possible over an unreliable network with external systems; at-least-once plus idempotency gives the same visible result.

Next: SD-H-021 · Exactly-Once Processing: Claims and Limits backendroadmap.com

Exactly-Once Processing: Claims and Limits

what 'exactly-once' really promises

Watch the 4-min video

In one line: no network can deliver every message exactly once, but you can get exactly-once *effects*: at-least-once delivery plus idempotent or transactional processing; Kafka's exactly-once covers read-process-write inside Kafka, and anything outside it still needs idempotency keys or deduplication.

1 The vendor's promise

A vendor promises **exactly-once delivery**: every message delivered once, never lost, never duplicated. It sounds perfect. But over an unreliable network with machines that crash, a sender can never know whether a message whose acknowledgement was lost arrived or not (lesson 2). It must either resend (risking a duplicate) or not (risking a loss).

2 Three delivery guarantees

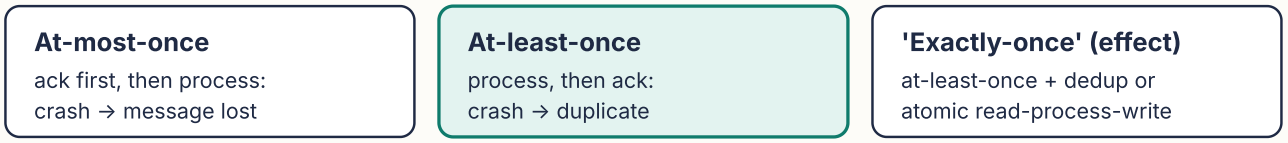


Figure 1. The order of 'process' and 'ack' decides between loss and duplicates. Exactly-once is built on top of at-least-once.

So the honest version is **exactly-once processing**, sometimes called **effectively-once**: messages may be delivered more than once, but their **effect** is applied once. Two ways to get it: make processing **idempotent** (lesson 20), or commit the output and the input position **atomically**, so either both happen or neither does.

3 How Kafka does it

Kafka combines three features: the **idempotent producer** (the broker drops duplicate retries using producer ID and sequence numbers), **transactions** (writes to several partitions commit or abort together), and committing the consumer's **offsets inside the same transaction**. Consumers that read with `isolation.level=read_committed` never see aborted output.

WORKED EXAMPLE: MERCHANT TOTALS WITH KAFKA STREAMS

A Kafka Streams app reads payment events, aggregates totals per merchant and writes results to an output topic. With `processing.guarantee=exactly_once_v2`, each batch's output records, state-store changelog updates and input offsets commit in one transaction. If the app crashes mid-batch, the transaction aborts, the batch is re-read from the last committed offsets, and the totals are not double counted. But if the same app also sent an email per payment, a crash could still send that email twice: the email server isn't part of Kafka's transaction.

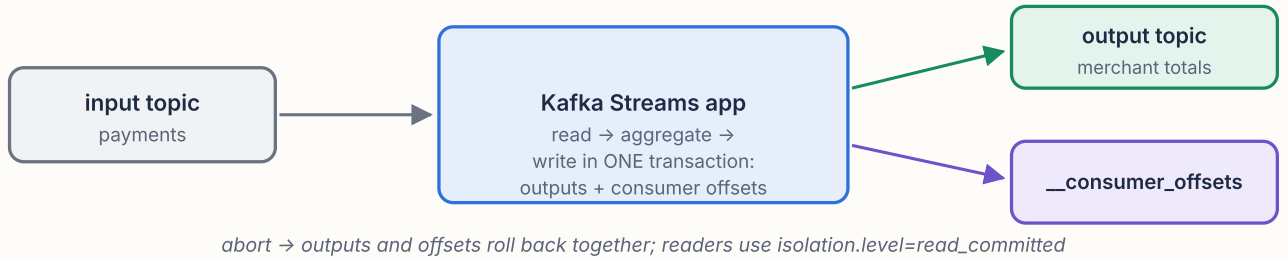


Figure 2. Outputs and input offsets in one transaction: the heart of Kafka's exactly-once.

4 Where the guarantee ends

Exactly-once holds only within the system that controls both the input position and the output. Writing to Postgres, calling a card API or sending email sits outside. There you need idempotency keys, a dedup table, or storing the input offset in the same database transaction as the result (then resuming from that stored offset).

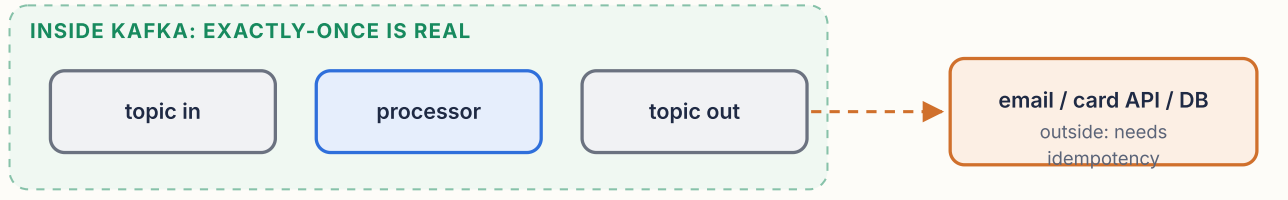


Figure 3. Exactly-once inside one system; idempotency at its edges.

COMMON MISTAKES

- Believing 'exactly-once' in a product means no duplicates anywhere.
- Turning on Kafka EOS and calling external APIs without idempotency keys.
- Downstream consumers reading with read_uncommitted and seeing aborted data.
- Paying EOS latency costs where idempotent upserts would be simpler.

5 Key terms

At-most-once: never duplicated, may be lost

At-least-once: never lost, may be duplicated

Exactly-once processing: the effect is applied once despite redelivery

Idempotent producer: broker drops duplicate retries

Kafka transaction: atomic writes to many partitions plus offsets

read_committed: consumers skip aborted records

6 Interview questions

Q1. Is exactly-once delivery possible? BASIC

Not over an unreliable network in general: a sender can't distinguish a lost message from a lost ack. Exactly-once effects are possible with retries plus deduplication or atomic commits.

Q2. How does Kafka provide exactly-once semantics? BASIC

Idempotent producers, transactions across partitions, and committing consumer offsets in the same transaction; consumers read only committed data.

Q3. Your Kafka consumer writes to Postgres. How do you avoid double writes after a crash? SCENARIO

Store the processed offset (or message ID) in the same Postgres transaction as the data and resume from it, or use idempotent upserts keyed by event ID.

Q4. A team enabled exactly-once in Kafka Streams but customers still get duplicate emails. Why? SCENARIO

The email send is outside Kafka's transaction. Make it idempotent, for example by recording sent notifications with a unique key before sending.

Q5. What's the cost of Kafka transactions? TRICKY

Extra latency (commit markers, coordinator round trips), lower throughput for small batches, and consumers waiting for open transactions before reading past them.

Q6. Why do people say 'effectively-once' instead? TRICKY

Because messages may still be delivered and processed several times internally; only the observable effect happens once.

Next: SD-H-022 · Linearizability and Serializability backendroadmap.com

Linearizability and Serializability

two words that sound alike and mean different things

Watch the 4-min video

In one line: linearizability is a recency guarantee for single operations on one object (reads see the latest completed write, in real-time order); serializability is an isolation guarantee for transactions over many objects (they behave as if run one at a time, in some order); strict serializability gives both.

1 Two words, two ideas

Linearizability and **serializability** are constantly mixed up in interviews. One is about seeing the **latest value of one object**. The other is about **transactions over many objects** not interfering. A system can have either without the other.

Linearizability (recency)
one object, single operations:
once a write completes, every later
read sees it, as if there were
one copy and one clock

Serializability (isolation)
many objects, whole transactions:
the result equals SOME serial
order of the transactions,
not necessarily real-time order

Figure 1. Linearizability is a consistency model for replicas; serializability is an isolation level for transactions.

2 Linearizability

A linearizable system behaves as if there were **one copy** of the data and every operation took effect atomically at some instant between its start and its end. So once a write has completed, any read that starts afterwards, from any client, must return that value or a newer one. It's what people mean by 'strong consistency' for a key-value store. You get it from single-leader systems that read from the leader (with care, lesson 9), consensus stores like etcd, and Spanner. Asynchronous replicas and most quorum setups don't provide it.

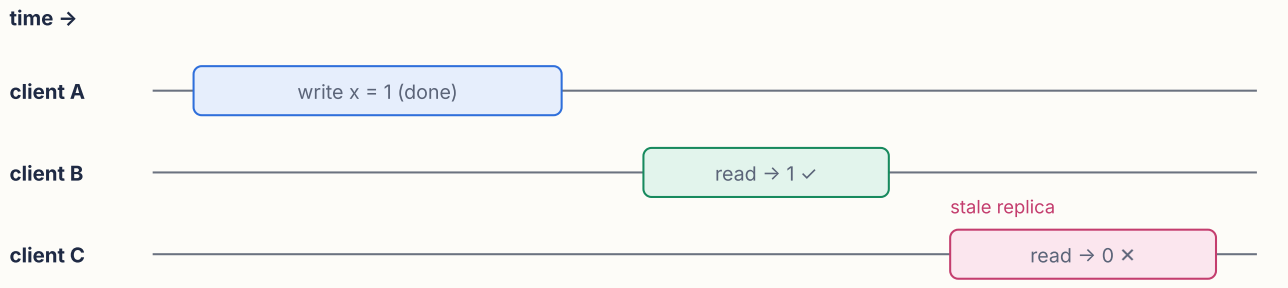


Figure 2. Not linearizable: C's read starts after B already saw x = 1, yet returns 0.

3 Serializability

A serializable database guarantees that concurrent transactions produce the same result as if they had run **one after another, in some order**. It prevents anomalies like lost updates, write skew and phantoms. It doesn't promise which order, or that the order matches real time: a transaction that committed earlier could be ordered after one that started later. Implementations: two-phase locking, serializable snapshot isolation (Postgres SERIALIZABLE), or actual serial execution.

WORKED EXAMPLE: UNIQUE USERNAMES

Two people try to register the name `alice` at the same moment, on different servers. The check 'is alice taken?' and the insert happen in a transaction. Under **snapshot isolation**, both transactions read 'not taken' and both insert: write skew, two Alices. Under **serializability**, one of them aborts. In a replicated system, you also need the check to see the latest committed data: a linearizable read, or a unique constraint enforced by a single leader. The simplest robust answer: a unique index on `username`, enforced in one place.

4 Both at once

Strict serializability combines them: transactions are serializable, and the serial order respects real time. Spanner, CockroachDB (close to it) and FaunaDB offer this; it costs coordination, such as TrueTime commit waits or consensus round trips.

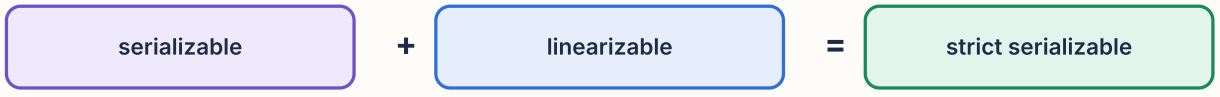


Figure 3. The strongest common guarantee, and the most expensive.

COMMON MISTAKES

- Saying 'serializable' when you mean reads see the latest data.
- Assuming the default isolation (often read committed) is serializable.
- Assuming quorum reads are linearizable.
- Checking uniqueness in application code without a constraint.

5 Key terms

Linearizability: single-object recency; real-time order

Strict serializability: serializable + real-time order

Write skew: two transactions read, then write different rows, breaking a rule

Serializability: transactions equal some serial order

Snapshot isolation: reads a consistent snapshot; allows write skew

6 Interview questions

Q1. What's the difference between linearizability and serializability? BASIC

Linearizability: single operations on one object appear instantaneous and in real-time order, so reads see the latest write. Serializability: transactions on many objects appear to run one at a time, in some order.

Q2. Is a single-leader database with async replicas linearizable? BASIC

Only for reads served by the leader. Reads from asynchronous replicas can be stale, which violates linearizability.

Q3. Two users claim the same username concurrently. How do you prevent duplicates? SCENARIO

Enforce a unique constraint in one strongly consistent place (a single leader or a consensus store); serializable transactions also work; app-level checks alone don't.

Q4. Which guarantee does a distributed lock service need? SCENARIO

Linearizability: every client must see the latest lock state, or two could think they hold the lock.

Q5. Can a system be serializable but not linearizable? TRICKY

Yes: transactions may be ordered in a valid serial order that doesn't match real time, for example reading from a slightly stale but consistent snapshot.

Q6. Why does CAP's 'C' refer to linearizability? TRICKY

CAP's proof uses linearizable reads and writes on a register: during a partition, you can't keep that recency guarantee and answer every request.

Next: SD-I-001 · Why Messaging

Why Messaging

stop making every service wait for every other

Watch the 4-min video

In one line: a message broker lets a service publish work or events and move on, while consumers process them independently; you gain decoupling, buffering for spikes, resilience to slow or failed consumers and easy fan-out, and you pay with eventual consistency, duplicates and harder debugging.

1 Checkout is only as fast as the slowest call

When an order is placed, the order service calls the email service, the analytics service and the inventory service, one after another. If email is slow, checkout is slow. If analytics is down, checkout fails, even though the customer doesn't care about analytics. Every new feature adds another call to the chain.

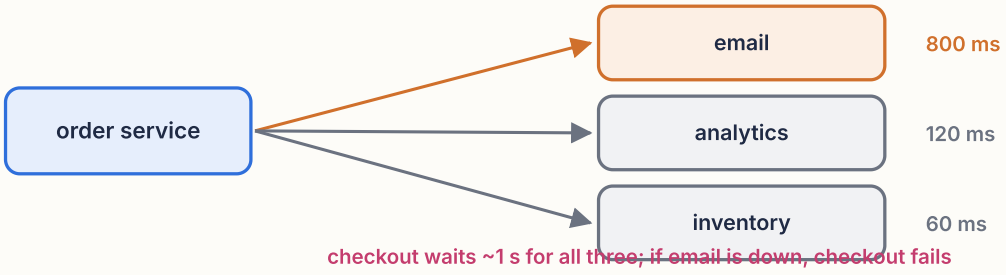


Figure 1. Synchronous calls: latencies add up, and any failure fails the whole request.

2 Put a broker in between

With **messaging**, the order service publishes one message, `OrderPlaced`, to a **broker** (Kafka, RabbitMQ, SQS, Pub/Sub...) and returns to the customer. The broker stores the message durably, and each interested service **consumes** it when it's ready. The producer doesn't know or care who the consumers are.

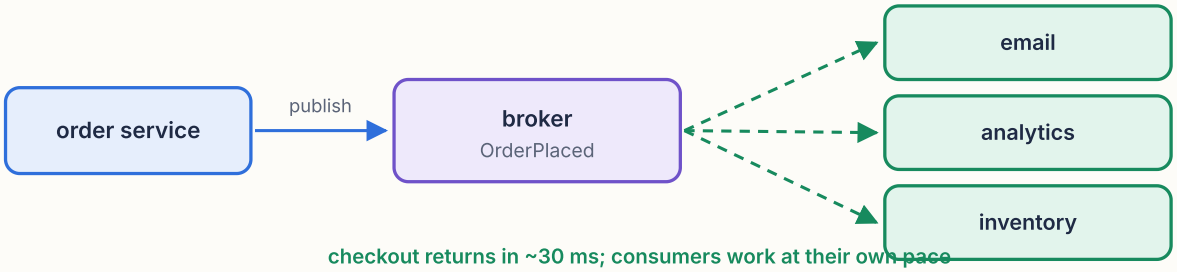


Figure 2. Asynchronous messaging: the producer publishes once; consumers process independently.

WORKED EXAMPLE: ORDERPLACED

The order service saves the order and publishes one event, `OrderPlaced`. The email service consumes it and sends a confirmation. Analytics records it. Inventory reserves the stock. On Black Friday, orders arrive ten times faster than email can send; the messages queue up and email catches up over the next twenty minutes, while checkout stays fast. When the email service is deployed or crashes, nothing upstream notices. Next month a fraud service subscribes to the same event, with no change to the order service.

3 What you gain



Figure 3. The four big benefits of messaging.

4 What it costs

- **Eventual consistency:** the email arrives a bit later; the stock is reserved after the response. Design the UX for it.
- **Duplicates and ordering:** most brokers deliver at least once; consumers must be idempotent (lesson 20) and handle order carefully (lesson 29).
- **Operations:** a broker to run, monitor lag, dead-letter queues (lesson 31).
- **Debugging:** a flow spread across services needs correlation IDs and tracing (lesson 40).

Keep synchronous calls when the caller needs the answer now (checking a price, authorising a payment) and use messages for work that can happen after the response.

COMMON MISTAKES

- Saving to the database and then publishing, so a crash in between loses the event (use an outbox, lesson 37).
- Using messages for request/response where the user waits anyway.
- No monitoring of consumer lag.

5 Key terms

Message broker: stores and delivers messages between services

Consumer / subscriber: receives and processes messages

Consumer lag: how far behind a consumer is

Producer / publisher: sends messages

Asynchronous: the sender doesn't wait for processing

Fan-out: one message to many consumers

6 Interview questions

Q1. Why introduce a message broker between services? BASIC

To decouple them: the producer publishes and returns, consumers process independently, spikes are buffered, and a slow or failed consumer doesn't break the producer.

Q2. What are the downsides of asynchronous messaging? BASIC

Eventual consistency, duplicate and out-of-order messages, more infrastructure to run, and harder end-to-end debugging.

Q3. Checkout calls five services and is slow. What do you move to messaging? SCENARIO

Everything the customer doesn't need before the response: emails, analytics, loyalty points, search indexing. Keep payment authorisation and stock checks synchronous if the response depends on them.

Q4. An email provider outage takes checkout down. Fix it. SCENARIO

Publish OrderPlaced and let the email service consume it; checkout succeeds and emails go out when the provider recovers.

Q5. When is synchronous better than messaging? TRICKY

When the caller needs the result to continue, like validating a coupon or authorising a card, or when the extra latency and complexity aren't justified.

Q6. What happens if the producer crashes right after saving the order but before publishing? TRICKY

The event is lost and consumers never hear about the order. Use the transactional outbox so saving and publishing are tied together.

Next: SD-I-002 · Message Queues vs Event Streams backendroadmap.com

Message Queues vs Event Streams

hand out work, or keep a log of what happened

Watch the 4-min video

In one line: a queue hands each message to one worker and deletes it when done, ideal for jobs; a stream is a durable, ordered log that many consumer groups read independently and can replay, ideal for events; many systems use both.

1 They are not interchangeable

RabbitMQ, SQS, Kafka, Kinesis: they all move messages from producers to consumers, so they seem interchangeable. They aren't. Some are **queues**, which hand out work and forget it once it's done. Others are **streams** (logs), which remember everything for a retention period and let anyone read from any point.

2 Queues: distribute work

Producers add messages; a pool of **competing consumers** takes them, so each message goes to one worker. The worker acknowledges after processing and the message is deleted; if it fails, the message becomes visible again and is retried, and after too many attempts it goes to a **dead-letter queue**. Queues track each message individually, which makes retries, delays and priorities easy. Scaling means adding workers.

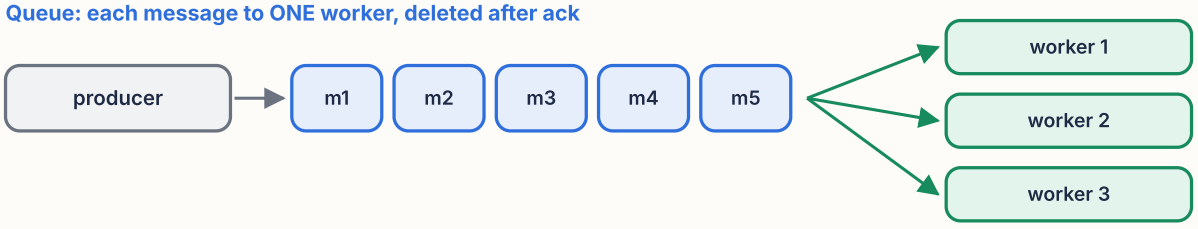


Figure 1. A queue load-balances messages across workers.

3 Streams: a shared log of events

A stream appends messages to an ordered, partitioned log that's kept for days or forever. Each **consumer group** tracks its own **offset** (position). Reading doesn't remove anything, so many independent groups read the same events, and a new service, or a fixed one, can **replay** history from the start. Ordering is guaranteed within a partition (lesson 26).

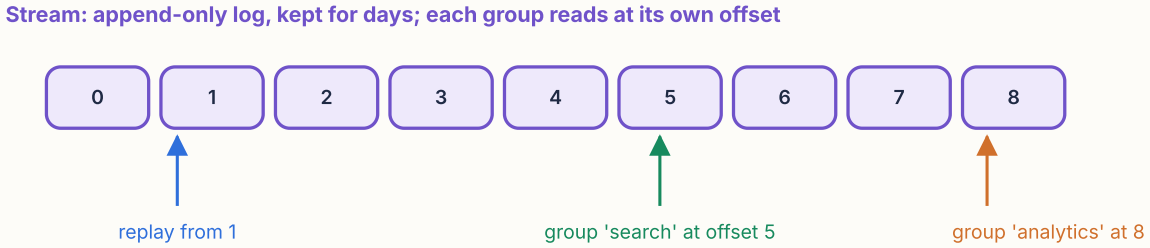


Figure 2. A stream keeps the log; each consumer group has its own position and can rewind.

	Queue	Stream
examples	RabbitMQ, SQS, Service Bus	Kafka, Kinesis, Pulsar, Event Hubs
after reading	message deleted	message kept (retention)
readers	competing workers share	many groups, each sees all
replay	no	yes, rewind the offset
ordering	best effort (FIFO options)	per partition
per-message	ack, retry, delay, DLQ	commit an offset

Figure 3. Queue vs stream at a glance.

WORKED EXAMPLE: USING BOTH

Email-sending jobs go to **SQS**: each job is picked up by one worker, retried a few times on failure, and moved to a dead-letter queue if it keeps failing; nobody needs it afterwards. Order events go to **Kafka**: the search indexer, analytics and the recommendations team each read every event in their own consumer group, and when analytics ships a bug fix they reset their offset to last Monday and reprocess a week of orders.

4 How to choose

- A **task** to be done once by someone (send email, resize image)? Queue.
- A **fact** that many services care about, or that you may want to replay or audit? Stream.

- Need per-message retries, delays and priorities? Queues make that easy; with streams you build it (lesson 32).
- Very high throughput and long retention? Streams.

COMMON MISTAKES

- Using Kafka as a job queue and fighting per-message retries and head-of-line blocking.
- Using a queue for events several teams need, then copying messages to many queues by hand.
- Expecting replay from a queue after messages were deleted.

5 Key terms

Queue: messages consumed once and deleted

Competing consumers: workers share one queue's messages

Offset: position in a stream partition

Stream / log: append-only, retained, replayable

Consumer group: readers that share one offset per partition

Retention: how long a stream keeps messages

6 Interview questions

Q1. What's the main difference between a queue and a stream? BASIC

A queue delivers each message to one consumer and deletes it after ack; a stream keeps an ordered log that many consumer groups read independently and can replay.

Q2. When would you pick RabbitMQ or SQS over Kafka? BASIC

For task distribution with per-message acks, retries, delays, priorities and dead-lettering, at moderate throughput, where nobody needs replay.

Q3. Order events must reach search, analytics and email. Queue or stream? SCENARIO

A stream: publish once, each service consumes in its own consumer group, and new consumers can be added or replay history without touching the producer.

Q4. A bug corrupted a week of analytics data. How does a stream help? SCENARIO

Fix the bug, reset the analytics consumer group's offsets to the start of the week, and reprocess the retained events.

Q5. Can Kafka act as a queue? TRICKY

Partly: a consumer group spreads partitions across consumers, but parallelism is capped by the partition count, one slow message blocks its partition, and per-message retry needs extra topics.

Q6. Why is ordering per partition, not per topic? TRICKY

Partitions let the log scale across brokers and consumers; a single total order would force everything through one sequence and one consumer.

Next: SD-I-003 · Producers and Consumers backendroadmap.com

Producers and Consumers

sending and receiving without losing messages

Watch the 4-min video

In one line: producers decide durability (acks, retries, idempotence, batching) and consumers decide correctness (when to commit, how many to fetch, how long processing may take); a safe default is acks=all with idempotence on the producer, and committing offsets only after processing on the consumer.

1 Two ways to lose data

Your producer sends a payment event and gets a success back. Then the broker leader crashes, and the event is gone. Your consumer, meanwhile, grabbed a thousand messages, auto-committed their offsets, and crashed halfway through; the other five hundred are never processed. Both sides have settings that decide whether messages survive.

2 Producer: durability

The **acks** setting says when the broker answers 'success'.

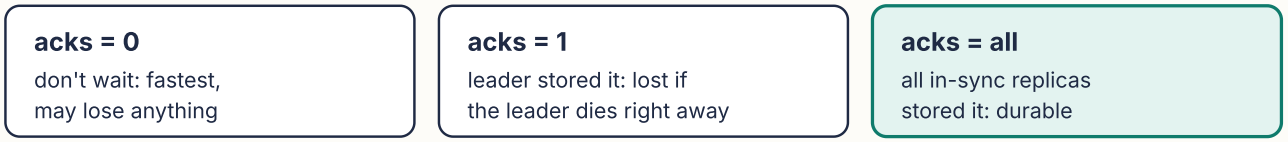


Figure 1. acks=all with min.insync.replicas = 2 (lesson 28) is the durable choice.

Retries handle transient errors, but a retry after a lost acknowledgement could write the message twice; **idempotence** makes the broker discard such duplicates using a producer ID and sequence numbers (default in recent Kafka clients). **Batching** (`linger.ms`, `batch.size`) and **compression** trade a few milliseconds of latency for much higher throughput. The **key** decides the partition, and so the ordering (lesson 29).

WORKED EXAMPLE: A PYTHON PRODUCER WITH AIOKAFKA

The payment service creates the producer with `acks='all'`, idempotence enabled, a 10 ms linger and lz4 compression, and sends each event keyed by order ID. `send_and_wait` returns only after all in-sync replicas have the event, so the API can safely tell the client the payment was recorded. If the leader dies a moment later, a follower that already has the event becomes leader. Throughput stays high because events from concurrent requests travel in shared batches.

```
producer = AIOKafkaProducer(
    bootstrap_servers=BROKERS,
    acks='all',                # wait for in-sync replicas
    enable_idempotence=True,   # no duplicates from retries
    linger_ms=10,              # batch for 10 ms
    compression_type='lz4')
await producer.send_and_wait('payments', value=event, key=order_id)
```

Figure 2. A durable producer configuration.

3 Consumer: correctness

A consumer **polls** batches and **commits offsets** to record progress. Auto-commit commits on a timer, possibly before processing finished: a crash then **loses** messages. The safe pattern: disable auto-commit, process the batch, then commit (at-least-once), and make processing idempotent so the occasional re-read is harmless (lesson 30).



Figure 3. Commit after processing. Duplicates are possible; losses are not.

Watch the time budget: if one poll's batch takes longer than `max.poll.interval.ms`, the broker assumes the consumer died and rebalances its partitions (lesson 27). Use a smaller `max.poll.records` or hand slow work to another pool.

COMMON MISTAKES

- acks=1 for payment data.
- Fire-and-forget sends without checking the result or using a callback.
- Auto-commit with slow or failing processing.
- Huge batches that exceed the poll interval and cause rebalances.

4 Key terms

acks: how many replicas must store a write

linger.ms / batch.size: how long and how much to batch

Auto-commit: periodic commit, possibly before processing

Idempotent producer: broker drops duplicate retries

Offset commit: recording the consumer's progress

max.poll.interval.ms: max time between polls before a rebalance

5 Interview questions

Q1. What does acks=all mean? BASIC

The leader replies success only after all in-sync replicas have the record, so it survives the loss of the leader.

Q2. Why commit offsets after processing? BASIC

If the consumer crashes after committing but before processing, those messages are skipped forever. Committing after gives at-least-once.

Q3. A payment event was acknowledged and later disappeared. What settings do you check? SCENARIO

Producer acks (should be all), topic replication factor and min.insync.replicas, and unclean leader election (should be off).

Q4. Consumers rebalance every few minutes under load. Why? SCENARIO

Processing a poll batch exceeds max.poll.interval.ms, so the group thinks the consumer died. Reduce max.poll.records, speed up processing, or increase the interval.

Q5. Why might enabling retries create duplicates, and how does Kafka prevent it? TRICKY

A retry after a lost ack rewrites a stored record. The idempotent producer attaches sequence numbers per partition so the broker drops the duplicate.

Q6. How does batching improve throughput? TRICKY

Many records share one request and one disk write, and compression works better on bigger batches, at the cost of a few milliseconds of latency.

Next: SD-I-004 · Topics, Partitions and Offsets backendroadmap.com

Topics, Partitions and Offsets

how a stream scales across machines

Watch the 4-min video

In one line: a topic is split into partitions, each an ordered append-only log stored on (and replicated across) brokers; records with the same key go to the same partition, offsets number records within a partition, and the partition count sets the maximum consumer parallelism.

1 One topic, many machines

A Kafka topic called `orders` receives 50,000 events per second. No single machine could store and serve that alone. So Kafka splits each **topic** into **partitions**, spread across **brokers**. Each partition is an independent, ordered, append-only log; together they hold the topic's data and its load.

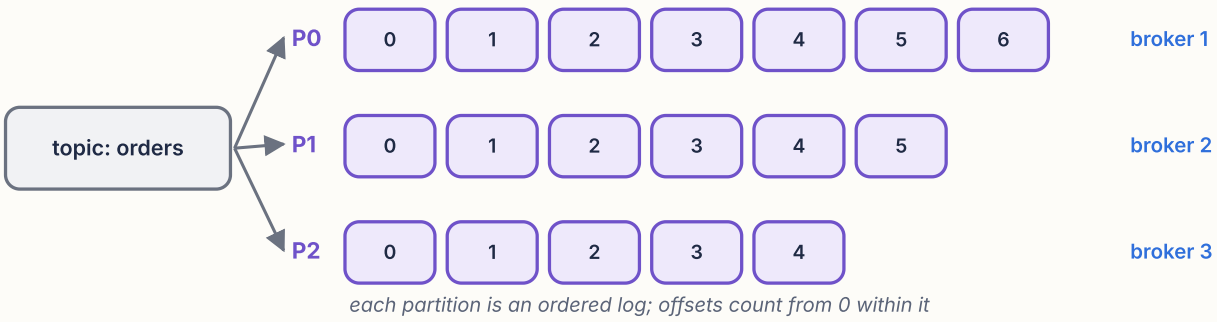


Figure 1. Three partitions of one topic on three brokers. Each grows independently.

2 Offsets

Within a partition, every record gets an **offset**: 0, 1, 2... Offsets are only meaningful inside their partition; there is no global order across partitions. A consumer's progress is a set of (partition, offset) pairs, and replay means moving those offsets back. Records stay until **retention** (time or size) removes old segments, or **compaction** keeps only the latest record per key.

3 Keys decide the partition

The producer picks a partition: with a key, it's `hash(key) % partitions`, so all records with the same key land in the same partition, in order. Without a key, records are spread (sticky batches in recent clients) for even load but with no ordering between them.

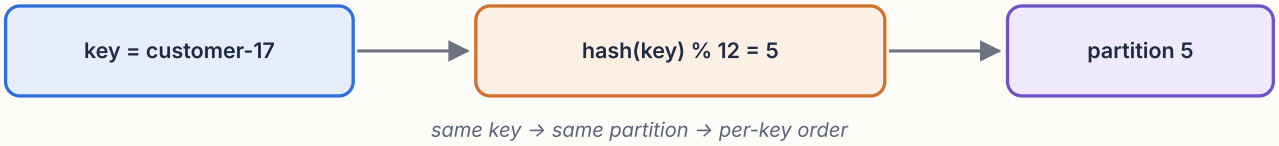


Figure 2. Keying by customer keeps each customer's events in order.

WORKED EXAMPLE: SIZING THE ORDERS TOPIC

The orders topic needs 50 MB/s at peak. Tests show one partition handles about 10 MB/s on the producer side, and one consumer about 5 MB/s of processing. Consumers are the bottleneck: 50 ÷ 5 = 10 partitions, rounded up to 12 for growth and an even spread over 3 brokers. Records are keyed by `customer_id`, so one customer's orders stay in order. A hot key (one huge customer) would still land in one partition, which is a limit to plan for.

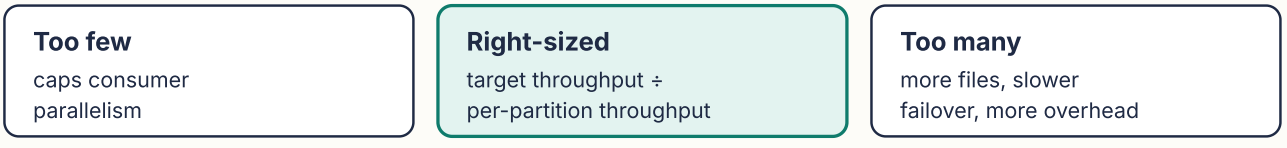


Figure 3. Choosing a partition count.

4 Changing partition counts

You can add partitions later, but the key-to-partition mapping changes for many keys, so per-key order is broken during the transition. Plan the count up front for a year or two of growth. You can't remove partitions without recreating the topic.

COMMON MISTAKES

- One partition 'to keep order', which caps throughput at one consumer.
- Thousands of partitions per topic 'just in case'.
- No key, then expecting per-entity ordering.
- Adding partitions to a keyed topic without considering ordering.

5 Key terms

Topic: named stream of records

Partition: ordered log, unit of parallelism and placement

Offset: record position within a partition

Broker: server storing partitions

Retention: how long or how much data is kept

Log compaction: keep only the latest record per key

6 Interview questions

Q1. Why are Kafka topics partitioned? BASIC

To spread storage and load across brokers and let several consumers read in parallel; each partition is an ordered log.

Q2. Is order guaranteed across a whole topic? BASIC

No, only within a partition. Use a key so related records share a partition.

Q3. How many partitions for a topic needing 100 MB/s, if one consumer handles 10 MB/s? SCENARIO

At least 10, plus headroom, say 12 to 16, so you can add consumers up to the partition count.

Q4. One partition has far more traffic than others. Why, and what do you do? SCENARIO

A hot key. Use a better key, add a suffix to spread a hot key (losing strict order for it), or handle that key separately.

Q5. What happens to ordering if you add partitions? TRICKY

The hash-to-partition mapping changes for many keys, so new records for a key may go to a different partition and be processed out of order with older ones.

Q6. What's log compaction good for? TRICKY

Topics that represent the latest state per key, like a changelog or configuration; consumers can rebuild current state without keeping every historical update.

Next: SD-I-005 · Consumer Groups and Rebalancing backendroadmap.com

Consumer Groups and Rebalancing

sharing a topic's partitions between consumers

Watch the 4-min video

In one line: consumers with the same group ID split a topic's partitions so each partition is read by one member; scaling past the partition count doesn't help, and every join, leave or timeout triggers a rebalance, so use cooperative rebalancing, static membership and well-tuned timeouts.

1 Scaling that made it worse

Your consumers can't keep up and lag is climbing. You scale from four instances to twelve. Lag keeps growing, and every deploy makes things worse as the whole group stops to **rebalance**. To fix it you need to know how consumer groups divide the work.

2 How a consumer group works

Consumers that share a **group ID** form a group. The broker's **group coordinator** assigns each partition to exactly one member, and each member reads its partitions and commits offsets for the group. Another group reading the same topic gets its own full copy of the stream. Inside a group, the **partition count caps parallelism**.

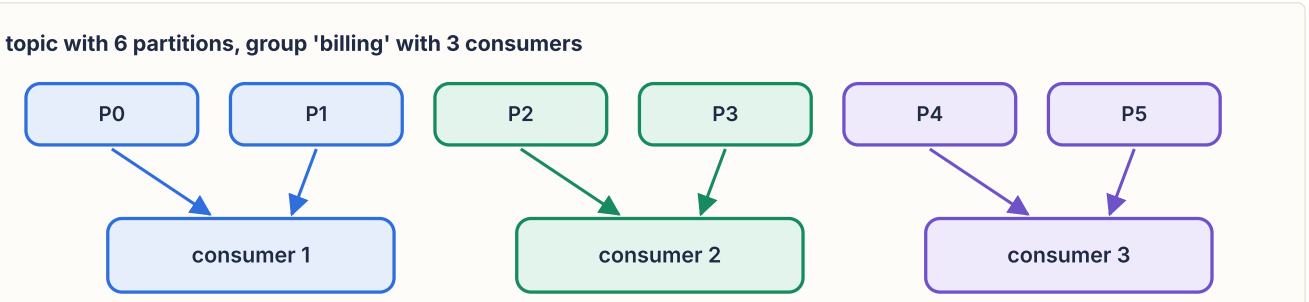


Figure 1. Each partition belongs to one consumer of the group; one consumer can own several partitions.

24 partitions, 12 consumers → 2 each ✓

24 partitions, 30 consumers → 6 consumers sit idle ✕

a partition is read by at most one consumer in a group, so partitions cap parallelism

Figure 2. Extra consumers beyond the partition count do nothing.

3 Rebalancing

When a member joins, leaves, crashes, or misses its deadline (no heartbeat within `session.timeout.ms`, or no poll within `max.poll.interval.ms`), the group **rebalances**: partitions are reassigned. With the classic **eager** protocol, every consumer stops, gives up all its partitions and waits for the new assignment. A rolling deploy of 12 instances can cause a dozen stop-the-world pauses.

Eager rebalance
everyone stops and gives up all partitions, then gets new ones: the whole group pauses

Cooperative (incremental)
only partitions that must move are revoked; the rest keep working. Plus static membership on restarts

Figure 3. Eager vs cooperative rebalancing.

Modern fixes: the **cooperative sticky** assignor moves only the partitions that need to move; **static membership** (`group.instance.id`) lets a restarting instance come back without triggering a rebalance if it returns within the session timeout; and Kafka 4's new consumer group protocol moves assignment to the broker and makes it incremental.

WORKED EXAMPLE: FOUR CONSUMERS, TWENTY-FOUR PARTITIONS

A group of four consumers reads a topic with 24 partitions, six each, and lag is growing. Scaling to 12 instances gives two partitions each and roughly triples throughput, but going to 30 would leave six idle. Each processing batch takes up to 6 minutes because of a slow downstream call, longer than the 5-minute `max.poll.interval.ms`, so consumers keep getting kicked out, causing rebalance storms. The fix: `max.poll.records` from 500 to 100, the cooperative sticky assignor, static membership for Kubernetes pods, and a separate thread pool for the slow call. Lag drains within the hour.

COMMON MISTAKES

- More consumers than partitions, expecting more throughput.
- Slow processing in the poll loop, exceeding max.poll.interval.ms.
- Eager rebalancing with frequent deploys or autoscaling.
- Two unrelated services sharing a group ID by accident.

4 Key terms

Consumer group: consumers sharing a group ID and the partitions

Rebalance: reassigning partitions when membership changes

Static membership: stable member ID avoids rebalances on restart

Group coordinator: broker that manages membership and offsets

Cooperative sticky: incremental assignor that moves few partitions

Consumer lag: latest offset minus committed offset

5 Interview questions

Q1. How do consumer groups share work? BASIC

Each partition is assigned to exactly one consumer in the group; consumers can own several partitions. Different groups each read the whole topic.

Q2. What triggers a rebalance? BASIC

A consumer joining or leaving, a crash, missed heartbeats beyond the session timeout, exceeding max.poll.interval.ms, or a change in subscribed partitions.

Q3. You doubled consumers but lag didn't drop. Why? SCENARIO

There may be no more partitions than consumers, the bottleneck may be downstream, or rebalances keep pausing the group. Check partition count, processing time and rebalance frequency.

Q4. Every Kubernetes deploy pauses consumption for a minute. Fix it. SCENARIO

Use the cooperative sticky assignor, static membership with group.instance.id, and graceful shutdown that leaves the group cleanly.

Q5. What's the difference between session.timeout.ms and max.poll.interval.ms? TRICKY

Session timeout covers heartbeats from the background thread (is the process alive?); max poll interval covers progress (is the poll loop still running?).

Q6. Can two consumers in one group read the same partition at once? TRICKY

Not normally; during an eager rebalance there's a gap, and with cooperative rebalancing a partition is revoked before it's reassigned, so ownership doesn't overlap.

Next: SD-I-006 · Kafka Replication and Durability backendroadmap.com

Kafka Replication and Durability

making acknowledged events survive broker loss

Watch the 4-min video

In one line: each partition has a leader and follower replicas; the in-sync replica set (ISR) holds followers that are caught up, and with replication factor 3, min.insync.replicas 2, acks=all and unclean leader election off, an acknowledged write survives the loss of any one broker.

1 Acknowledged, then gone

Your Kafka cluster loses a broker, and with it the last few seconds of payment events, events the producers had been told were saved. Kafka can be extremely durable, but only if replication is configured for it.

2 Leaders, followers and the ISR

Each partition has a **replication factor** (RF): the number of copies. One copy is the **leader**, which handles writes (and normally reads); the others are **followers** that fetch from the leader. Followers that are caught up within `replica.lag.time.max.ms` form the **in-sync replica set** (ISR). If the leader fails, the controller elects a new leader from the ISR, so it already has every committed record.

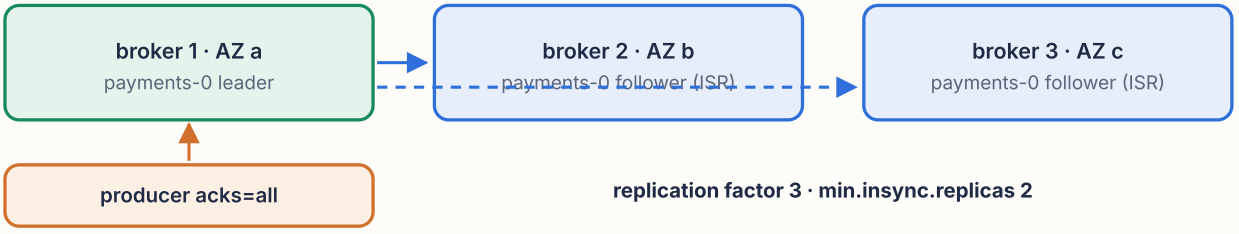


Figure 1. One partition, three replicas in three zones. The leader takes writes; followers copy them.

3 When is a write durable?

With `acks=all`, the leader acknowledges only after every replica currently in the ISR has the record. But if the ISR shrinks to just the leader, 'all' means one copy. `min.insync.replicas` sets a floor: with 2, a write is rejected (`NotEnoughReplicas`) rather than acknowledged with a single copy. **Unclean leader election** lets an out-of-sync replica become leader to restore availability, losing records; keep it off for data that matters.

replication.factor = 3	three copies, one per zone
min.insync.replicas = 2	acks=all needs at least 2 copies
acks = all	producer waits for the in-sync replicas
unclean.leader.election = false	never elect an out-of-date replica
rack awareness	spread replicas across availability zones

Figure 2. The durable configuration.

WORKED EXAMPLE: THE PAYMENTS TOPIC

Replication factor 3, one replica in each of three availability zones (rack awareness), `min.insync.replicas=2`, producers with `acks=all` and idempotence, unclean leader election disabled. Zone B goes down: its follower drops out of the ISR, two copies remain, writes continue. Then the leader in zone A crashes: the follower in zone C, which is in sync, becomes leader with every acknowledged record. Writes pause (one replica left, below the minimum of 2), but nothing acknowledged is ever lost.



Figure 3. What happens as failures pile up with RF 3 and min.insync.replicas 2.

4 Other durability details

- Kafka acknowledges after writing to the OS page cache, not after fsync; replication across zones is what protects you against machine loss.
- Under-replicated partitions are the key alert: they mean you're one failure away from unavailability.
- Cross-region disaster recovery uses MirrorMaker 2 or cluster linking, which is asynchronous.

COMMON MISTAKES

- RF 3 but min.insync.replicas 1, so acks=all can mean one copy.
- All replicas in one availability zone.
- Unclean leader election turned on for financial data.
- Not alerting on under-replicated partitions or a shrinking ISR.

5 Key terms

Replication factor: number of copies of a partition

ISR: replicas that are caught up

Unclean leader election: electing an out-of-sync replica, losing data

Leader / follower: the writable copy and the copies that fetch from it

min.insync.replicas: minimum ISR size for acks=all writes

Under-replicated partition: ISR smaller than RF

6 Interview questions

Q1. What is the ISR? BASIC

The in-sync replicas: the leader plus the followers that have caught up recently. Only ISR members can be elected leader in a clean election.

Q2. Which settings make Kafka writes durable? BASIC

Replication factor 3, min.insync.replicas 2, producer acks=all, unclean leader election off, and replicas spread across zones.

Q3. With RF 3 and min ISR 2, two brokers die. What happens? SCENARIO

Writes to affected partitions fail with NotEnoughReplicas until a replica returns; reads of committed data still work from the remaining replica. No acknowledged data is lost.

Q4. Acknowledged events vanished after a broker crash. What went wrong? SCENARIO

Probably acks=1 or 0, min.insync.replicas 1 with a shrunk ISR, or unclean leader election promoting a lagging replica.

Q5. Does Kafka fsync every write? TRICKY

Not by default; it relies on replication to several machines for durability, with flushing left to the OS. Correlated power loss across all replicas is the residual risk.

Q6. Why not set min.insync.replicas equal to RF? TRICKY

Then any single broker outage or slow follower blocks all writes; RF minus one balances durability and availability.

Next: SD-I-007 · Ordering Guarantees

Ordering Guarantees

keeping events in the order they happened

Watch the 4-min video

In one line: brokers usually order messages only within a partition or queue, so put events that must stay in order under the same key, keep the producer idempotent, preserve per-key order inside consumers, and make handlers tolerant with version numbers when order can still break.

1 The ghost account

A customer creates an account, updates their email, then deletes the account. Your consumer processes the delete first, then the update, and recreates a ghost record. Order matters, and messaging systems only promise it under specific conditions.

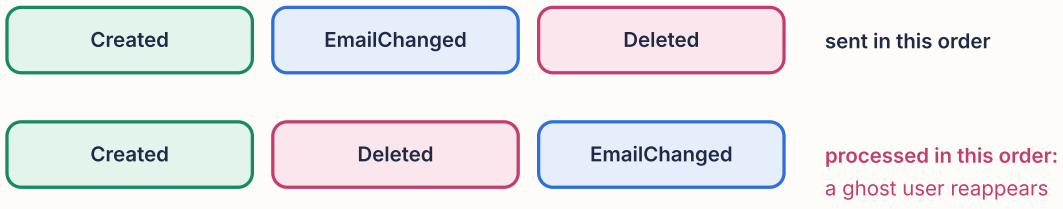


Figure 1. The same three events, processed in the wrong order.

2 What brokers promise

Kafka, Kinesis, Pulsar: order within one partition (shard). **SQS standard:** best effort, no guarantee; **SQS FIFO:** order per message group ID. **RabbitMQ:** order per queue for a single consumer, broken by competing consumers and requeues. **Azure Service Bus:** order per session. There is almost never a global order, and you rarely need one: you need **per-entity** order (per user, per order, per account).



Figure 2. Four ways order gets lost, even on a broker that orders partitions.

3 Getting per-key order end to end

- **Producer:** use the entity ID as the key, so its events share a partition. Enable idempotence so retries can't reorder batches (Kafka keeps order with up to 5 in-flight requests when idempotent).
- **Broker:** don't change the partition count of a keyed topic casually (lesson 26).
- **Consumer:** if you process in parallel, parallelise **by key**, never across a key's events. Libraries like Confluent's parallel consumer do this.
- **Retries:** a message moved to a retry topic falls out of order; for strict cases, block that key until it succeeds.

WORKED EXAMPLE: USER EVENTS

All events for a user are published to the `user-events` topic, keyed by user ID, with an idempotent producer. Every event for user 42 goes to partition 7, in order. The consumer runs 16 worker threads but routes each record to a worker by `hash(user_id)`, so user 42's events are handled by one worker, one at a time, while other users run in parallel. Each event carries a per-user `version`; the handler ignores any event older than the version already stored, as a final safety net.

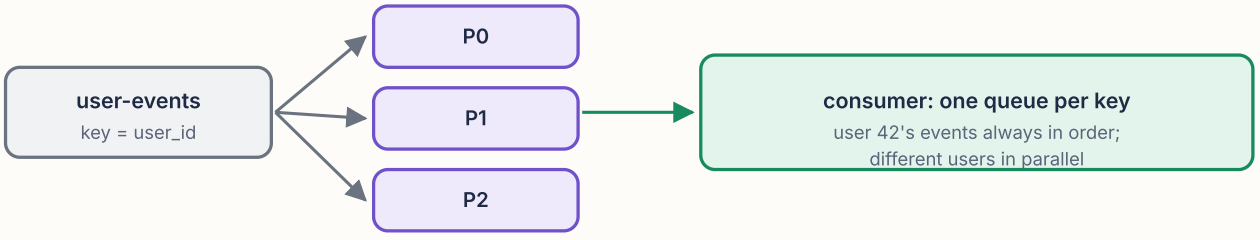


Figure 3. Key by entity, and keep per-key order inside the consumer too.

4 Designing for disorder

When order can't be guaranteed, make handlers tolerant: store a **version** or sequence per entity and drop stale events; use **tombstones** so a late update can't resurrect a deleted record; or send **full state** rather than deltas, so the latest version simply wins.

COMMON MISTAKES

- No key, or a random key, for events that must stay ordered.
- A thread pool consuming one partition without per-key routing.
- One partition for everything 'to be safe'.
- Assuming SQS standard or RabbitMQ with many consumers keeps order.

5 Key terms

Per-partition order: order guaranteed only within a partition

Message group / session: ordering unit in SQS FIFO / Service Bus

Entity version: sequence number to detect stale events

Partition key: decides which partition a record goes to

Head-of-line blocking: one stuck message delays those behind it

6 Interview questions

Q1. Does Kafka guarantee ordering? BASIC

Only within a partition. Records with the same key go to the same partition, so per-key order holds if producers and consumers preserve it.

Q2. How do you keep a user's events in order? BASIC

Key them by user ID, use an idempotent producer, and process one user's events sequentially in the consumer.

Q3. You added a thread pool to a consumer and now see out-of-order updates. Fix it. SCENARIO

Route records to threads by key so each key is processed by one thread in order, and commit offsets only up to the lowest completed record.

Q4. Ordering can't be guaranteed across two topics. How do you stay correct? SCENARIO

Version every entity, ignore events with a lower version than stored, use tombstones for deletes, or merge into one topic keyed by entity.

Q5. Can producer retries reorder messages in Kafka? TRICKY

Without idempotence, with several in-flight requests, a failed batch retried after a later batch succeeded lands out of order. Idempotence fixes this.

Q6. Why is global ordering usually a bad requirement? TRICKY

It forces one partition and one consumer, so throughput is capped at one machine; per-entity ordering almost always suffices.

Next: SD-I-008 · Delivery Semantics backendroadmap.com

Delivery Semantics

at-most-once, at-least-once, exactly-once

Watch the 4-min video

In one line: whether a consumer commits its position before or after processing decides between losing messages (at-most-once) and repeating them (at-least-once); most systems choose at-least-once with idempotent handlers, and get exactly-once effects by committing results and offsets atomically.

1 One decision

A consumer reads a message, writes the result to the database, and crashes. On restart, does it read that message again, or skip it? The answer depends on one decision: **when** the consumer records its progress (commits the offset or acknowledges the message) relative to doing the work.

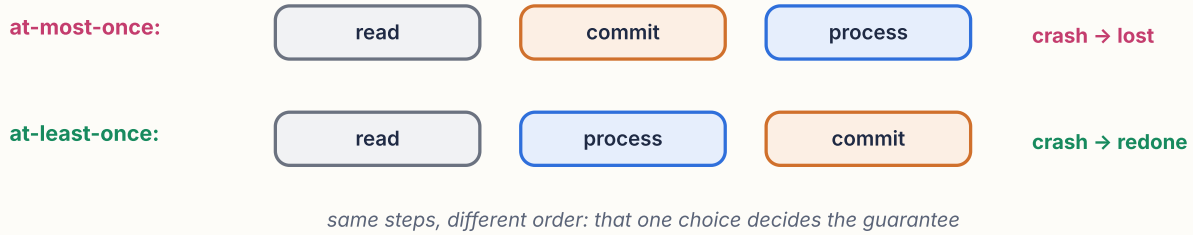


Figure 1. Commit before processing loses messages on a crash; commit after processing repeats them.

2 The three semantics

At-most-once: commit first, then process. A crash loses the message, but it's never handled twice. **At-least-once:** process first, then commit. A crash means the message is processed again. **Exactly-once:** the effect happens once. You get it by making processing idempotent, or by committing the result and the offset in one atomic step (lesson 21). The producer side has the same choice: no retries (at-most-once) or retries (at-least-once, made safe by an idempotent producer).

WORKED EXAMPLE: ORDERS INTO POSTGRES

A consumer reads order events and writes them into Postgres. It disables auto-commit. For each batch, it upserts the orders by `order_id` in one database transaction, then commits the Kafka offsets. If it crashes after the database commit but before the offset commit, the batch is re-read on restart, and the upserts change nothing: at-least-once delivery with an idempotent sink, so exactly-once effect. A stricter variant stores the last processed offset per partition in the same Postgres transaction and seeks to it on startup.

```
consumer = KafkaConsumer('orders', enable_auto_commit=False, ...)
for batch in consumer:
    with db.transaction():
        for r in batch:
            db.execute('INSERT ... ON CONFLICT (order_id) DO UPDATE ...', r)
    consumer.commit()        # after the DB commit
```

Figure 2. At-least-once with an idempotent upsert: duplicates are harmless.

3 Which to choose

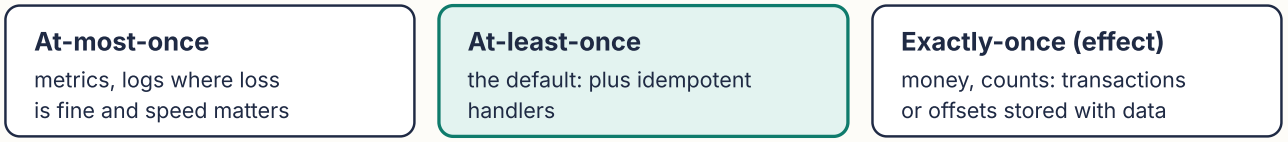


Figure 3. Choose per use case. At-least-once plus idempotency covers most needs.

Auto-commit in Kafka clients commits on a timer in the background, so it can commit offsets for messages your code hasn't finished: effectively at-most-once for those, at-least-once for others. Prefer explicit commits for anything important. In queues like SQS or RabbitMQ, the same rule applies to when you **delete or ack** the message.

COMMON MISTAKES

- Auto-commit with asynchronous processing that may fail later.
- Acking a message before handing it to a background task.
- At-least-once without idempotent handlers.
- Committing the offset of the last record when earlier ones in the batch failed.

4 Key terms

At-most-once: possibly lost, never duplicated

At-least-once: never lost, possibly duplicated

Exactly-once effect: duplicates delivered but applied once

Offset commit / ack: recording that a message is done

Idempotent sink: writing the same record twice changes nothing

5 Interview questions

Q1. What decides at-most-once vs at-least-once on the consumer? BASIC

Whether the offset or ack is committed before processing (at-most-once) or after (at-least-once).

Q2. Why is at-least-once the usual default? BASIC

Losing data is usually worse than processing it twice, and duplicates can be made harmless with idempotent handlers.

Q3. Design a consumer that writes events to a database without duplicates or losses. SCENARIO

Disable auto-commit, upsert by a unique event or entity key inside a DB transaction, then commit offsets; or store offsets in the same DB transaction.

Q4. Metrics events at 1 million per second: which semantics? SCENARIO

At-most-once or at-least-once with auto-commit is fine; losing or double-counting a few metrics is acceptable, and speed matters more.

Q5. Can auto-commit lose messages? TRICKY

Yes: it may commit offsets for records that were fetched but not yet processed; a crash then skips them.

Q6. Where can duplicates come from even with careful commits? TRICKY

Producer retries without idempotence, rebalances before a commit, redelivery after a visibility timeout, and replays.

Next: SD-I-009 · Dead-Letter Queues

Dead-Letter Queues

where messages go when they can't be processed

Watch the 4-min video

In one line: a dead-letter queue (DLQ) collects messages that keep failing so they stop blocking the main queue; set a max retry count, keep the original payload with error details, alert on DLQ growth, and have a tested way to fix and replay them.

1 The poison message

One malformed message arrives in your queue. The consumer tries to process it, crashes, and the message goes back to the queue. It's retried, it crashes again, and this loops forever. Meanwhile healthy messages behind it wait, or workers burn CPU on the same failure. That's a **poison message**.

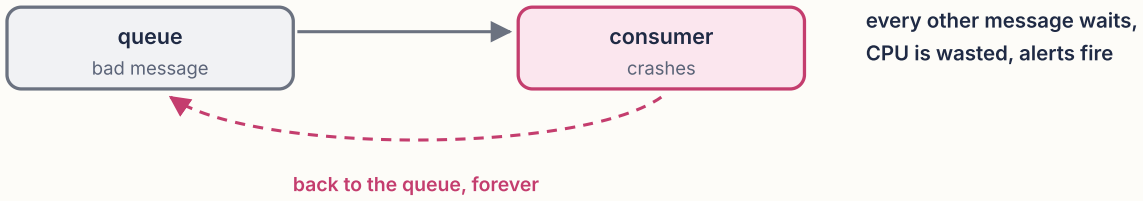


Figure 1. Without a limit, one bad message keeps failing forever.

2 What a DLQ does

A **dead-letter queue** is a separate queue or topic for messages that failed too many times. After N attempts the message is moved aside, the main flow continues, and someone can inspect the dead letters, fix the cause, and **replay** them. SQS, Azure Service Bus and RabbitMQ (dead-letter exchanges) support this natively; with Kafka, your consumer (or a framework like Spring Kafka or Kafka Connect) publishes failures to a `*.dlq` topic itself.

WORKED EXAMPLE: AN SQS REDRIVE POLICY

The orders queue has a dead-letter target and a max receive count of 5. A consumer receives a message, fails (missing customer ID), and doesn't delete it; after the visibility timeout it reappears. After the fifth receive, SQS moves it to `orders-dlq`. A CloudWatch alarm fires when the DLQ has any messages. The on-call engineer sees the error, deploys a fix that handles missing IDs, and uses SQS's redrive to move the messages back to the main queue, where they now succeed.

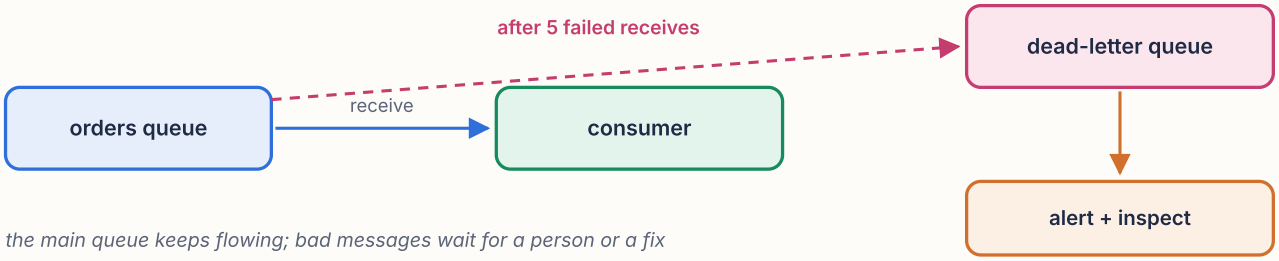


Figure 2. After five failed attempts, the message moves to the DLQ and an alert fires.

3 Doing it well

- **Separate failure types:** transient errors (timeouts) deserve retries with backoff (lesson 32); permanent ones (invalid data) can go to the DLQ immediately.
- **Keep context:** store the original message plus error details.
- **Alert** on DLQ depth and age; a DLQ nobody looks at is a data-loss bin.
- **Replay safely:** handlers must be idempotent, and replayed messages may now be out of order.
- Set the DLQ's retention longer than the main queue's.

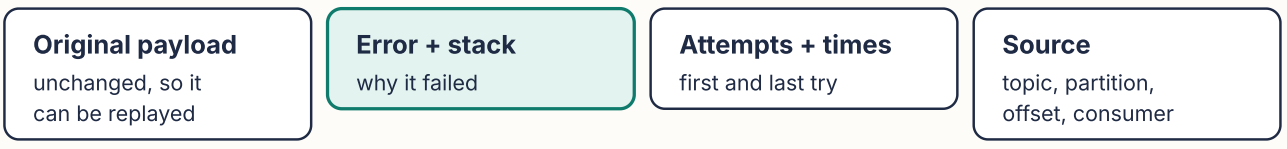


Figure 3. What to keep with each dead letter.

COMMON MISTAKES

- No DLQ, so poison messages block or loop.
- A DLQ with no alert and no owner.
- Sending transient failures to the DLQ on the first error.
- Replaying a DLQ before fixing the bug, so everything fails again.

4 Key terms

Poison message: a message that always fails processing

Max receive count: attempts before dead-lettering

Visibility timeout: time a received SQS message is hidden from others

Dead-letter queue: holding area for failed messages

Redrive / replay: moving dead letters back for processing

5 Interview questions

Q1. What is a dead-letter queue for? BASIC

To move messages that repeatedly fail out of the main queue, so they stop blocking or looping, and keep them for inspection and replay.

Q2. When should a message go to a DLQ immediately? BASIC

When the error is permanent, like invalid format or a missing required field, so retries can't succeed.

Q3. Your Kafka consumer is stuck on one bad record and lag grows. What do you do? SCENARIO

Catch the error, publish the record with error metadata to a DLQ topic, commit past it, alert, and fix; for transient errors use retry topics with backoff.

Q4. Thousands of messages land in the DLQ after a deploy. How do you recover? SCENARIO

Roll back or fix the bug, verify on a few messages, then replay the DLQ in batches with idempotent handlers, watching for order-sensitive cases.

Q5. Why can a DLQ hurt ordering? TRICKY

A dead-lettered message is processed later than the ones after it, so per-key order breaks; strict cases must block the key or carry versions.

Q6. What's the risk of a DLQ with no monitoring? TRICKY

Messages silently pile up and expire after retention, which is data loss that nobody notices.

Next: SD-I-010 · Retries and Delayed Messages

Retries and Delayed Messages

try again later, without blocking everything

Watch the 4-min video

In one line: transient failures deserve retries with exponential backoff and jitter, but sleeping inside a consumer blocks its partition, so move failed messages to delayed retry queues or topics, cap the attempts, and send what still fails to a DLQ.

1 Don't sleep in the consumer

A webhook call to a partner fails because their server is down. You want to try again in five minutes. In a Kafka consumer, the obvious move is to sleep and retry. But while it sleeps, every other message in that partition waits, the consumer may miss `max.poll.interval.ms` and trigger a rebalance, and one partner's outage delays webhooks for everyone.

2 Transient vs permanent

Retry **transient** failures: timeouts, 503s, connection resets, rate limits (respect `Retry-After`). Don't retry **permanent** ones: 400 Bad Request, validation errors, 'customer not found'. Those go straight to a DLQ (lesson 31). Retrying the wrong kind wastes capacity and hides bugs.

3 Backoff and jitter

Wait longer after each failure: **exponential backoff** (1 s, 2 s, 4 s, 8 s…) with a cap, so a struggling service gets room to recover. Add **jitter** (randomness) so thousands of clients don't retry at the same instant and knock it over again. Cap the total number of attempts or the total time.

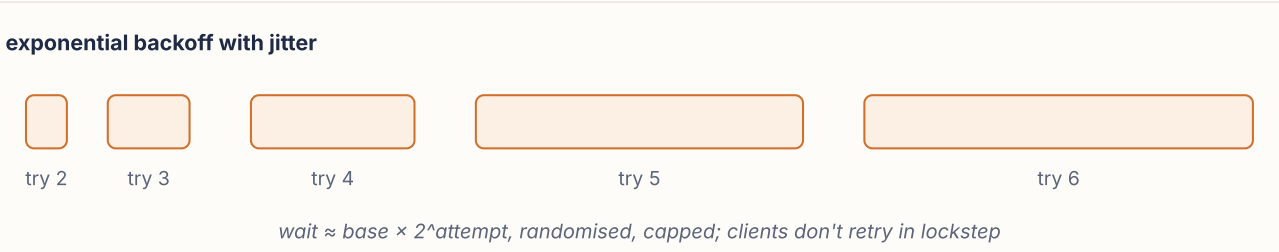


Figure 1. Growing, randomised waits spread retries out.

4 Delayed messages

For queues, use the broker's delay feature. For streams, use **retry topics** with fixed delays: the main consumer publishes a failed message to `retry-5m` with a 'not before' timestamp and moves on. The retry consumer pauses its partition until the earliest message is due, then reprocesses it. Order per key is lost for retried messages, so handlers should use versions or tolerate it (lesson 29).



Figure 2. Built-in delay mechanisms by broker.

WORKED EXAMPLE: RETRY-5M AND RETRY-1H FOR FAILED WEBHOOKS

The webhook consumer reads the `webhooks` topic and calls the partner. If the call fails with a 503, it publishes the message to `retry-5m` with headers `attempt=1` and `due=now+5m`, commits, and continues with other partners' webhooks. The retry-5m consumer sleeps until the message is due and retries; on failure it forwards to `retry-1h`; after that, to `webhooks-dlq` with an alert. A partner outage now delays only that partner's webhooks, and the partner receives the same idempotency key on every attempt.

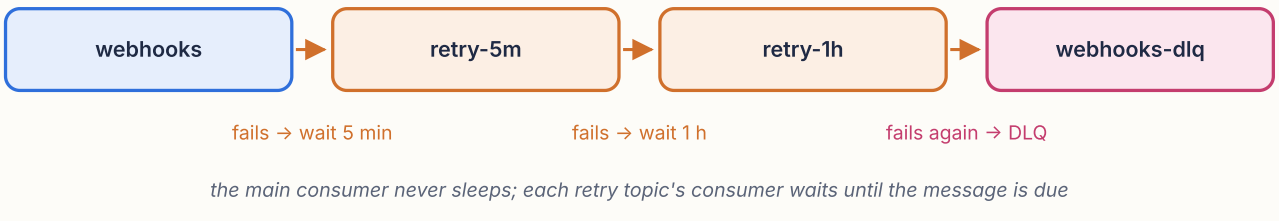


Figure 3. A retry ladder of topics with growing delays, ending in a DLQ.

COMMON MISTAKES

- Sleeping in the poll loop.
- Retrying immediately and forever, without backoff or a cap.
- Retrying non-idempotent calls without an idempotency key.
- Retrying permanent errors.

5 Key terms

Transient failure: likely to succeed if retried later

Jitter: randomness added to waits

Retry budget: limit on retries as a share of traffic

Exponential backoff: waits double after each failure

Retry topic: topic holding messages until their next attempt

Retry-After: HTTP header saying when to try again

6 Interview questions

Q1. Why use exponential backoff with jitter? BASIC

Backoff gives a failing service time to recover; jitter prevents many clients from retrying at the same moment and creating load spikes.

Q2. Why shouldn't a Kafka consumer sleep before retrying? BASIC

It blocks every message behind it in the partition and can exceed max.poll.interval.ms, triggering a rebalance.

Q3. Design retries for webhook delivery to thousands of partners. SCENARIO

Classify errors, send transient failures to retry topics with growing delays, cap attempts, DLQ the rest, use idempotency keys, and isolate partners so one outage doesn't delay others.

Q4. A downstream outage made your retries multiply the load ten times. Fix it. SCENARIO

Add backoff with jitter, a retry budget, a circuit breaker that stops calls during the outage, and a single retry layer instead of retries at every level.

Q5. What do retry topics do to ordering? TRICKY

A retried message is processed after later messages for the same key; if order matters, use versions or block the key until its retry succeeds.

Q6. How long should you keep retrying? TRICKY

As long as the business value lasts and the dependency's typical outage takes: minutes for user-facing actions, hours or days for webhooks, then DLQ and alert.

Next: SD-I-011 · Backpressure in Messaging backendroadmap.com

Backpressure in Messaging

when producers outrun consumers

Watch the 4-min video

In one line: a broker absorbs bursts as lag, but sustained overload needs a response: scale consumers (up to the partition count), slow producers with quotas or bounded buffers, or shed low-value work, while watching lag and protecting the systems your consumers write to.

1 Black Friday

Orders pour in at ten times the normal rate and your consumers can't keep up. Messages pile up in the broker, minute after minute. Nothing has crashed, but confirmation emails are now 40 minutes late, and the backlog keeps growing. This is the messaging version of **backpressure**: the downstream signalling, or being made to signal, 'slow down'.

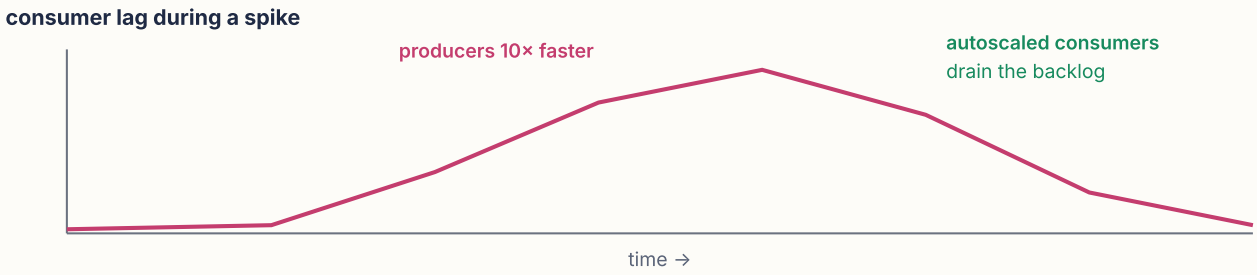


Figure 1. Lag grows while producers outrun consumers, and shrinks when consumers catch up.

2 The broker is a buffer, not a solution

Decoupling producers from consumers is the point of a broker (lesson 23): a short spike simply becomes **lag** and drains afterwards. But if the input rate stays above the processing rate, lag grows without limit: messages get old, may hit retention limits and be deleted, and catching up takes hours. Measure **lag in messages and in time** (age of the oldest unprocessed message) and alert on both.

3 Ways to respond

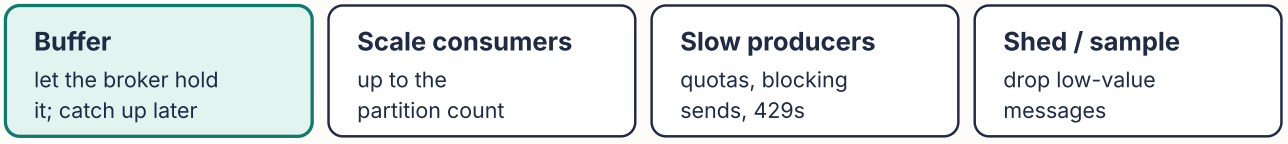
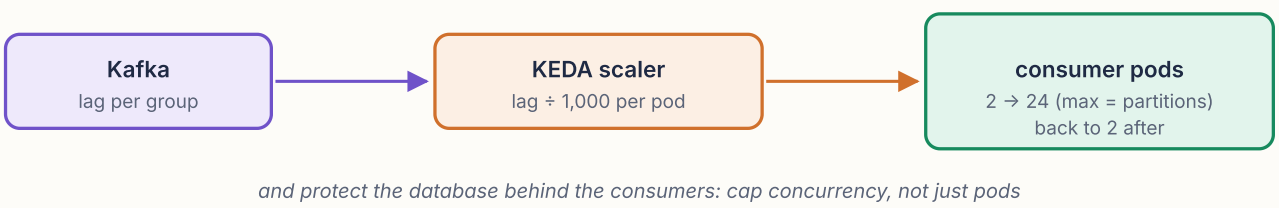


Figure 2. The four responses to sustained overload. Usually you combine the first two.

- **Scale consumers** on lag, up to the partition count, and make each consumer faster (batching, async I/O).
- **Slow producers:** Kafka quotas throttle clients; a full bounded producer buffer blocks `send()`; an API in front can return 429. With RabbitMQ, queue length limits and flow control push back on publishers.
- **Shed or sample:** drop debug events or analytics samples first; never orders.
- **Prioritise:** separate topics or queues for critical and bulk work, so bulk can't starve critical.

WORKED EXAMPLE: KEDA SCALING CONSUMERS ON KAFKA LAG

KEDA is an event-driven autoscaler for Kubernetes. You give it a Kafka scaler with the consumer group, a target of 1,000 messages of lag per pod, and a maximum of 24 pods, the topic's partition count. When the spike hits, lag climbs past 20,000 and KEDA scales from 2 to 24 pods within a minute or two; when lag drains, it scales back. Because 24 pods would overload the orders database, each pod caps its concurrent writes and uses a shared connection pool limit, so the backlog drains at the rate the database can take.



and protect the database behind the consumers: cap concurrency, not just pods

Figure 3. Lag-based autoscaling, bounded by partitions and by what downstream can handle.

COMMON MISTAKES

- Scaling on CPU instead of lag.
- Scaling consumers beyond the partition count.
- Unbounded in-memory queues inside consumers, which run out of memory.
- Letting retention delete unprocessed messages during a long backlog.

4 Key terms

Backpressure: signal from a slow consumer that slows the producer

Quota: broker-enforced rate limit per client

KEDA: Kubernetes autoscaler driven by event sources like Kafka lag

Consumer lag: messages (or time) the consumer is behind

Load shedding: dropping low-value work under overload

5 Interview questions

Q1. How does a message broker help with traffic spikes? BASIC

It buffers messages so producers aren't slowed; consumers work at their own pace and catch up later, as lag.

Q2. Which metric should drive consumer autoscaling? BASIC

Consumer lag, ideally both message count and age of the oldest message, rather than CPU.

Q3. Lag keeps growing for hours. What do you do? SCENARIO

Find the bottleneck (consumer CPU, downstream DB, partition count). Scale consumers up to the partitions, add partitions if needed, batch writes, and throttle or shed low-priority producers.

Q4. Scaling consumers overloaded the database. Fix it. SCENARIO

Limit concurrency per consumer and in total, use bulk writes, and let lag absorb the rest; the database's capacity is the real limit.

Q5. How can a Kafka producer feel backpressure? TRICKY

Its buffer fills when brokers are slow or throttled by quotas; send() then blocks up to max.block.ms or fails, which you can surface as 429s upstream.

Q6. When is dropping messages acceptable? TRICKY

For low-value or aggregatable data like metrics, logs and analytics samples, never for orders or payments; make it explicit and measured.

Next: SD-I-012 · Event Types: Notifications, State Transfer, Commands backendroadmap.com

Event Types: Notifications, State Transfer, Commands

what a message should carry

Watch the 4-min video

In one line: an event notification says something happened and readers fetch details, event-carried state transfer includes the data so readers don't call back, and a command asks one service to do something; name events in the past tense, commands in the imperative, and choose payloads deliberately.

1 The callback storm

Your order service publishes an event called `OrderPlaced` containing only the order ID. The shipping team reads it and immediately calls the order service back to get the address. So do billing and analytics. Every order now creates three extra API calls, and when the order service is slow, every consumer is slow too: the coupling you tried to remove is back.

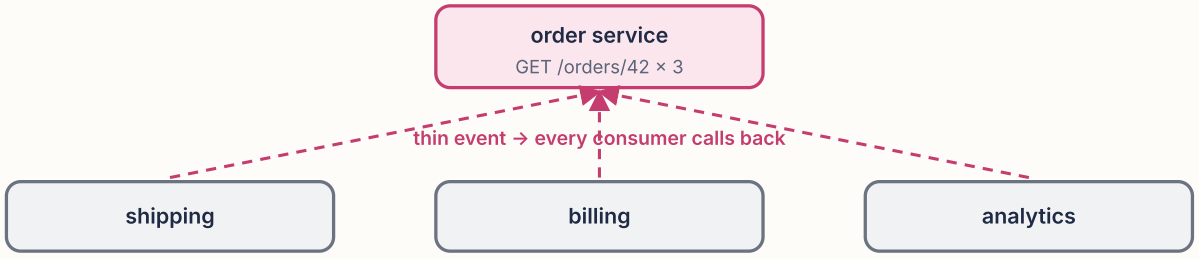


Figure 1. Thin events make consumers call the producer, re-coupling them.

2 Three kinds of message

Event notification
'OrderPlaced id=42'
small; readers call back for details

Event-carried state
full order in the event: readers need no callback

Command
'ShipOrder 42': a request to ONE service to act

Figure 2. Events state facts in the past tense; commands request an action.

Event notification ('OrderPlaced, id 42'): tiny and stable, but consumers need callbacks for details, and the details may have changed by then. **Event-carried state transfer**: the event includes what consumers need (items, prices, address), so they can act, or keep their own copy, with no callback. Bigger messages and more schema to evolve (lesson 39), but real decoupling. **Commands** ('ShipOrder'): addressed to one owner, which may refuse; they're requests, not facts, and often go to a queue rather than a shared topic.

WORKED EXAMPLE: ORDERPLACED WITH THE FULL PAYLOAD

The order service publishes `OrderPlaced` with the order ID, customer ID, line items with prices, the total and the shipping address, plus an `event_id`, schema version and `occurred_at`. Shipping creates a shipment from the event alone; billing records revenue; analytics stores it. None of them call the order service, and they all see the order exactly as it was when placed, even if the customer edits their address later. Sensitive data (full card numbers) stays out of the event.

```
{ "type": "OrderPlaced", "event_id": "e-912", "version": 2,
  "occurred_at": "2026-12-05T09:01:12Z",
  "order": { "id": 42, "customer_id": 17, "total": 59.80,
    "items": [{"sku": "BK-1", "qty": 2, "price": 29.90}],
    "shipping_address": { "city": "Pune", "pin": "411001" } } }
```

Figure 3. An event-carried state transfer payload with metadata.

3 Naming and design

- Events: past-tense facts in business language: `OrderPlaced`, `PaymentFailed`, not `OrderTableUpdated`.
- Commands: imperative: `ReserveStock`, `SendEmail`.
- Always include an event ID (for dedup), a type and version, a timestamp, and a correlation ID.
- Prefer **domain events** designed for consumers over raw database change events, which leak your schema.

COMMON MISTAKES

- Thin events that force every consumer to call back.
- Commands disguised as events ('OrderShouldBeShipped').
- Publishing internal table changes as the public contract.
- Putting secrets or excessive personal data in widely-read events.

4 Key terms

Event: a fact that happened, past tense

Event notification: minimal event; details fetched separately

Domain event: business-level event, not a DB change

Command: a request for one service to act

Event-carried state transfer: event includes the needed data

Correlation ID: ties all messages of one flow together

5 Interview questions

Q1. What's the difference between an event and a command? BASIC

An event states a fact that already happened and can have many consumers; a command asks one specific service to do something and can be rejected.

Q2. What is event-carried state transfer? BASIC

Including the relevant data in the event so consumers can act or keep a local copy without calling the producer.

Q3. Consumers keep calling your service after every event. What do you change? SCENARIO

Add the fields they need to the event (event-carried state transfer), version the schema, and keep sensitive fields out.

Q4. Should you publish Debezium row changes directly to other teams? SCENARIO

Usually not as the public contract: they expose your table structure. Transform them into domain events, or use an outbox with designed payloads.

Q5. What's the downside of fat events? TRICKY

Larger messages, more fields to keep compatible, data copies spread across services, and a risk of leaking sensitive data.

Q6. Can a consumer trust that event data is current? TRICKY

It's current as of the event time. If it needs the latest state, it must apply later events in order or query the owner.

Next: SD-I-013 · Event Sourcing

Event Sourcing

store what happened, derive what is

Watch the 4-min video

In one line: event sourcing stores every change to an entity as an immutable event in an append-only stream and derives current state by replaying them; you gain a complete audit trail, replay and time travel, and you pay with schema evolution, snapshots and separate read models for queries.

1 How did the balance get here?

A customer calls: 'My balance is wrong; how did it get to this number?' Your accounts table stores one value, the current balance. Every update overwrote the last one, so the history is gone. Accountants solved this centuries ago with a **ledger**: you never erase, you only add entries. Event sourcing applies that idea to software.

2 How it works

Each entity (an account, an order) has an **event stream**: an ordered list of facts such as `Deposited` or `Withdrew`. Events are **immutable**, only appended. To get the current state, load the stream and apply the events in order. To change state, a command loads the state, checks the business rules, and appends new events.

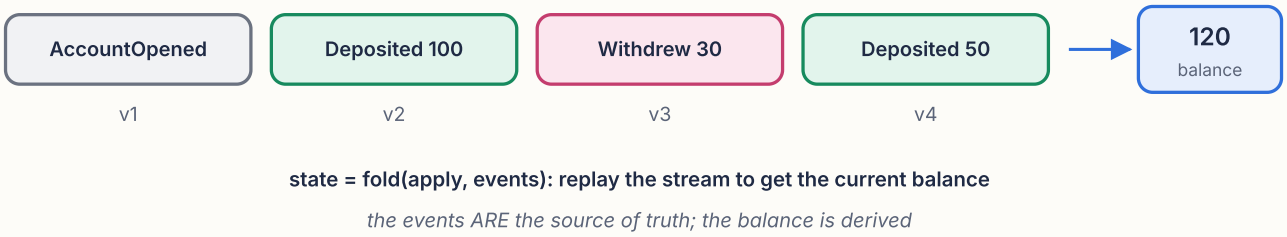


Figure 1. The balance is computed from the events, not stored as the truth.

WORKED EXAMPLE: A BANK ACCOUNT BALANCE

The stream for account 7 holds `AccountOpened`, `Deposited 100`, `Withdrew 30`, `Deposited 50`: balance 120 at version 4. A request to withdraw 200 loads the stream, sees 120, and is rejected. A request to withdraw 20 appends `Withdrew 20` with **expected version 4**. If another withdrawal appended version 5 in the meantime, this append fails with a concurrency error, and the command reloads and re-checks, so two withdrawals can't both pass a balance check that only one would. When the customer calls, support lists the events and shows exactly how 100 became 120.



Figure 2. Optimistic concurrency with an expected version keeps business rules safe.

3 Practical pieces

- **Event store:** EventStoreDB, or a Postgres table with (stream_id, version) unique, or Kafka plus a database for the per-stream check.
- **Snapshots:** for long streams, store the state every N events and replay only the rest.
- **Projections:** to query 'all accounts with balance below 0', build read models from the events (CQRS, lesson 36).
- **Versioning:** old events never change, so readers must understand every old version (upcasting).
- **Personal data:** 'right to be forgotten' needs crypto-shredding or keeping PII out of events.

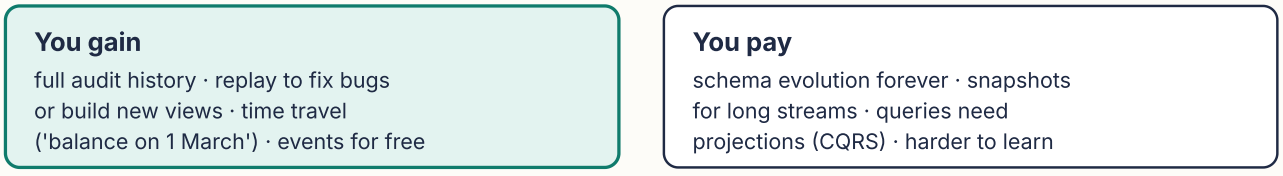


Figure 3. Event sourcing's trade-off. Use it for domains where history matters.

COMMON MISTAKES

- Event sourcing every entity in the system, including simple CRUD.
- No expected-version check, so concurrent commands break rules.
- Querying the event store directly instead of building projections.
- Editing or deleting old events.

4 Key terms

Event store: append-only storage of event streams

Event stream: all events of one entity, in order

Rehydration: rebuilding state by replaying events

Snapshot: saved state to shorten replay

Projection: read model built from events

Expected version: optimistic concurrency check on append

5 Interview questions

Q1. What is event sourcing? BASIC

Persisting each change as an immutable event and deriving current state by replaying the events, instead of storing and overwriting the current state.

Q2. What are its main benefits? BASIC

A complete audit trail, the ability to replay events to fix bugs or build new views, time travel to past states, and a natural stream of events for other services.

Q3. Two withdrawals arrive at once for an account with 120. How do you prevent overdraft? SCENARIO

Both load version 4; each appends with expected version 4; one succeeds, the other gets a concurrency error, reloads, sees the new balance and is rejected if funds are short.

Q4. An account has 2 million events and loads slowly. Fix it. SCENARIO

Store snapshots every N events and replay only events after the latest snapshot.

Q5. How do you change the structure of an event that already exists in the store? TRICKY

You don't edit stored events; you add a new version and upcast old ones when reading, or write a migration to a new stream.

Q6. When is event sourcing a bad idea? TRICKY

For simple CRUD data with no need for history, teams new to it, or domains where querying current state is all that matters; the complexity isn't repaid.

Next: SD-I-014 · CQRS

CQRS

separate models for writing and reading

Watch the 4-min video

In one line: CQRS (command query responsibility segregation) uses one model to handle changes and enforce rules, and one or more separate read models shaped for queries, kept up to date from events or change data capture; reads become fast and flexible, at the cost of eventual consistency and more moving parts.

1 One table can't serve both

Your product team wants a search page: filter orders by status, customer name and date, sort by total, with full-text search on the items. But the `orders` table is designed for safe writes: normalised, indexed for lookups by ID, protected by constraints. Adding every search index slows writes, and full-text search is a poor fit for it anyway.

2 The idea

Commands (PlaceOrder, CancelOrder) go to a **write model** that validates them and stores the result. **Queries** go to **read models** built for them: a search index, a denormalised table per screen, a cache, a graph. The read models are updated from the write side, usually through events (lesson 34), the outbox (lesson 37) or change data capture. Each side can use different storage and scale independently.

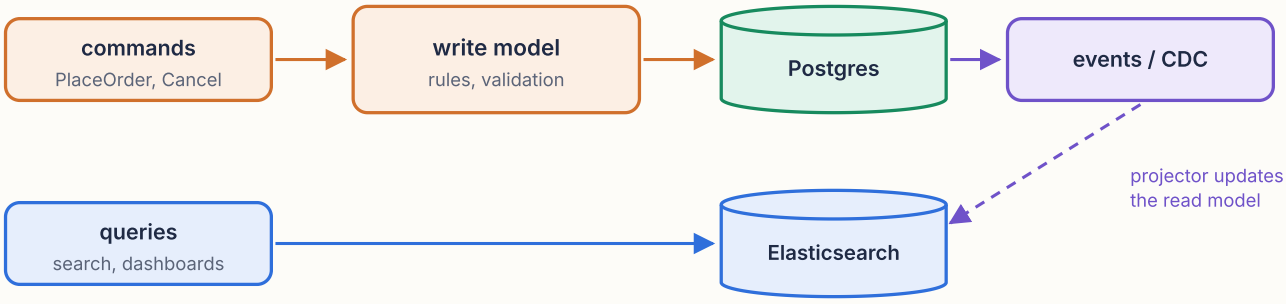


Figure 1. Writes go to Postgres; a projector turns changes into a search-friendly read model.

WORKED EXAMPLE: ORDER WRITES IN POSTGRES, READ VIEWS IN ELASTICSEARCH

The order service handles commands like PlaceOrder and CancelOrder and writes to Postgres in transactions, publishing events through its outbox. A projector consumes `OrderPlaced`, `OrderCancelled` and `CustomerRenamed` and upserts one denormalised document per order into Elasticsearch, with customer name, item text and total. The search page queries Elasticsearch only. When the team needs a new 'orders by region' dashboard, they write a second projector and replay the events to build it.

3 Living with lag

The read model trails the write model by milliseconds to seconds. Right after placing an order, the user's search may not show it. Handle it: update the UI optimistically from the command result, read your own writes from the write model for a short window, or return the write's version and let the UI wait until the read model has reached it.



hide the lag: optimistic UI, or wait until the projection reaches the write's version

Figure 2. Hide projection lag from the user who just wrote.

4 When it's worth it

Use CQRS when

reads and writes need very different shapes, scale or storage; many views of the same data; event-sourced systems

Skip it when

simple CRUD; one query model is enough; the team can't absorb eventual consistency

Figure 3. CQRS is a tool for specific pressures, not a default.

Projections must be **idempotent** and handle out-of-order or replayed events (versions per document). Keep the ability to **rebuild** a read model from scratch, and monitor projection lag like consumer lag.

COMMON MISTAKES

- Applying CQRS to every simple CRUD screen.
- Updating the database and the search index from the API in two separate steps, without an outbox.
- Projections that can't be rebuilt.
- Ignoring the lag, so users think their action failed.

5 Key terms

CQRS: separate models for commands and queries

Read model / projection: query-optimised copy built from changes

Projection lag: how far the read model trails the writes

Write model: validates commands, enforces rules

Projector: consumer that updates a read model

6 Interview questions

Q1. What is CQRS? BASIC

Separating the model that handles writes (commands and rules) from the models that serve reads, usually with different storage, kept in sync asynchronously.

Q2. Does CQRS require event sourcing? BASIC

No. They fit well together, but CQRS can use a normal database for writes and CDC or an outbox to update the read models.

Q3. Design order search with filters and full-text over a transactional orders database. SCENARIO

Keep orders in Postgres, publish changes via outbox or CDC, project them into Elasticsearch documents, and query only the index; handle lag in the UI.

Q4. Users don't see their new order in search for two seconds. What do you do? SCENARIO

Show it optimistically from the command response, or wait for the projection to reach the write's version; also check projector lag.

Q5. How do you fix a bug in a projection that corrupted the read model? TRICKY

Fix the projector, then rebuild the read model by replaying the events or re-exporting from the write store into a new index, and switch over.

Q6. What's the biggest cost of CQRS? TRICKY

Eventual consistency between models, plus more components to build, run and keep in sync.

Next: SD-I-015 · Transactional Outbox Pattern

Transactional Outbox Pattern

save data and publish an event, reliably

Watch the 4-min video

In one line: writing to the database and publishing to a broker are two separate actions that can't share a transaction, so write the event into an outbox table in the same database transaction as the data, and let a relay or CDC tool publish it; you get at-least-once publishing with no lost events.

1 The dual-write problem

Your order service saves a new order in Postgres, then publishes `OrderPlaced` to Kafka. It works perfectly in testing. Then one day the service crashes right after the database commit and before the publish. The order exists, but shipping never hears about it. Reversing the order (publish first) is no better: the event goes out for an order whose insert then fails. Writing to two systems with no shared transaction is the **dual-write problem**.

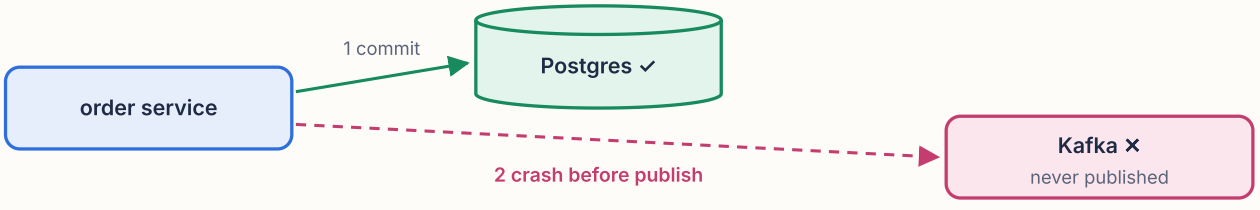


Figure 1. A crash between the two writes leaves the database and the broker disagreeing.

2 The outbox

Add an `outbox` table to the service's own database. In the **same local transaction** that changes business data, insert a row describing the event. Either both commit or neither does. A separate **relay** then reads the outbox and publishes each row to the broker, marking it sent (or relying on CDC offsets).

```
BEGIN;
INSERT INTO orders (id, customer_id, total) VALUES (42, 17, 59.80);
INSERT INTO outbox (event_id, aggregate_id, type, payload)
  VALUES ('e-912', '42', 'OrderPlaced', '{...}');
COMMIT;   -- both rows, or neither
```

Figure 2. The business row and the event row commit together.

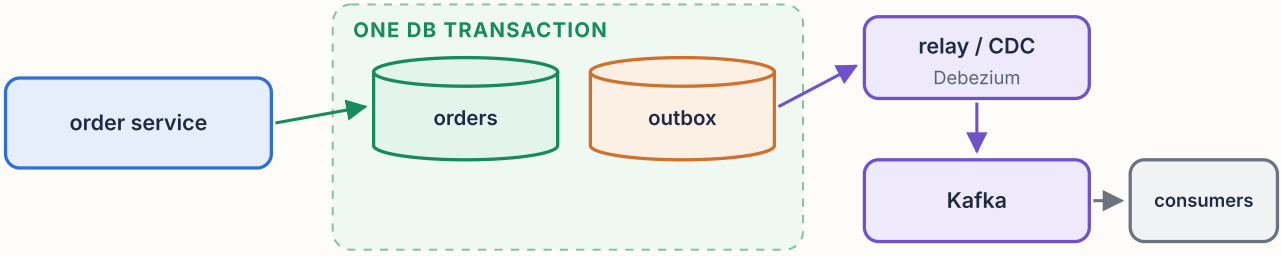


Figure 3. Outbox with CDC: Debezium reads the database log and publishes outbox rows to Kafka.

3 Two ways to relay

- **Polling publisher:** a job selects unsent rows (`FOR UPDATE SKIP LOCKED`), publishes them in order, and marks them sent. Simple; adds some latency and DB load.
- **Log-based CDC:** Debezium tails the database's write-ahead log and turns outbox inserts into Kafka messages (its outbox event router routes by aggregate type and keys by aggregate ID). Low latency, no polling.

Either way, a crash after publishing but before marking means the event is published again: **at-least-once**. Consumers deduplicate by `event_id` (lesson 38). Delete or archive old outbox rows regularly.

WORKED EXAMPLE: ORDERPLACED, RELIABLY

In one transaction, the order service inserts order 42 and an outbox row with event ID `e-912`, type `OrderPlaced`, aggregate ID 42 and the full payload. Debezium sees the outbox insert in Postgres' WAL a few milliseconds later and publishes it to the `orders` topic with key 42. If the service crashes right after the commit, the event still goes out, because it's in the database. If Debezium restarts, it may publish `e-912` again; the shipping service's inbox drops the duplicate.

COMMON MISTAKES

- Publishing inside the transaction (the publish can't roll back).
- Publishing after commit in application code and calling it reliable.
- A relay that publishes out of order for the same aggregate.
- Never cleaning up the outbox table.

4 Key terms

Dual write: updating two systems with no shared transaction

Relay / message relay: process that publishes outbox rows

Debezium: open-source CDC platform for Kafka

Outbox table: events stored with business data

CDC: change data capture from the database log

5 Interview questions

Q1. What problem does the transactional outbox solve? BASIC

The dual-write problem: making sure a database change and its event either both happen or neither, without a distributed transaction.

Q2. How do events get from the outbox to the broker? BASIC

A relay polls the outbox table and publishes unsent rows, or a CDC tool like Debezium reads the database log and publishes the inserts.

Q3. Some orders never reach the shipping service. The code saves, then publishes. Fix it. SCENARIO

Use an outbox: insert the event in the same transaction as the order, relay it with CDC or a poller, and make shipping deduplicate by event ID.

Q4. Polling the outbox every 100 ms loads the DB. Alternatives? SCENARIO

Use log-based CDC (Debezium) instead of polling, or poll less often with batched, indexed queries and SKIP LOCKED.

Q5. Does the outbox give exactly-once publishing? TRICKY

No, at-least-once: a relay can crash after publishing and before recording progress. Consumers must deduplicate.

Q6. How do you keep events for one order in order? TRICKY

Use the aggregate ID as the message key and publish outbox rows in commit order per aggregate.

Next: SD-I-016 · Inbox Pattern and Deduplication backendroadmap.com

Inbox Pattern and Deduplication

receive at least once, process once

Watch the 4-min video

In one line: the inbox pattern stores each received message ID in the consumer's database in the same transaction as the work it triggers, so a redelivered message is detected and skipped; together with the outbox on the producer side, it gives effectively-once processing between services.

1 The same message, twice

The payment service receives `PaymentRequested` for order 42 and charges the customer's card. Then the same message is delivered again, maybe because the outbox relay re-published it, or the consumer crashed before committing its offset. Without protection, the customer is charged twice.



Figure 1. At-least-once systems create duplicates in many places. The consumer must handle all of them.

2 The inbox

The consumer keeps an `inbox` table (or processed_messages) with the **message ID as the primary key**. For each message, it opens a transaction, inserts the ID, does its work, and commits. If the ID is already there, the insert fails, the transaction rolls back, and the consumer simply acknowledges: the duplicate has no effect. Because the ID and the effects commit atomically, there's no window where one is saved without the other.

```
BEGIN;
INSERT INTO inbox (message_id, received_at) VALUES ('m-77', now());
--  duplicate key → ROLLBACK; just ack the message
INSERT INTO payments (order_id, amount, status) VALUES (42, 59.80, 'PENDING');
INSERT INTO outbox (...) VALUES (... 'PaymentRequested' ...);
COMMIT;
```

Figure 2. Inbox, business change and the next outbox event in one local transaction.

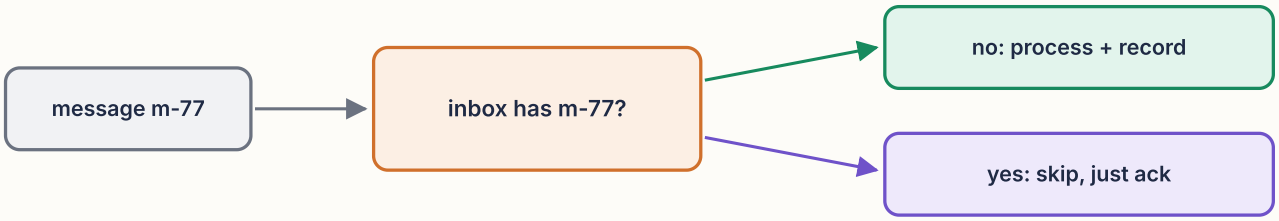


Figure 3. Check-and-record happens atomically through the unique key, not as a separate read.

WORKED EXAMPLE: IGNORING A REDELIVERED PAYMENTREQUESTED

The first delivery, with message ID `m-77`, opens a transaction, inserts `m-77` into the inbox, creates a PENDING payment and writes a `ChargeCard` outbox event, then commits and acks. The card is charged with idempotency key `m-77`. Ten minutes later the relay re-publishes the same event after a restart. The inbox insert fails with a duplicate key, the consumer rolls back and acks, and nothing else happens. The inbox row expires after 14 days, longer than any possible redelivery.

3 Practical details

- Use the **producer's event ID**, stable across retries, not a broker delivery tag.
- Keep inbox rows longer than the redelivery and replay window; clean up with a TTL job or partitions.
- Side effects outside your database (card APIs, email) still need their own idempotency keys.
- Where natural idempotency exists (an upsert by order ID), the inbox may be unnecessary.

COMMON MISTAKES

- SELECT to check the inbox, then INSERT later in another transaction (race condition).
- Deduplicating in memory or in Redis without a transaction with the effect.
- Inbox retention shorter than the topic's replay window.

4 Key terms

Inbox pattern: record received message IDs with the work, atomically

Deduplication window: how long IDs are remembered

Message ID: stable unique ID assigned by the producer

Effectively-once: at-least-once delivery + dedup = one effect

5 Interview questions

Q1. What is the inbox pattern? BASIC

Storing each processed message ID in the consumer's database, in the same transaction as the work, so duplicates are detected by the unique key and skipped.

Q2. How do outbox and inbox work together? BASIC

The outbox guarantees events are published at least once; the inbox makes processing them idempotent, so each event's effect happens once.

Q3. Customers are occasionally charged twice. Messages come from Kafka. Fix it. SCENARIO

Add an inbox keyed by event ID in the same transaction as recording the payment, and pass the event ID as the idempotency key to the card provider.

Q4. How long do you keep inbox rows? SCENARIO

Longer than any duplicate can arrive: broker retention, retry and DLQ replay windows; then delete with a scheduled job.

Q5. Why not check 'already processed?' with a SELECT first? TRICKY

Two deliveries processed concurrently can both see 'not processed' and both run. The unique insert in the same transaction makes the check atomic.

Q6. Does an inbox protect external API calls? TRICKY

Not by itself; the external call isn't in your transaction. Use the provider's idempotency key, or record intent first and make the call idempotent.

Next: SD-I-017 · Schema Evolution and Registries backendroadmap.com

Schema Evolution and Registries

changing events without breaking consumers

Watch the 4-min video

In one line: events outlive the code that wrote them, so change their schemas only in compatible ways (add optional fields with defaults, never rename or reuse), and let a schema registry enforce compatibility before a producer can publish a breaking change.

1 A Friday deploy

The order team adds a field to the `OrderPlaced` event and renames another. They deploy on a Friday afternoon. Within minutes, the shipping consumer fails to deserialize every new event, and orders stop shipping. Events are a **contract** between teams who deploy independently, and old events stay in the topic for days or forever.

2 Compatibility



Figure 1. Three kinds of compatibility, and what each one lets you deploy first.

Safe changes: **add an optional field with a default**; remove an optional field. Breaking changes: **rename** a field, **change its type**, add a required field without a default, or reuse an old field name or number for something else. For a truly breaking change, publish a new event type or topic (`OrderPlaced.v2`) and run both during migration.

3 Formats

JSON is readable but has no schema by default; use JSON Schema and tolerant readers that ignore unknown fields. **Avro** stores data without field names and resolves the writer's schema against the reader's, applying defaults. **Protobuf** identifies fields by number: never reuse a number, and mark removed ones as reserved.

4 Schema registries

A **schema registry** (Confluent Schema Registry, Apicurio, AWS Glue, Azure Schema Registry) stores every version of every schema and checks new versions against a compatibility rule before accepting them. Producers embed a small **schema ID** in each message; consumers fetch and cache the schema to decode it. A breaking change fails at registration, in CI, not in production.

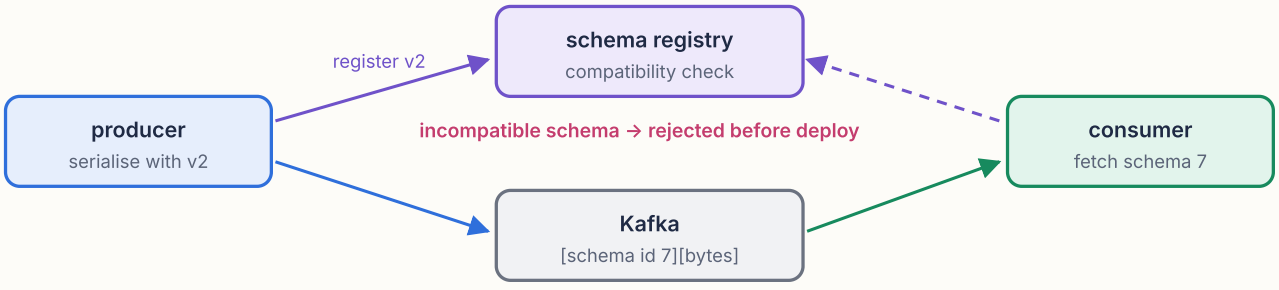


Figure 2. The registry rejects incompatible schemas and lets consumers find the exact writer schema.

WORKED EXAMPLE: ADDING AN OPTIONAL FIELD IN AVRO

Version 1 of `OrderPlaced` has `order_id`, `customer_id` and `total`. Version 2 adds `coupon_code` as a nullable string with default null. The registry, set to BACKWARD compatibility, accepts it. The shipping consumer, still on v1, reads v2 events and simply ignores the new field. The analytics consumer upgrades to v2 and reads old v1 events with `coupon_code = null`. Had the team instead renamed `total` to `amount`, the registry would have rejected it in the CI pipeline.

```
{"type": "record", "name": "OrderPlaced", "fields": [
  {"name": "order_id",    "type": "string"},
  {"name": "customer_id", "type": "string"},
  {"name": "total",       "type": "double"},
  {"name": "coupon_code", "type": ["null", "string"], "default": null}   // v2
]}
```

Figure 3. A compatible change: a new optional field with a default.

COMMON MISTAKES

- Renaming fields in place.
- Required new fields without defaults.
- Reusing a Protobuf field number.
- Consumers that fail on unknown JSON fields.
- No compatibility check in CI.

5 Key terms

Schema evolution: changing a schema while old data still exists

Forward compatible: old schema reads new data

Tolerant reader: ignores fields it doesn't know

Backward compatible: new schema reads old data

Schema registry: stores schemas and enforces compatibility

6 Interview questions

Q1. What is backward compatibility for event schemas? BASIC

Consumers using the new schema can read data written with the old schema; you upgrade consumers before producers start writing new data.

Q2. Which schema changes are safe? BASIC

Adding optional fields with defaults and removing optional fields. Renaming, changing types and adding required fields are breaking.

Q3. You must rename a field that ten teams consume. How? SCENARIO

Add the new field alongside the old, populate both, migrate consumers, then stop populating and finally remove the old one; or publish a new versioned event.

Q4. How does a schema registry prevent outages? SCENARIO

It checks every new schema version against the compatibility rule; incompatible versions are rejected at registration or in CI, before any producer publishes them.

Q5. Why must Protobuf field numbers never be reused? TRICKY

Old data still has the old field under that number; a new meaning would make old messages decode into wrong values.

Q6. Why is FULL compatibility often chosen for shared topics? TRICKY

Producers and consumers owned by different teams deploy in any order, and replaying old events must still work.

Next: SD-I-018 · Event-Driven Microservices backendroadmap.com

Event-Driven Microservices

services that talk through events, and how to keep them understandable

Watch the 4-min video

In one line: event-driven microservices publish domain events and react to each other's events, which removes runtime coupling, but needs outboxes, idempotent consumers, correlation IDs, tracing and flow monitoring, or nobody can tell where an order got stuck.

1 Decoupled, and lost

Your system has forty services that talk only through events. Nobody calls anybody directly, so everything is decoupled. Then a customer asks why their order never shipped. Which service dropped it? There's no stack trace across events, no single log, and no one place where the flow is written down.

2 The building blocks

An event-driven microservice owns its data, changes it in local transactions, and publishes **domain events** through an **outbox** (lesson 37). It consumes other services' events with an **inbox** or idempotent handlers (lesson 38), and keeps any data it needs from them as a local copy built from events (event-carried state transfer, lesson 34). Long business flows are **sagas** (lesson 17).

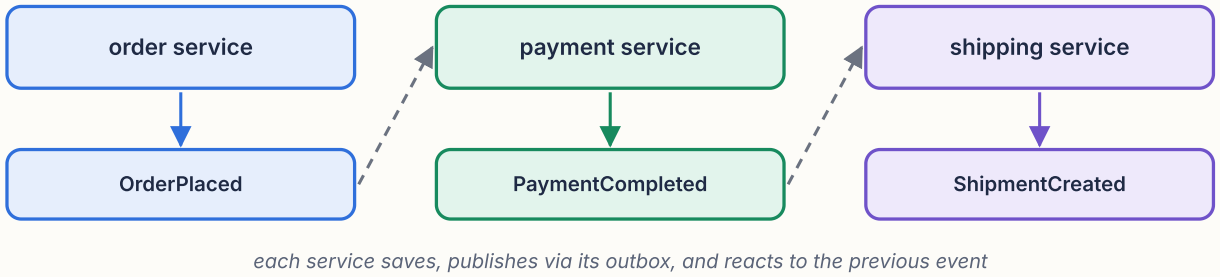


Figure 1. Order to payment to shipping, entirely through events.

WORKED EXAMPLE: ORDER TO PAYMENT TO SHIPPING

The order service saves order 42 and publishes `OrderPlaced` through its outbox, with correlation ID `c-5f1`. The payment service consumes it, charges the card, and publishes `PaymentCompleted` with the same correlation ID. The shipping service consumes that and publishes `ShipmentCreated`. When the customer asks, support searches logs and traces for `c-5f1` and sees the payment event consumed but no shipment: the shipping consumer had sent the event to its DLQ because of an unknown postcode format. A flow monitor that expects `ShipmentCreated` within an hour of `PaymentCompleted` would have alerted first.

3 Making the flow visible



Figure 2. Observability you need before you have forty services.

Pass a **correlation ID** (and causation ID: which event caused this one) in every message and log. Propagate **OpenTelemetry** trace context in message headers so a trace spans services and brokers. Keep an **event catalog** (AsyncAPI, a schema registry) listing producers, consumers and schemas.

4 Choreography or orchestration

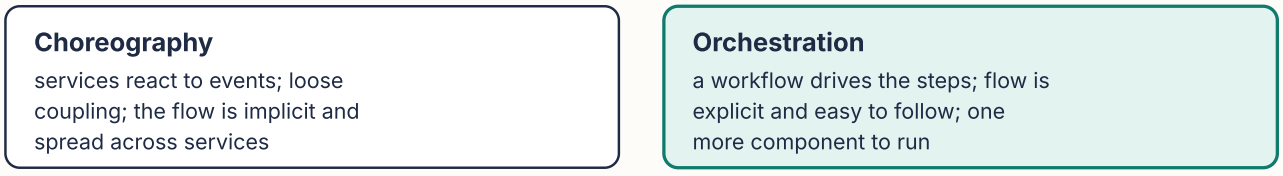


Figure 3. For long or critical flows, an explicit orchestrator is often easier to operate.

COMMON MISTAKES

- No correlation IDs, so flows can't be traced.
- Event chains ten services long with no owner for the whole flow.
- Shared databases between 'event-driven' services.
- Events used for queries that need an immediate answer.

5 Key terms

Domain event: business fact published by the owning service

Causation ID: ID of the message that caused this one

AsyncAPI: specification format for event-driven APIs

Correlation ID: ID shared by all messages of one flow

Event catalog: registry of events, producers and consumers

6 Interview questions

Q1. What are the benefits of event-driven microservices? BASIC

Services don't depend on each other being up, can be added without changing producers, scale independently, and keep their own data.

Q2. What makes them hard to operate? BASIC

Flows are implicit and spread across services, failures are silent, data is eventually consistent, and debugging needs tracing across brokers.

Q3. An order is paid but never shipped. How do you find out why? SCENARIO

Search by correlation ID in logs and traces, check the shipping consumer's lag and DLQ, and add a flow monitor that alerts when a step doesn't follow within its expected time.

Q4. When would you add an orchestrator to a choreographed flow? SCENARIO

When the flow is long, business-critical, changes often, or needs timeouts and compensations that are hard to see across services.

Q5. How do you avoid an event-driven 'distributed monolith'? TRICKY

Keep services owning their data, design events around business facts, avoid synchronous call chains disguised as events, and version event contracts.

Q6. How do traces cross a message broker? TRICKY

The producer puts trace context (W3C traceparent) into message headers; the consumer extracts it and starts a linked span.

Next: SD-I-019 · Cloud Messaging Services backendroadmap.com

Cloud Messaging Services

let the provider run the broker

Watch the 4-min video

In one line: every cloud offers managed queues, pub/sub topics, event streams and workflows, which remove broker operations and scale automatically; choose by delivery model, ordering, throughput, retention and integrations, and know each service's limits, such as message size, retention and FIFO throughput.

1 Three months of patching

Your team needs a message queue, and someone suggests running Kafka. Three months later you're patching brokers, rebalancing partitions and debugging disk alerts at night. For many teams a **managed** service would have done the job, with nothing to run.

	AWS	Azure	Google Cloud
queue	SQS (+FIFO)	Service Bus queues	Cloud Tasks / Pub/Sub
pub/sub	SNS, EventBridge	Service Bus topics, Event Grid	Pub/Sub
stream	Kinesis, MSK (Kafka)	Event Hubs (Kafka API)	Pub/Sub, Managed Kafka
workflow	Step Functions	Durable Functions, Logic Apps	Workflows

Figure 1. Rough equivalents across the three big clouds. Names change; the categories don't.

2 How to choose

- **Delivery model:** work queue (one consumer per message) or fan-out (many subscribers) or a replayable stream (lesson 24)?
- **Ordering:** SQS FIFO per message group, Service Bus sessions, Pub/Sub ordering keys, Kinesis/Event Hubs per shard or partition.
- **Throughput and size limits:** e.g. SQS messages up to 256 KB (larger via S3), FIFO throughput limits per group, shard limits in Kinesis.
- **Retention and replay:** queues keep messages days; streams keep them days to years.
- **Features:** DLQs, delays, scheduled messages, filtering, exactly-once options, Kafka-compatible APIs (MSK, Event Hubs, Confluent Cloud).
- **Cost:** per request, per throughput unit, or per hour; batch to save money.

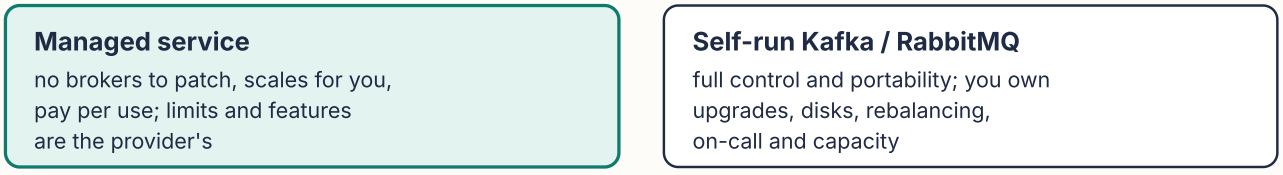


Figure 2. Managed services trade control for far less operational work.

WORKED EXAMPLE: AZURE SERVICE BUS SESSIONS FOR PER-CUSTOMER ORDERING

An order system publishes events for many customers to one queue, setting `SessionId` to the customer ID. Workers accept sessions: Service Bus locks each session to one worker at a time, so customer A's events are processed in order, while customers B and C are handled in parallel by other workers. Failed messages are retried up to a max delivery count, then moved to the built-in dead-letter sub-queue. The same pattern on AWS is SQS FIFO with `MessageGroupId`.

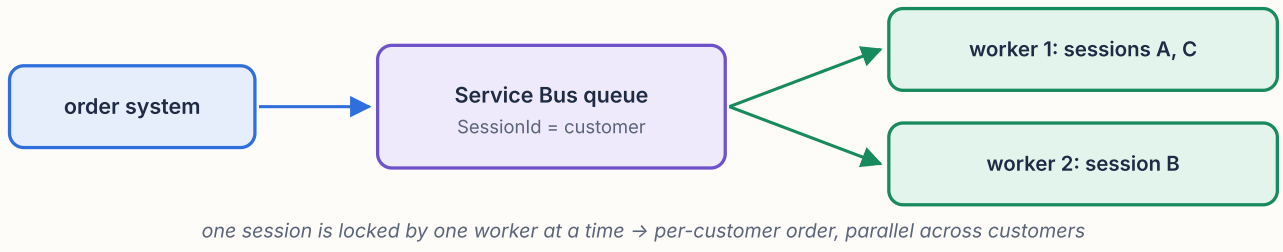


Figure 3. Sessions give ordering per key with parallelism across keys.

COMMON MISTAKES

- Running your own broker for modest traffic with no platform team.
- Ignoring service limits until production (message size, FIFO throughput).
- Polling SQS with short polls and paying for empty receives (use long polling).
- Assuming every 'pub/sub' service keeps messages for replay.

4 Key terms

SQS / SNS: AWS queue / AWS pub-sub notifications

Kinesis / Event Hubs: managed event streams

Long polling: wait for messages instead of returning empty

EventBridge / Event Grid: event routers with filtering rules

Service Bus session: ordered, locked group of messages

5 Interview questions

Q1. When would you choose a managed messaging service over self-hosted Kafka? BASIC

When you want no broker operations, have moderate or spiky load, and the service's features and limits fit; self-host when you need full control, portability or very high throughput economics.

Q2. How do you get ordered processing in SQS? BASIC

Use a FIFO queue with MessageGroupId set to the entity; messages in a group are delivered in order, one at a time, while different groups run in parallel.

Q3. Fan out OrderPlaced to three services on AWS. SCENARIO

Publish to an SNS topic (or EventBridge) and subscribe one SQS queue per service, each with its own DLQ.

Q4. You need Kafka semantics but no Kafka operations on Azure. SCENARIO

Use Event Hubs with its Kafka-compatible endpoint, or Confluent Cloud; keep clients unchanged.

Q5. What limits catch teams out with SQS? TRICKY

256 KB message size, FIFO throughput per message group, visibility timeout shorter than processing time causing duplicates, and 14-day maximum retention.

Q6. Is EventBridge a replacement for Kafka? TRICKY

No: it's a router with rules and integrations, not a high-throughput, replayable log with consumer groups, though it has archive and replay for some use cases.

Next: SD-I-020 · Pub/Sub and Fan-Out

Pub/Sub and Fan-Out

one event, many independent subscribers

Watch the 4-min video

In one line: publish/subscribe sends each message to every subscriber, so the producer publishes once and new consumers subscribe without changing it; give each subscriber its own queue or consumer group so a slow one can't hold back the others, and use filtering to keep subscribers to what they need.

1 Who needs to know?

An order is placed. The email service needs to know. So do analytics, inventory and, next month, a new fraud service. If the order service calls each one directly, it grows a new dependency for every consumer, and becomes as slow and fragile as the slowest one (lesson 23). With **publish/subscribe**, it publishes one event to a **topic**, and every **subscriber** gets its own copy.

2 Queues vs pub/sub

In a **queue**, consumers compete, and each message is handled once. In **pub/sub**, every subscription receives every message. Most real systems combine them: pub/sub to fan out to services, and within each service a queue or consumer group so its instances share that service's copy.

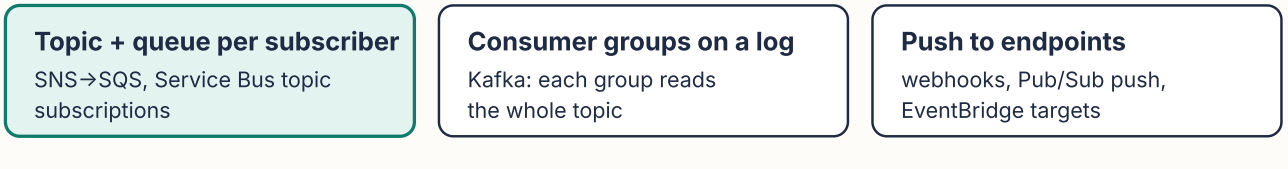


Figure 1. Three ways to implement fan-out.

WORKED EXAMPLE: AN SNS TOPIC FANNING OUT TO THREE SQS QUEUES

The order service publishes `OrderPlaced` to one SNS topic. Three SQS queues subscribe: email, analytics and fraud. Each queue has its own workers, retry settings and dead-letter queue. The fraud subscription uses an SNS **filter policy** so it receives only orders above 500. When the analytics workers fall behind during a sale, only the analytics queue grows; emails still go out on time. Adding a loyalty service next month means subscribing a fourth queue, with no change to the order service.

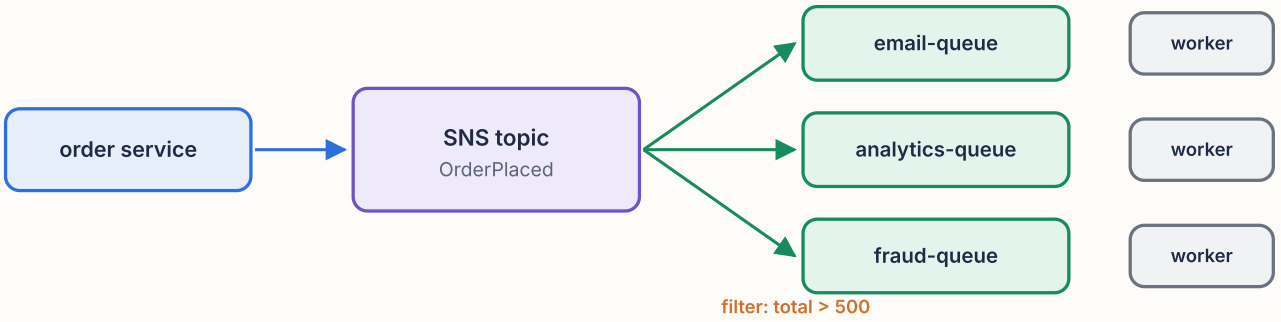


Figure 2. Topic → queue per subscriber: isolation, independent retries and filtering.

3 Things to watch



Figure 3. Pub/sub's common issues.

- Give each subscriber **its own buffer** (queue or consumer group); pushing directly to many HTTP endpoints couples their availability to yours.
- Delivery is usually at-least-once and, across subscribers, unordered: make each subscriber idempotent.
- With many subscribers, the event schema is a public contract (lesson 39).
- Fan-out can multiply load: one event per follower for a celebrity post is millions of writes; use batching or pull-on-read for very large fan-outs.

COMMON MISTAKES

- Multiple services sharing one queue and stealing each other's messages.
- Subscribers sharing one consumer group in Kafka by accident.
- No DLQ per subscription.
- Filtering in every consumer instead of at the subscription.

4 Key terms

Publish/subscribe: each message goes to every subscription

Subscription: a subscriber's own copy of the stream

Filter policy: subscription-side rule selecting messages

Topic: named channel publishers send to

Fan-out: one message delivered to many consumers

5 Interview questions

Q1. What's the difference between a queue and pub/sub? BASIC

In a queue, consumers compete and each message is processed once; in pub/sub, every subscriber receives its own copy of every message.

Q2. Why put a queue behind each subscription? BASIC

So each subscriber consumes at its own pace with its own retries and DLQ, and a slow or failing one doesn't affect the others or the publisher.

Q3. Design notifications when an order is placed, for email, SMS and analytics. SCENARIO

Publish OrderPlaced to a topic; subscribe one queue per channel with its own workers, retries and DLQ; filter where useful; make each consumer idempotent.

Q4. How do you fan out on Kafka? SCENARIO

Publish once to a topic; each service uses its own consumer group, so each receives every record, and instances inside a group share partitions.

Q5. What's the risk of very large fan-out? TRICKY

Write amplification and load spikes, as with timelines for accounts with millions of followers; mitigate with batching, rate limits, or computing on read for the largest fan-outs.

Q6. Does pub/sub guarantee order across subscribers? TRICKY

No. Each subscription processes at its own pace, so different subscribers can see events at different times, and order within each depends on the broker's per-key guarantees.

Next: Phase 5 · SD-K-001 · Availability Calculations backendroadmap.com